



De La Salle University

TARA CUDA: A Hybrid SIMT Implementation of
Zhang's Six-Frame Algorithm using CUDA

A Thesis Presented to
the Faculty of the College of Computer Studies
De La Salle University

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Computer Science
with Specialization in Computer Systems Engineering

by
Arceta, Althea Zyrie M.
Mendoza, Antonio Gabriel N.
Tan, Jose Tristan T.

Mr. Roger Luis T. Uy
Faculty Adviser

August 9, 2025



Abstract

DNA-protein alignment is a type of sequence alignment used to identify similarities between DNA and protein sequences, which has a wide range of applications in bioinformatics. Zhang's six-frame alignment algorithm is one such method which translates the DNA sequence across all six possible reading frames to specifically account for frameshift errors and variations. Implementing Zhang's six-frame alignment algorithm sequentially can be computationally expensive, particularly when dealing with large datasets. To address this issue, parallel computing execution models such as SIMD and SIMT can be used to enhance the performance of this algorithm. While it has been implemented with the SIMD paradigm, there has yet to be an SIMT approach. Hence, this study explores another approach that uses NVIDIA's CUDA programming model. The developed parallel implementation was evaluated against the sequential version and the existing SIMD implementation using the *Drosophila melanogaster* (fruit fly) and *Arabidopsis thaliana* (mouse-ear cress) datasets. The implementation showed speed-up results averaging 3.51x and a maximum of 6.03x for the fruit fly dataset, while an average of 3.44x and a maximum of 6.44x was obtained on the mouse-ear cress dataset, when compared to the sequential version. When compared against the SIMD implementation, the CUDA implementation achieved speedups averaging 3.14x, demonstrating the capabilities of CUDA as another viable approach to parallelizing.

Keywords—Zhang's Six-frame alignment, DNA-Protein alignment, SIMT, CUDA, Parallel Computing, High Performance Computing



Table of Contents

1. Introduction	1
1.1 Overview	1
1.2 Problem Statement	3
1.3 Research Objectives	3
1.3.1 General Objective	3
1.3.2 Specific Objectives	3
1.4 Project Scope and Limitations	4
1.5 Resources	4
1.5.1 Hardware	4
1.5.2 Software	5
1.6 Test and Verification Method	5
1.6.1 Correctness Test	5
1.6.2 Execution Time Test	5
1.6.3 Scalability Test	6
1.6.4 Cross-Platform Performance Test	6
1.7 Significance of the Study	6
1.8 Research Methodology	6
1.9 Research Project Schedule	8
2. Review of Related References	9
2.1 Review of Related Works	9
2.1.1 Local Alignment Methods	12
2.1.2 Parallel Implementation	18
2.2 Review of Related Literature - DNA-Protein Alignment	21
3. Theoretical Framework	28
3.1 Modified Zhang's Three-Frame Algorithm	28
3.2 CUDA and SIMT Paradigm	29
3.3 Wavefront Paradigm	31
3.4 BLOSUM62 Matrix	32
4. System Design	34
4.1 Zhang's Six-Frame Algorithm	35
4.2 Dataset	37
4.3 Preparation and Preprocessing	38



4.3.1 Data Initialization	38
4.3.2 Translation and BLOSUM62	39
4.3.3 Grid and Block Configuration	39
4.4 Matrix Parallelization	39
4.5 Traceback	41
5. System Implementation	43
5.1 Data Gathering & Dataset	44
5.2 Input Processing	45
5.3 Translation & BLOSUM62	48
5.4 Matrix Initialization	50
5.5 Sequential Implementation	52
5.6 CUDA Implementation	54
5.6.1 CUDA Error Handling	55
5.6.2 Grid and Block Configuration	55
5.6.3 Memory Management	56
5.6.4 Matrix Parallelization	57
5.6.4.1 Host-Side Loop	59
5.6.4.2 CUDA Kernels	59
5.7 Traceback	62
6. Results and Analysis	66
6.1 Correctness Test	67
6.1.1 Manual Checking	69
6.1.2 External Cross-Checking	81
6.2 Runtime Test	86
6.2.1 SIMT vs Sequential	87
6.2.1.1 Stress Test	93
6.2.2 SIMT vs SIMD	99
6.3 Scalability Test	104
6.3.1 Input Scalability	104
6.3.2 Compiler Optimization	108
6.4 Cross-Platform Performance Test	112
7. Conclusion & Recommendations	116
7.1 Conclusion	116



7.2 Recommendations	116
Appendix A. Bibliography	118

List of Tables

Table 2-1. Summary of Related Works.....	9
Table 2-2. Review of Related Literature - DNA-Protein Alignment.....	21
Table 6-1: Traceback Summary (match case).....	72
Table 6-2: Traceback Summary (With Insertion).....	74
Table 6-3: Traceback Summary (With Deletion).....	77
Table 6-4: Traceback Summary (With Insertion and Deletion).....	80
Table 6-5: Average Runtime Contribution of Each Phase For Single Read.....	92
Table 6-6: Summary Statistics for Sequential and SIMT Implementation (for Fruit Fly Dataset).....	96
Table 6-7: Total Runtime and Speedup for Sequential and SIMT Implementation (for Fruit Fly Dataset).....	96
Table 6-8: Summary Statistics for SIMT and Sequential Implementation (for Mouse Ear Cress Dataset).....	98
Table 6-9: Total Runtime and Speedup for Sequential and SIMT Implementation (for Mouse Ear Cress Dataset).....	99
Table 6-10: Summary Statistics for SIMD and SIMT Implementation.....	101
Table 6-11: Summary of Average Speed-up and Runtime per Implementation based on Different CPU.....	103
Table 6-12: Summary of Total Runtime Contribution of Each Phase (Sequential).....	107
Table 6-13: Summary of Total Runtime Contribution of Each Phase (Parallel).....	108
Table 6-14: Average Runtime for Sequential Implementation on Different Compiler Optimization Levels.....	110
Table 6-15: Average Runtime for Parallel Implementation on Different Compiler Optimization Levels.....	111
Table 6-16: Speedup Comparison of CUDA vs Sequential Across GPU Platforms.....	114
Table 6-17: Speedup Comparison of CUDA Across Different GPUs.....	114
Table 6-18: Runtime Comparison Across GPU Platforms.....	115



List of Figures

Figure 1-1 Methodology.....	6
Figure 2-1 Hardware architecture for the Gapped BLAST algorithm with the two-hit method	14
Figure 2-2 Execution flow of GPU-BLAST. The blue blocks are executed on the CPU, and the orange ones on the GPU.....	15
Figure 2-3 A schematic overview of the logic of the straightforward approach to parallelizing FASTA.....	16
Figure 2-4 Row-Wise Technique Table.....	19
Figure 2-5 Anti-Diagonal Technique.....	19
Figure 2-6 Parallelization of Needle program that run based on the Needleman-Wunsch algorithm.....	20
Figure 2-7. System Overview of Melanoma Detection System.....	25
Figure 2-8. System Overview of PPS Analysis.....	25
Figure 2-9. Quasi-mapping Algorithm.....	26
Figure 3-1 Three-Frame DNA-Protein Alignment. Asterisks mean frameshift.....	28
Figure 3-2 Sample CUDA code (ElementwiseOp Function).....	30
Figure 3-3 Sample CUDA code (CUDA's unified memory and prefetching functions).....	30
Figure 3-5: Anti-Diagonal Technique.....	31
Figure 3-6: BLOSUM62 Matrix.....	32
Figure 4: System Overview.....	34
Figure 4-1 Six-Frame DNA-Protein Alignment.....	35
Figure 4-2: Concept of Parallelization, Cycle 1.....	40
Figure 4-3: Concept of Parallelization, Cycle 2.....	40
Figure 4-4: Concept of Parallelization, Cycle 3.....	40
Figure 4-5: Concept of Parallelization, Cycle 4.....	40
Figure 4-6: Concept of Parallelization, Cycle 8.....	40
Figure 4-7: Concept of Parallelization, Cycle 9.....	41
Figure 4-8: Concept of Parallelization, Cycle 10.....	41
Figure 4-9: Finding the Max Score from three matrices.....	42
Figure 4-10: Traceback Process Cycle 1-2.....	42
Figure 4-11: Traceback Process Cycle 3-7. Moved to the other matrix indicating a shift.....	42
Figure 4-12: Traceback Process Cycle END.....	42



Figure 5-1: Complete System Diagram of TARA CUDA.....	43
Figure 5-2: Data Preprocessing: Validation and Completeness Check in Fruit Fly Dataset...	44
Figure 5-3: Data Preprocessing: Validation and Completeness Check in Mouse Ear Cress Dataset.....	45
Figure 5-4: System Prompt for DNA Sequence and Proteome Input.....	46
Figure 5-5: Code Snippet of FASTA Sequence Read Function.....	46
Figure 5-6: Code Snippet: FASTA Header (ID) Extraction Function.....	47
Figure 5-7: System Prompt for Algorithm Implementation Type.....	47
Figure 5-8: System Prompts for Three-Frame or Six-Frame Selection.....	47
Figure 5-9: Code Snippet of a Series of if-else Statements for Codon-to-Protein Translation... 49	
Figure 5-10: Code Snippet of Array Lookup of BLOSUM62.....	50
Figure 5-11: Code Snippet of Matrix Initialization Process.....	52
Figure 5-12: Code Snippet of Sequential Scoring Process.....	54
Figure 5-13: Code Snippet of CUDA Error Checking.....	55
Figure 5-14: Code Snippet of Dynamic Grid Size Allocation.....	56
Figure 5-15: Code Snippet of Input Sequence Initialization & Host-to-Device Memory Transfer.....	56
Figure 5-16: Code Snippet of CUDA Memory Allocation for Alignment Matrices.....	56
Figure 5-17: Code Snippet of Device Matrix Initialization.....	57
Figure 5-18: Sample Visualization of Submatrix Division.....	58
Figure 5-19: Sample Visualization of Macro-Level Matrix Anti-Diagonals.....	58
Figure 5-20: Sample Visualization of Submatrix-Level Anti-Diagonals.....	59
Figure 5-21: Code Snippet of Host Side Loop for Macro Level Anti-diagonals.....	59
Figure 5-22: Sample Visualization of Grid-Stride Anti-Diagonal Computation in CUDA Kernel.....	60
Figure 5-23: Code Snippet of Device Matrix Initialization.....	62
Figure 5-24: Visualization of the Traceback Traversal Logic.....	63
Figure 5-25: Code Snippet of the Traceback Traversal Logic.....	64
Figure 5-26: Code Snippet of Traceback Process.....	65
Figure 6-1: DNA Substring (read30) - Match Case.....	67
Figure 6-2: DNA Substring - With Insertion.....	68
Figure 6-3: DNA Substring - With Deletion.....	68
Figure 6-4: DNA Substring - With Insertion and Deletion.....	69



Figure 6-5: Excel Snapshot of Manual Computation of the I matrix (match case).....	69
Figure 6-6: Excel Snapshot of Manual Computation of the D matrix (match case).....	70
Figure 6-7: Excel Snapshot of Manual Computation of the C matrix (match case).....	70
Figure 6-8: Excel Snapshot of Program Output vs. Manual Calculation Comparison (match case).....	71
Figure 6-9: Excel Snapshot of Manual Traceback Process (match case).....	71
Figure 6-10: Excel Snapshot of Manual Computation of the I matrix (With Insertion).....	72
Figure 6-11: Excel Snapshot of Manual Computation of the D matrix (With Insertion).....	73
Figure 6-12: Excel Snapshot of Manual Computation of the C matrix (With Insertion).....	73
Figure 6-13: Excel Snapshot of Program Output vs. Manual Calculation Comparison (With Insertion).....	74
Figure 6-14: Excel Snapshot of Manual Traceback Process (With Insertion).....	74
Figure 6-15: Excel Snapshot of Manual Computation of the I matrix (With Deletion).....	75
Figure 6-16: Excel Snapshot of Manual Computation of the D matrix (With Deletion).....	76
Figure 6-17: Excel Snapshot of Manual Computation of the C matrix (With Deletion).....	76
Figure 6-18: Excel Snapshot of Program Output vs. Manual Calculation Comparison (With Deletion).....	76
Figure 6-19: Excel Snapshot of Manual Traceback Process (With Deletion).....	77
Figure 6-20: Excel Snapshot of Manual Computation of the I matrix (With Insertion and Deletion).....	77
Figure 6-21: Excel Snapshot of Manual Computation of the D matrix (With Insertion and Deletion).....	78
Figure 6-22: Excel Snapshot of Manual Computation of the C matrix (With Insertion and Deletion).....	79
Figure 6-23: Excel Snapshot of Program Output vs. Manual Calculation Comparison (With Insertion and Deletion).....	79
Figure 6-24: Excel Snapshot of Manual Traceback Process (With Insertion and Deletion)...	80
Figure 6-25: Correctness Verification of Alignment Results (FFread10).....	81
Figure 6-26: BLASTX Results for Perfect Match.....	82
Figure 6-27: BLASTX Results for Alignment (Perfect Match).....	82
Figure 6-28: BLASTX Results for With Insertion.....	83
Figure 6-29: BLASTX Results for Alignment (With Insertion).....	83
Figure 6-30: BLASTX Results for With Deletion.....	84
Figure 6-31: BLASTX Results for Alignment (With Deletion).....	84
Figure 6-32: BLASTX Results for With Insertion and Deletion.....	85



Figure 6-33: BLASTX Results for Alignment (With Insertion and Deletion).....	85
Figure 6-34: BLASTX Results for Complete Mismatch.....	85
Figure 6-35: Excel Snapshot of SIMD and CUDA Score Results Comparison.....	86
Figure 6-36: Screenshot for Sequential Runtime (START).....	87
Figure 6-37: Screenshot for Sequential Runtime (MIDDLE).....	88
Figure 6-38: Screenshot for Sequential Runtime (END).....	88
Figure 6-39: Screenshot for CUDA Runtime (START) - 1st Loop.....	88
Figure 6-40: Screenshot for CUDA Runtime (MIDDLE) - 1st Loop.....	89
Figure 6-41: Screenshot for CUDA Runtime (END) - 1st Loop.....	89
Figure 6-42: Screenshot for CUDA Runtime (START) - 2nd Loop.....	90
Figure 6-43: Screenshot for CUDA Runtime (MIDDLE) - 2nd Loop.....	90
Figure 6-44: Screenshot for CUDA Runtime (END) - 2nd Loop.....	91
Figure 6-45: Visualization of Average Runtime Contribution of Each Stage For Single Read... 92	
Figure 6-46: Runtime vs. Sequence Length for Sequential and Parallel Implementations (for Fruit Fly Dataset).....	94
Figure 6-47: Speedup vs. Sequence Length for Sequential and Parallel Implementations (for Fruit Fly Dataset).....	95
Figure 6-48: Runtime vs. Sequence Length for Sequential and Parallel Implementations (for Mouse Ear Cress Dataset).....	97
Figure 6-49: Speedup vs. Sequence Length for Sequential and Parallel Implementations (for Mouse Ear Cress Dataset).....	98
Figure 6-50: TARA CUDA Parallel Implementation vs Sequential Implementation.....	100
Figure 6-51: FICS Parallel (SIMD) implementation vs Sequential Implementation.....	101
Figure 6-52: TARA CUDA Parallel Implementation vs FICS Parallel (SIMD) Implementation.....	101
Figure 6-53: Sequential Runtime Comparison (Ryzen 7 vs Ryzen 5).....	102
Figure 6-54: SIMD Runtime Comparison (Ryzen 7 vs Ryzen 5).....	103
Figure 6-55: Scalability of Fruit Fly Dataset.....	104
Figure 6-56: Scalability of Mouse Ear Cress Dataset.....	105
Figure 6-57: Scalability of the Runtime Contribution of Each Phase (Sequential).....	106
Figure 6-58: Scalability of the Runtime Contribution of Each Phase (Parallel).....	106
Figure 6-59: Sample Command Line and Additional Options.....	110
Figure 6-60: Average Runtime Comparison of Sequential Implementation Across Compiler Optimization Levels.....	111



Figure 6-61: Average Runtime Comparison of Parallel Implementation Across Compiler Optimization Levels.....	112
Figure 6-62: Parallel Implementation Runtime Across GPU Platforms.....	113
Figure 6-63: Sequential Implementation Runtime Across GPU Platforms.....	114



List of Equations

Equation 2-1.....	25
Equation 2-2.....	25
Equation 2-3.....	25
Equation 2-4.....	26
Equation 2-5.....	26
Equation 2-6.....	26
Equation 2-7. Initialization for traceback matrices of matrix I and D, respectively.....	26
Equation 2-8. Initialization for traceback matrices of matrix C.....	27
Equation 2-9 Formula for filling Traceback Matrices.....	27
Equation 2-10.....	29
Equation 2-11.....	29
Equation 2-12.....	29
Equation 2-14.....	29
Equation 2-15. Scoring Scheme.....	32
Equation 2-16. Alignment Scheme.....	33
Equation 2-17. Score and Model Parameters.....	33
Equation 3-1.....	38
Equation 4-1 Matrix Initialization Formula.....	45
Equation 4-2.....	45
Equation 4-3.....	46



1. Introduction

1.1 Overview

Sequence alignment is the process of arranging DNA, RNA, or protein sequences to detect regions of similarity. Identifying these shared regions is crucial to understanding the functional, structural, and evolutionary connections between them [1]. Consequently, given its role as the foundational first stage in genomic workflows, achieving faster and more efficient sequence alignment is highly crucial. As such, acceleration benefits researchers, enabling them to proceed more rapidly with downstream analysis. Examples like Smith-Waterman [2], BLAST [3], and BWA [4] are tools developed for sequence alignment. These techniques align sequencing reads with reference sequences, ensuring mapping even in the presence of insertions, deletions, and mismatches. For example, the Smith-Waterman algorithm is a dynamic programming algorithm used for local sequence alignment, which detects regions of high similarity between pairs of sequences. BLAST, on the other hand, detects regions of similarity, which makes it possible to compare nucleotide or protein sequences to sequence databases and determines their statistical significance, making it crucial for gene discovery, functional annotation, and identifying evolutionary relationships across diverse organisms. Lastly, the Burrows-Wheeler Aligner (BWA) uses the Burrow-Wheeler transform and FM index to provide quick alignment, which is crucial given the vast amount of sequences.

DNA-protein alignment is a type of sequence alignment that compares DNA read sequences to a reference proteome based on the codon's position and strand, while accounting for frameshift errors. Its application ranges from metagenomics to gene analysis [5] [6] [7] [8] [9], and techniques like LAST [10], BLASTX [11], and DIAMOND [12] aid in the alignment process by enabling more efficient sequence comparisons. Another DNA-protein alignment is Zhang's three-frame alignment algorithm [13]. This algorithm is appropriate for comparing DNA sequences that suffer from sequencing errors with a reference protein sequence. The three frames are obtained by translating the DNA sequence from the different starting codon locations. For example, in the sequence ATGCCTAGGT, Frame 1 would start from ATG (*methionine*) then CCT (*proline*), Frame 2 from TGC (*cysteine*) to CTA (*leucine*), and Frame 3 would have GCC (*alanine*) then TAG (stop codon), and so on. Using three frames allows the algorithm to compare all possible protein translations from the DNA sequence to the reference proteome. Otherwise, if only one frame is used, frameshifts are impossible to detect because insertions and deletions in the DNA sequence are ignored.



DNA is typically composed of two complementary strands, the 5'-end to 3'-end and 3'-end to 5'-end directional strands, that form a double helix structure. Zhang's three-frame alignment algorithm, however, only compares the 5' to 3' direction strand of the DNA sequence to the protein sequence. Hence, it is important to test both orientations to test all possible protein combinations. Zhang's six-frame alignment algorithm [14] utilizes 3 more frames that are obtained from the reverse complement (3'-end to 5'-end direction strand) of the DNA sequence for a total of 6 frames. It computes the alignment of the six frames by using the three-frame alignment algorithm twice, accounting for both directional strands.

While Zhang's six-frame alignment algorithm can be considered as a well-established method, its sequential implementation can be computationally expensive [16], especially when dealing with large datasets. However, recent advancements of next-generation sequencing technologies have accelerated RNA-seq development, opening doors to exciting new techniques [15]. Thus, a need for efficient solutions to sequential implementations arises to keep pace with the growth of data used in the algorithms. To address this, researchers have explored alternative approaches and parallel computing techniques. A study by Santiago et al [8] introduces an alternative approach that quickly aligns RNA-seq reads to protein sequences without full alignment, called quasi-mapping. By reducing the computational complexity, quasi-mapping can significantly speed up the alignment process. Another alternative is to utilize parallel computing techniques to enhance the execution of these algorithms, leading to faster alignments and the capability to process larger datasets. SIMT (Single Instruction, Multiple Threads) and SIMD (Single Instruction, Multiple Data) are two parallel computing paradigms that enable effective parallel processing [17].

Single instruction, multiple data (SIMD) achieves parallelism by allowing a single instruction to operate on multiple data elements simultaneously through vectorization by utilizing multiple execution units. It provides a range of feasible approaches for faster algorithm executions and is employed in most applications requiring large data processing [18]. A SIMD implementation of Zhang's six-frame algorithm [14] using AVX2 was implemented using a staircase-like method to partially circumvent the data dependencies inherent in the three-frame alignment algorithm [13], allowing the algorithm to be implemented in the SIMD paradigm, resulting in a threefold increase in execution speed over the sequential implementation.

On the other hand, Graphics Processing Units (GPUs) use the Single Instruction, Multiple Threads (SIMT) execution model for parallel computing. This method allows a large number of distinct threads to execute the same instruction concurrently while operating on different data [19]. This makes the utilization of GPUs a viable approach for computational workloads that can be divided across multiple processing units.



Existing frameworks such as NVIDIA's Compute Unified Device Architecture (CUDA) and OpenCL enable programmers to leverage the SIMT capabilities of GPUs. CUDA, specifically designed for NVIDIA GPUs, offers a full programming environment for General Purpose computing on a Graphics Processing Unit (GPGPU) processing. It frequently performs best due to its close connection with NVIDIA hardware [20]. On the other hand, OpenCL takes a more general-purpose approach, allowing code to be portable across multiple SIMT-compatible hardware platforms, including AMD and Intel GPUs. However, when targeting specific hardware, this portability may sacrifice some speed compared to CUDA.

As of the writing of this study, there are currently no direct implementations of a frame alignment algorithm in SIMT. With the implementation of the SIMD AVX2, the researchers would like to explore SIMT as a promising option. Thus, the goal of this study is to implement Zhang's Six-Frame Alignment Algorithm with the SIMT paradigm. Since obtaining a maximum speedup is the priority for this study, we will utilize CUDA instead of OpenCL for our SIMT implementation.

1.2 Problem Statement

Sequence alignment algorithms typically have high computational costs due to their inherent data dependencies. This results in high execution times, especially when using sequential implementations to process large datasets and many sequence reads. NVIDIA's CUDA provides a potential solution by utilizing the SIMT execution model through the use of GPUs to process data in parallel for general-purpose computations.

1.3 Research Objectives

1.3.1 General Objective

This research aims to develop an approach to implementing Zhang's Six-Frame Alignment Algorithm in parallel, specifically using the CUDA SIMT execution model.

1.3.2 Specific Objectives

- To develop a sequential implementation of Zhang's Six-Frame Alignment Algorithm.
- To create an implementation scheme for Zhang's Six-Frame Alignment Algorithm to better fit the parallel framework of the SIMT model.



- To implement Zhang's Six-Frame Alignment Algorithm with the SIMT execution model using CUDA.
- To evaluate the developed parallel implementation by comparing and contrasting its performance to the sequential version and the existing SIMD implementation.

1.4 Project Scope and Limitations

This research mainly focuses on DNA-protein sequence alignment. The algorithm to be implemented is Zhang's Six-Frame Alignment Algorithm. While other algorithms are available, the study will only focus on the aforementioned algorithm. The goal of this study is to implement a parallelized version of Zhang's Six-Frame Algorithm. Initially, a sequential implementation of the algorithm will be implemented, and this implementation will be used as a point of reference for subsequent evaluation.

The aforementioned algorithm will be implemented using CUDA's SIMT execution model. To do this, a new implementation scheme will be created to meet the SIMT model's data-parallel structure. While other data parallelization methods exist, this study will focus only on the SIMT execution model using CUDA.

While this study presents promising results, it is important to acknowledge its limitations. The proposed parallel implementation of this research will focus on parallelizing the scoring part of Zhang's six-frame algorithm only, as this plays a large role in the alignment process. Hence, improvements to other parts of the algorithm, such as traceback, are not within the scope.

Finally, an evaluation of the proposed parallel implementation will be conducted. First, the proposed implementation will undergo a correctness test by using a synthetic input for the parallel algorithm and comparing its results with those produced by the reference sequential version. In addition to correctness, the proposed implementation will also undergo execution time tests. This is done by measuring the runtime of the proposed implementation and comparing it to the sequential and existing SIMD implementations. Lastly, the proposed implementation will undergo a scalability test by testing varying input proteome sizes and compiler optimizations. All in all, by analyzing the results, this research evaluates the performance benefits of the proposed parallelization approach.

1.5 Resources

1.5.1 Hardware

- NVIDIA CUDA-Compatible GPU



- An NVIDIA CUDA-compatible GPU is required to create CUDA-accelerated programs.

1.5.2 Software

- Microsoft Visual Studio 2022
 - Microsoft Visual Studio 2022 is a suitable environment for CUDA programming.
- CUDA Developer Toolkit
 - The NVIDIA CUDA Toolkit provides a development environment to enable the development of CUDA-accelerated programs.
- CCSCloud (Jupyter Notebook)
 - CCSCloud is a hosted Jupyter Notebook that provides an environment to write programs such as Python scripts as well as CUDA programs.

1.6 Test and Verification Method

1.6.1 Correctness Test

The objective of this test is to verify the correctness of the implementation. This will be accomplished by using synthetic data on the developed implementation and comparing its output to the output of the reference sequential implementation. To verify correctness, the test will involve two aligned datasets: an array of DNA sequences and an equivalent Protein sequence dataset. The datasets are such that each element of the DNA dataset is the translation of the corresponding element of the Protein dataset. That is, the i^{th} DNA sequence corresponds to the i^{th} Protein sequence. An extracted substring from the DNA sequence will then be processed through the implementation. The extracted substring will have four versions to replicate different alignment scenarios: one with perfect alignment, another simulating an insertion (by randomly adding an additional nucleotide to the extracted substring), a third simulating a deletion (by removing a nucleotide from the extracted substring), and a fourth containing both an insertion and a deletion. If the output index of the Protein sequence matches accurately with the input DNA index, the output is considered correct.

1.6.2 Execution Time Test

The objective of this test is to evaluate the runtime performance of the parallel implementation. This will be done by recording the time elapsed before and after the execution of the kernel and computing the time taken by the kernel. This performance is then compared to the performance of the sequential implementation, as well as the existing SIMD implementation. The speedup of the implementation will also be taken.



This is done by dividing the runtime of the sequential implementation by the runtime of the parallel implementation.

1.6.3 Scalability Test

The objective of this test is to check the runtime scaling of the implementations as the input data size increases. This will be achieved by testing the different implementations on increasing input proteome sizes, and determining the correlation between the growth of input size with runtime. In addition to this, the impact of different compiler optimization levels will also be tested.

1.6.4 Cross-Platform Performance Test

The objective of this test is to evaluate the performance of the CUDA parallel implementation of the algorithm through its execution time across various hardware configurations. Testing will be conducted on a range of devices equipped with NVIDIA GPUs.

1.7 Significance of the Study

The findings of this study will contribute to the knowledge regarding parallelization approaches using SIMT and CUDA in the field of Computer Science. Furthermore, the findings of this study will provide a novel approach implementation of sequence alignment algorithms. Lastly, the findings of this study can aid processes such as RNA sequencing by enabling the identification of aligned sequences and mutations much faster.

1.8 Research Methodology

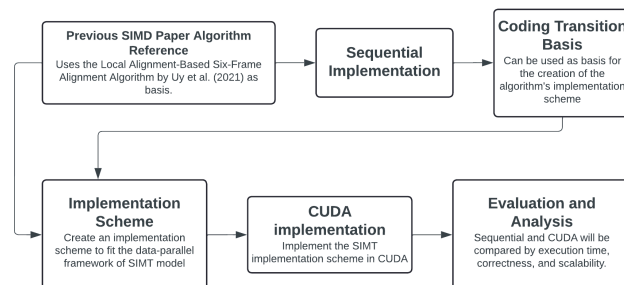


Figure 1-1 Methodology



The initial step for this study is the sequential implementation of Zhang's Six Frame Alignment algorithm. A sequential version of the algorithm needs to be implemented to have a clear understanding of the algorithm. Furthermore, this implementation will then be used to evaluate and analyze the proposed parallel implementation of the algorithm. This implementation will employ the modified algorithm for Zhang's Six Frame Alignment proposed in the study [14] entitled SIMD Implementation of Local Alignment-Based Six-Frame Alignment Algorithm.

After having an in-depth understanding of the algorithm, the researchers will proceed to develop the implementation scheme. So far, the matrix computation in the study of [21] proposed a staircase-like approach to parallelize the computation of values on the matrix. However, the matrix computation may not necessarily fit the SIMT model of the current parallelization approach, so the researchers will create an implementation scheme that will fit the data-parallel framework of the SIMT.

The proposed implementation scheme will be implemented using CUDA. This programming language supports the parallelization of the program by utilizing an NVIDIA GPU. CUDA uses threads that enable the program's parallel computation capacity. With that, the researchers aim to implement the modified Zhang's Six Frame Alignment Algorithm to speed up its computation capability using CUDA's GPU utilization.

After implementing the algorithm, the researchers will evaluate the CUDA version of the algorithm. If the implementation scheme does not suit the CUDA programming model and the results from the program are not ideal, then remodification of the scheme will occur. If the code passes all the conditions for the evaluation, the researchers will proceed to assessment and analysis compared to other implementations.

Evaluation and analysis will be conducted after implementing the sequential and CUDA versions of the proposed algorithm. The dataset to be used in this research is *Drosophila melanogaster* (fruit fly) and *Arabidopsis thaliana* (mouse-ear cress) for both input DNA query sequences and the input reference proteome. The researchers intend to use the results that the study gathered for the comparison of the sequential and CUDA implementations of the algorithm. Thus, the execution time, correctness, scalability, and cross-platform performance of the programs will be assessed.



1.9 Research Project Schedule

Activity	May '24	Jun '24	Jul '24	Aug '24
Topic & Adviser Search				
Related Literature Search				
Proposal Writing				
Consultation				
Sequential Implementation				

Activity	Sept '24	Oct '24	Nov '24	Dec '25
Parallel Implementation				
Sequential Implementation				
Testing/Debugging				
Thesis Writing				
Consultation				

Activity	Jan '25	Feb '25	Mar '25	Apr '25
Testing/Debugging				
Parallel Implementation				
Analysis				
Thesis Writing				
System Evaluation				
Consultation				

Activity	May '25	Jun '25	Jul '25	Aug '25
System Evaluation				
Testing				
Debugging				
Analysis				
Thesis Writing				
Consultation				



2. Review of Related References

2.1 Review of Related Works

Table 2-1. Summary of Related Works

Author/s	Title	Description	Approach	Contribution
Lim S.V., Lim, S.E., Ting, C., Uy, R., & Wong A. (2021)	SIMD implementation of Zhang's Three-Frame Alignment Algorithm	Presents a SIMD implementation of Zhang's Three-Frame Alignment algorithm to enhance performance.	SIMD using AVX2 instructions	• Achieves a speedup ranging from 1.81x to 6.1x
Lim S.V., Lim, S.E., Ting, C., Uy, R., & Wong A. (2021)	SIMD Implementation of Local Alignment-Based Six-Frame Alignment Algorithm	Develops a local alignment algorithm that computes the optimal alignment of six frames based on DNA sequence	SIMD using AVX2 instructions	• Takes into account the double strand of the DNA • Made Zhang's Frame alignment more scalable.
Khajeh-Saeed, A., Poole, S., & Perot, J. B. (2010)	Acceleration of the Smith-Waterman algorithm using single and multiple graphics processors	Explores the parallelization of the Smith-Waterman algorithm using GPU, demonstrating significant performance improvements.	CUDA	• A single GPU is up to 45 times faster than a CPU implementation
T. R. P. Siriwardena and	An efficient implementation of Smith	Focuses on a CUDA-based implementation	CUDA	• Achieves over 3.5 times higher per-core



D. N. Ranasinghe	Waterman algorithm on GPU using CUDA, for massively parallel scanning of sequence databases	n of the Smith-Waterman algorithm, optimizing performance for sequence database searches.		performance compared to previous GPU implementations of the Smith-Waterman algorithm.
Guo, X., Wang, H., & Devabhaktuni, V. (2012)	A systolic Array-Based FPGA parallel architecture for the BLAST algorithm.	Proposes an FPGA-based architecture for the BLAST algorithm, allowing for efficient sequence alignment.	FPGA	The proposed GPU-BLAST algorithm was able to speed up 3 times to 4 times the NCBI-BLAST algorithm using parallelization techniques
Vouzis, P. D., & Sahinidis, N. V. (2010)	GPU-BLAST: using graphics processors to accelerate protein sequence alignment.	Introduces GPU-BLAST, an accelerated version of NCBI-BLAST for protein sequence alignment.	GPU Implementation	Reports a speedup of 3 to 4 times compared to the sequential NCBI-BLAST algorithm.
Miller, P. L., Nadkarni, P. M., & Carriero, N. M. (1991).	Parallel computation and FASTA: confronting the problem of parallel database search for a fast sequence comparison algorithm	Discusses parallelizing the FASTA algorithm to reduce computation time and address bottlenecks in database searches.	Linda programming model	Speedup of FASA algorithm using FPGA hardware and parallelization techniques with threading
Zhang, Z.,	Aligning a DNA sequence	Provides foundational	Dynamic Programming	Establishes early



Pearson, W. R., & Miller, W. (1997, January)	with a protein sequence	work on aligning DNA sequences to protein sequences, crucial for understanding genetic functions.		methodologies for sequence alignment
Lee, Y. S., Kim, Y. S., & Uy, R. L. (2020).	Serial and parallel implementation of Needleman-Wunsch algorithm.	Explores both serial and parallel implementations of the Needleman-Wunsch algorithm, addressing performance and scalability.	OpenCL	● Achieves a speedup up to three times faster than the serial implementation,
Muhamad, F. N., Ahmad, R. B., Asi, S. M., & Murad, M. N. (2018, June)	Performance Analysis of Needleman-Wunsch algorithm (global) and Smith-Waterman algorithm (local) in reducing search space and time for DNA sequence alignment.	Analyzes the performance of both global and local alignment algorithms to assess their efficiency in DNA sequence alignment.	OpenMp	● Achieves a 60% reduction in execution time for the Needleman-Wunsch algorithm



2.1.1 Local Alignment Methods

In this section, the researchers summarize and review related works on sequence alignment using local alignment methods. Three methods in the regions of the highest similarity may only be part of the sequences. These methods are Zhang's Six Frame Alignment, the Smith-Waterman Algorithm, FASTA, and BLAST.

Six Frame Alignment Algorithm. This algorithm takes into consideration the two strands of the DNA sequence. The methodology [14] computes the optimal alignment of the six-frame by executing the three-frame algorithm twice, where the first alignment is for the original DNA sequence and the second is for the reverse complement. The highest score will be taken from the two C matrices. The optimal alignment of the DNA sequence is obtained by performing a traceback starting from the index of the highest score in the modified three-frame alignment. Traceback matrices are filled for each of the C, I, and D matrices using the recurrence formula in Equation 2-7 and Equation 2-8, which are all filled simultaneously. Then, each mattress will be filled using the recurrence in equation 2-9. The optimal frame alignment is determined from the path taken from matrix C from the traceback of local alignment, which will start on the index where the highest score is located and will end whenever the traceback reaches an index where the value of the C matrix is 0. The Six Frame approach has proven more scalable than the three-frame alignment approach since it is more time-efficient. However, a critical factor that can affect the execution time of the Six-Frame algorithm approach is the sequential implementation for the traceback implementation of finding the optimal alignment.

Smith-Watermann Algorithm. This algorithm makes use of a dynamic programming approach as a solution to finding the optimal alignment of two sequences. The study conducted by [22] discusses the acceleration of the Smith-Watermann algorithm using single and multiple graphics processors. The study parallelizes their proposed optimized version of the algorithm as shown in Equation 2-1 to address the high storage requirement of the following algorithm. Their original algorithm is decomposed into three parts for implementing the parallelized algorithm, shown from Equation 2-2 to Equation 2-3. The performance of this proposed algorithm indicates that for large problems a single GPU is up to 45 times faster than a CPU for this application, and the parallel implementation shows linear speed up on up to 4 GPUs which is demonstrated using the SSCA#1 (Bioinformatics) benchmark running on one GPU and up to four GPUs executing in parallel. A discussion of another efficient approach for the



algorithm which is implemented using CUDA [23]. Their proposed score of the best alignment terminated at index $(i-1, j-1)$ is given by the formula in Equation 2-5, and Equation 2-3 shows the formula for an alignment with a gap along the column terminated at index (i, j) and score for an alignment with a gap along row terminated at index (i, j) . The results show that the accessibility of general-purpose computation capabilities allowed by the CUDA programming environment on Nvidia GPU, allowed the algorithm to achieve more than 3.5 times higher per-core performance than previously published implementation of the Smith-Waterman algorithm on GPU.

BLAST. This tool [11] is used for searching both protein and DNA sequences for databases for sequence similarities. In addition, it is a heuristic local alignment algorithm that approximates the dynamic programming Smith-Waterman algorithm. In contrast, BLAST is much faster than the Smith-Waterman algorithm. The study designs and implements the Blast Sequence Algorithms on an FPGA. The study's methodology utilizes Gapped BLAST which is an advancement of the the BLAST with the two-hit method. This has been proven to be faster and gives better alignments and alignment scores. In addition, the study implements BLAST with two-hit method which was devised to reduce the time spent in the extension rep. Figure 2-1 shows the hardware design architecture of the Gaped Blast Algorithm with the two-hit method using the FPGA. This hardware design performs the preprocessing query steps in parallel. Furthermore, their core architecture is parameterized in terms of sequence lengths, match scores, gap penalties and cut-offs, and threshold values. Results show their implementation allows high-performance and efficient FPGA implementations. Another approach was proposed [24] which utilizes the GPU to implement BLAST. The study designed and developed the sequence alignment algorithm called “GPU-BLAST” which is an accelerated version of the popular NCBI-BLAST shown in Figure 2-2. The results of their study showed a speed up mostly between 3 times to 4 times in comparison to the sequential NCBI-BLAST algorithm.

FASTA. This sequence alignment tool compares two sequences to determine their closeness based on a set of substitutions, insertions, and deletions, that transforms one sequence to another. There is a research [25] that presents a study that accelerates the computation time for FASTA through parallelization using Linda, a concurrent programming model. Their study utilizes a straightforward approach to parallelize FASTA where they divide their code into two parts: a master program and a worker program. The straightforward approach for parallelizing FASTA is summarized by the



diagram as shown in Figure 2-3, where there will be one master process and multiple worker processes, and the number of workers depends on the number of available hardware processors. Their results illustrate that when the amount of computation is reduced, the disk I/O becomes a bottleneck which may prevent additional speed-up as the number of processors is increased.

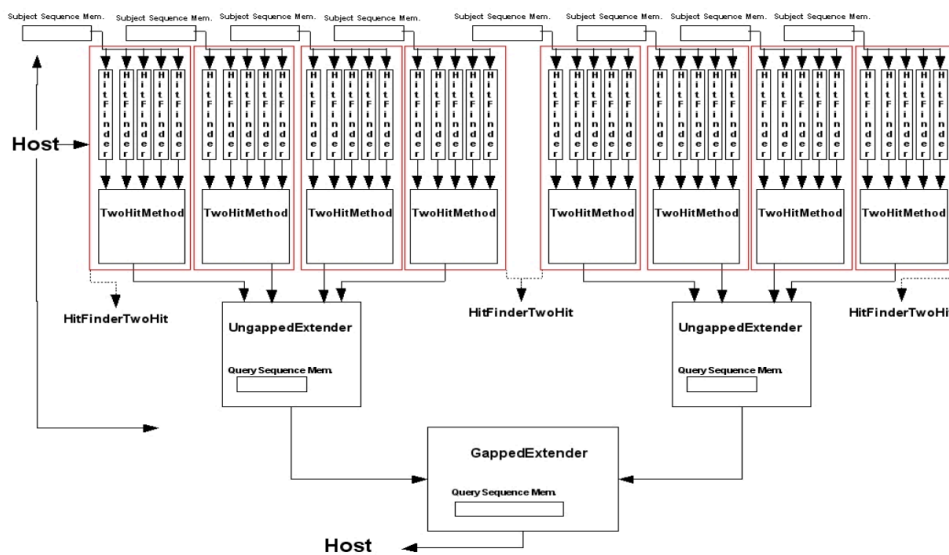


Figure 2-1 Hardware architecture for the Gapped BLAST algorithm with the two-hit method

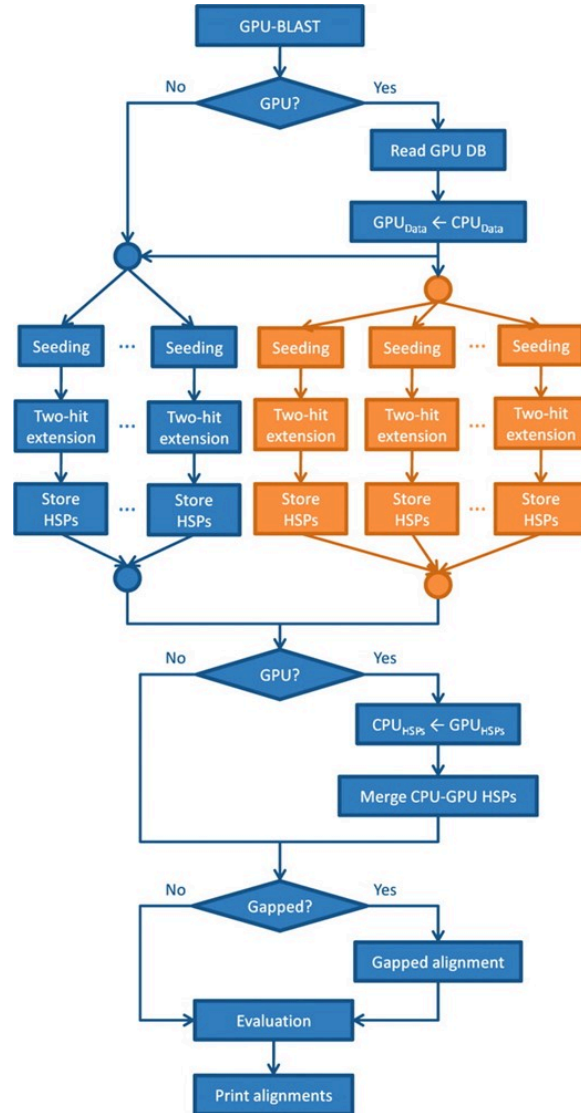


Figure 2-2 Execution flow of GPU-BLAST. The blue blocks are executed on the CPU, and the orange ones on the GPU

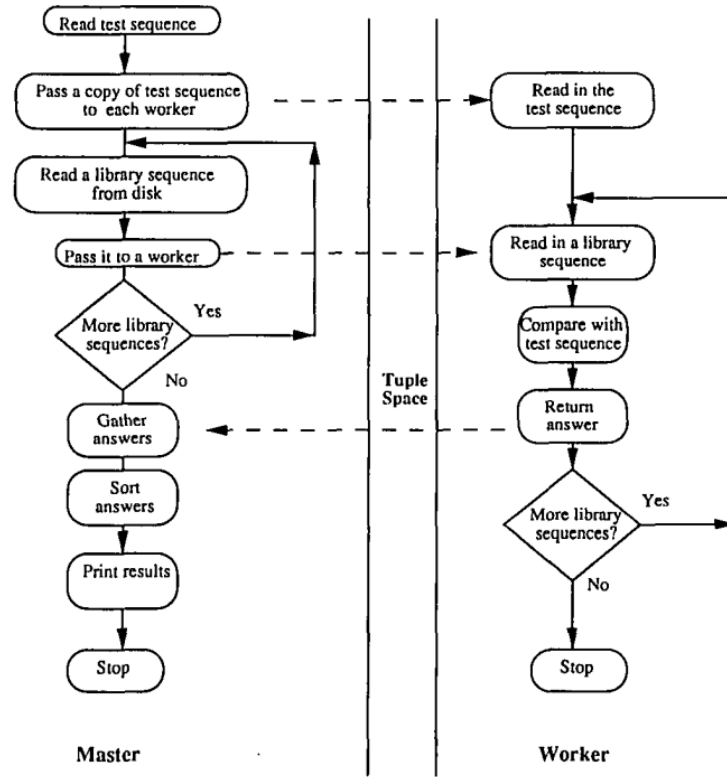


Figure 2-3 A schematic overview of the logic of the straightforward approach to parallelizing FASTA.

$$\begin{aligned}
 E_{i,j} &= \text{Max}(E_{i-1,j}, H_{i-1,j} - G_s) - G_e \\
 F_{i,j} &= \text{Max}(F_{i-1,j}, H_{i-1,j} - G_s) - G_e \\
 H_{i,j} &= \text{Max}(H_{i-1,j-1} + S_{i,j}, E_{i,j}, F_{i,j}, 0)
 \end{aligned}$$

Equation 2-1

$$\begin{aligned}
 F_{i,j} &= \text{Max}(F_{i-1,j}, H_{i-1,j} - G_s) - G_e \\
 \tilde{H}_{i,j} &= \text{Max}(H_{i-1,j-1} + S_{i,j}, F_{i,j}, 0)
 \end{aligned}$$

Equation 2-2

$$H_{i,j} = \text{Max}(\tilde{H}_{i,j}, \tilde{E}_{i,j} - G_s)$$

Equation 2-3



$$H_{i,j} = \max \left\{ \begin{array}{c} 0 \\ E_{i,j} \\ F_{i,j} \\ H_{i-1,j-1} + A_{i,j} \end{array} \right\}$$

Equation 2-4

$$H_{i,j} = \max \left\{ \begin{array}{c} 0 \\ E_{i,j} \\ F_{i,j} \\ H_{i-1,j-1} + A_{i,j} \end{array} \right\}$$

Equation 2-5

$$E_{i,j} = \max \left\{ \begin{array}{c} E_{i,j-1} - G_{ext} \\ H_{i,j-1} - G_{open} \end{array} \right\}$$

$$F_{i,j} = \max \left\{ \begin{array}{c} F_{i-1,j} - G_{ext} \\ H_{i-1,j} - G_{open} \end{array} \right\}$$

Equation 2-6

$$\begin{aligned} TI(j, 0) &= -\infty \\ TD(0, j) &= TD(2, j) = TD(3, j) = -\infty \\ TD(1, j) &= 1 \end{aligned}$$

Equation 2-7. Initialization for traceback matrices of matrix I and D, respectively

$$\begin{aligned} TC(0, 0) &= 0 \\ TC(0, j) &= 0 \\ TC(j, 0) &= 0 \\ TC(1, j) &= \begin{cases} -2, & C(1, j) = I(1, j) \\ -1, & C(1, j) = D(1, j) \\ 1, & C(1, j) = C(0, j-1) + S[b_1, a_j] \end{cases} \\ TC(2, j) &= \begin{cases} -2, & C(2, j) = I(2, j) \\ 2, & C(2, j) = C(0, j-1) + S[b_2, a_j] - \pi \end{cases} \\ TC(3, j) &= \begin{cases} -2, & C(3, j) = I(3, j) \\ 2, & C(3, j) = C(1, j-1) + S[b_3, a_j] - \pi \end{cases} \\ TC(4, j) &= \begin{cases} -2, & C(4, j) = I(4, j) \\ -1, & C(4, j) = D(4, j) \\ 3, & C(4, j) = C(1, j-1) + S[b_4, a_j] \\ 2, & C(4, j) = C(2, j-1) + S[b_4, a_j] - \pi \end{cases} \end{aligned}$$



Equation 2-8. Initialization for traceback matrices of matrix C

$$TI(i, j) = \begin{cases} 0, & I(i, j) = I(i, j-1) - gep \\ 1, & I(i, j) = C(i, j-1) - gop - gep \end{cases} \quad (35)$$

$$TD(i, j) = \begin{cases} 0, & D(i, j) = D(i-3, j) - gep \\ 1, & D(i, j) = C(i-3, j) - gop - gep \end{cases} \quad (36)$$

Equation 2-9 Formula for filling Traceback Matrices

2.1.2 Parallel Implementation

This section of the study discusses related works and implementations using parallel computing. Parallel Computing is the process in which tasks are broken down into smaller tasks and then allocated to multiple processors, which will execute those components concurrently. The parallel implementations discussed in this section are the CUDA and OpenMP implementations of the Needleman-Wunsch Algorithm.

CUDA Implementation. The NW algorithm is a popular dynamic programming approach that finds the optimal alignments between the two given genetic sequences among the many possible alignments. A study [26] developed a parallelization of the algorithm using CUDA and compared it to the sequential approach for the NW algorithm. Their study discusses two approaches for the sequential approach of the algorithm, which are the Row-Wise Technique and the Anti-Diagonal Technique. The Row-wise Technique solves the problem using two vectors, each acting as a reference and a current row, and the approach uses equation 3-1 to compute the score in the table shown in Figure 3-1. The Anti-Diagonal Approach addresses not only the sequence length limitation problem but also the data dependency issue for the parallel implementation. The equations 3-2, 3-3, and 3-4 are used for the former, mid, and latter phases, respectively. The paper parallelizes the Anti-diagonal approach through the heterogeneous Anti-diagonal approach implemented in a CPU-GPU heterogeneous manner using CUDA. The study results show that overall, the heterogeneous implementation performs approximately three times faster than the serial implementation. Furthermore, the results of the study for the sequential approach for the anti-diagonal technique grow $O(nm)$ given by the dimension of the length of the input sequence, and similarly, the computation time for the implementation of heterogeneous anti-diagonal technique grows $O(np)$ assuming that p is the worst case among the parallel executions of the program's inner loop.



OpenMP Implementation. A different study [27] implemented parallel techniques on the NW algorithm using a different parallel computing platform, OpenMP. Their proposed pipeline as their methodology for the parallelization of the Needleman-Wunsch algorithm, as shown in Figure 3-3, initially reads two DNA input sequences and then follows the Dot Plot Matrix, Scoring, Traceback, and searching for the Optimal Alignment is implemented using OpenMP. Implementing the parallel technique using OpenMP, the scoring and traceback techniques used in Needleman-Wunsch are able to align optimally, improve performance in searching for similarity, and reduce gaps and mismatches. Overall, it has achieved a 60% reduction in execution time using the OpenMP directive.

	b1	b2	b3	b4	b5
--a1					INITIALIZE
--a2					
--a3					
--a4					
--a5					

Figure 2-4 Row-Wise Technique Table

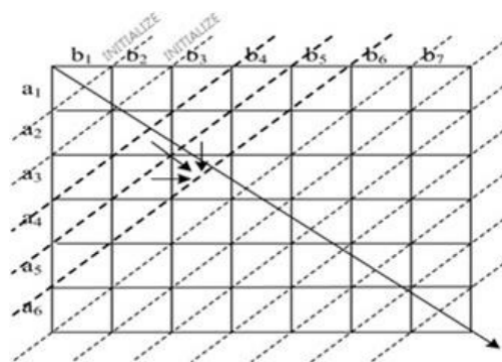


Figure 2-5 Anti-Diagonal Technique

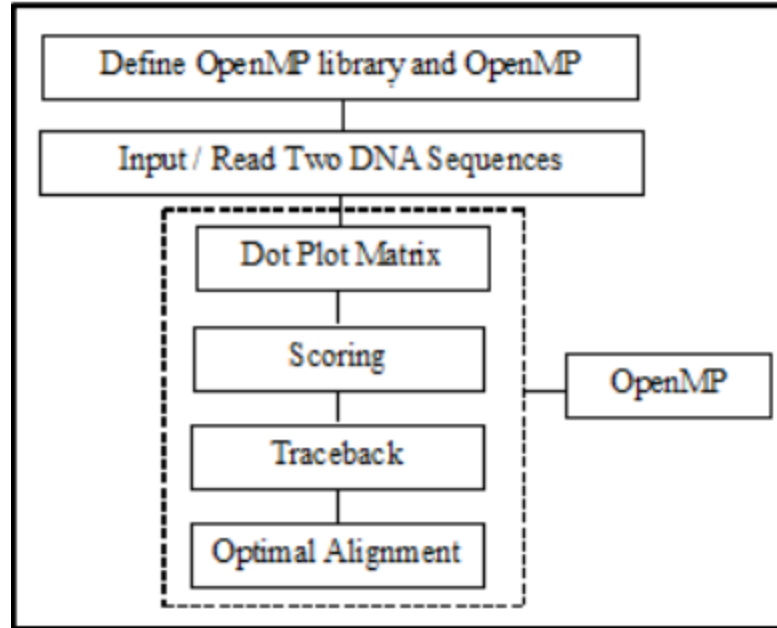


Figure 2-6 Parallelization of Needle program that run based on the Needleman-Wunsch algorithm.

$$CUR(i) = \text{MAX}(REF(i-1) + s(x_{row}, y_i), REF(i) + gap, CUR(i-1) + gap) \quad \text{Equation 2-10}$$

$$CUR(i) = \text{MAX}(REF_D(i-1) + s(x_{row}, y_{col}), REF_HV(i-1) + gap, REF_HV(i) + gap) \quad \text{Equation 2-11}$$

$$CUR(i) = \text{MAX}(REF_D(i-1) + s(x_{row}, y_{col}), REF_HV(i) + gap, REF_HV(i+1) + gap) \quad \text{Equation 2-12}$$

$$CUR(i) = \text{MAX}(REF_D(i+1) + s(x_{row}, y_{col}), REF_HV(i) + gap, REF_HV(i+1) + gap) \quad \text{Equation 2-14}$$



2.2 Review of Related Literature - DNA-Protein Alignment

Table 2-2. Review of Related Literature - DNA-Protein Alignment

Author/s	Title	Description	Algorithm Used
Yao, Y., & Frith, M. C. (2023)	Improved DNA-Versus-Protein in Homology Search for Protein Fossils	Developed a method based on Zhang's three frame alignment specially designed for fossil proteins.	Improved HomologySearch
Frith, M. C. (2022).	Paleozoic Protein Fossils Illuminate the Evolution of Vertebrate Genomes and Transposable Elements. Molecular Biology and Evolution	Explores ancient protein fossils within vertebrate genomes, specifically on "transposable elements" (TEs) and fragments of protein-coding DNA.	LAST software
Frith, M. C., Wan, R., & Horton, P. (2010)	Incorporating sequence quality data into alignment improves DNA read mapping.	Improved the LAST software, in particular the model's mapping accuracy.	LAST software
Kielbasa, S., Wan, R., Sato, K., Horton, P., & Frith, M. (2010)	Adaptive seeds tame genomic sequence comparison.	Developed the LAST software.	LAST software
Peiris, M., J. A. A. M. Jayaweera, & Ganegoda, G. U. (2022)	Melanoma Detection by Analysing Mutations in Gene DNA Sequences and Their Primary Protein Structures	Proposed an approach for detecting Melanoma by analyzing gene DNA sequences of a subject.	Smith-Waterman
Santiago, K. C., & Shrestha, A. (2024)	DNA-protein quasi-mapping for rapid differential gene expression analysis in	Provided a differential gene expression analysis pipeline for non-model organisms without a reference transcriptome.	Quasi Mapping



	non-model organisms		
Custer, J. M., White, R., Taylor, H., Schmidlin, K., Fontenele, R. S., Stainton, D., Kraberger, S., Briskie, J. V., & Arvind Varsani. (2021)	Diverse single-stranded DNA viruses identified in New Zealand (Aotearoa) South Island robin (<i>Petroica australis</i>) fecal samples	Aimed to fill the knowledge gap regarding the virome of this bird species, particularly in an isolated population on Nukuwaiata Island.	BLASTX
BLAST (2015)	Align translated DNA with proteins using BLAST	Elaborated on the different types of BLAST algorithms and how they work	BLASTX
Buchfink, B., Xie, C., & Huson, D. H. (2014)	Fast and sensitive protein alignment using DIAMOND	Developed an open-source algorithm based on double indexing that is 20,000 times faster than BLASTastX on short reads and has a similar degree of sensitivity	DIAMOND

In this section, the researchers summarize and review related literature on applications of DNA-Protein Alignment. DNA-protein alignment is a technique used to identify similarities between DNA sequences and protein sequences. Its applications range from fields such as metagenomics to virology.

Metagenomics. DNA-protein alignment is used to discover new transposable element [6] families by comparing ancient DNA sequences to known proteins. The study utilizes an optimized DNA-Protein method in LAST. This method is designed to detect protein fossils by allowing frameshift tolerance. Hence, this increases the likelihood of matching parameters. By doing so, it was able to identify ancient protein DNA across multiple genomes. LAST has 2 main steps: namely, matching and scoring [10]. LAST introduces



adaptive seeds that adjust based on the local sequence context, ignoring common matches and focusing on more informative ones. Adaptive seeds identify promising alignment regions by finding right-minimal exact matches. The matching phase starts by utilizing enhanced suffix arrays. This index allows LAST to locate the positions of each possible k-mer in the reference, streamlining the matching process. After this, it generates possible sequences using adaptive seeds by extending these seed matches to build high-scoring alignments using gap penalties and scoring matrices. LAST then performs an alignment process to refine these sequences by applying filters to remove low-quality or redundant alignments, ensuring that only the most relevant results are retained [28].

Additionally, a study by Yao et. al [5] developed a method inspired by the three-frame alignment that is designed for fossil proteins. This method introduces the concept of spaced seeds, which are patterns that allow gaps in the alignment. Spaced seeds allow for the capture of alignments that are undetectable with traditional seed models, making the approach more suited for faint, degraded sequences (i.e., protein fossils). Scoring (seen in Equation 2-15) was designed to match the evolutionary characteristics of protein fossils better. The study also included adjusting parameters to minimize false positives while enhancing sensitivity to weakly conserved sequences. After the parameters tuning test, Equation 2-17 showcases the final parameters used.

- Score for aligning amino acid u to base triplet V

$$S_{uV}$$

- Score for an insertion of k bases:

$$a_I + b_I k + \begin{cases} 0 & \text{if } k \bmod 3 = 0 \\ f_I & \text{if } k \bmod 3 = 1 \\ g_I & \text{if } k \bmod 3 = 2 \end{cases}$$

- Score for a deletion of k bases:

$$a_D + b_D k + \begin{cases} 0 & \text{if } k \bmod 3 = 0 \\ f_D & \text{if } k \bmod 3 = 1 \\ g_D & \text{if } k \bmod 3 = 2 \end{cases}$$

Equation 2-15. Scoring Scheme

$$\begin{aligned} y_1 &= Y_{i-1 \ j-2} + [b_D + f_D] & z_1 &= Z_{i \ j-1} + [b_I + f_I] \\ y_2 &= Y_{i-1 \ j-1} + [2b_D + g_D] & z_2 &= Z_{i \ j-2} + [2b_I + g_I] \\ y_3 &= Y_{i-1 \ j} + [3b_D] & z_3 &= Z_{i \ j-3} + [3b_I] \\ X_{ij} &= \max(X_{i-1 \ j-3} + S_{R_i Q_j}, y_1, y_2, y_3, z_1, z_2, z_3, 0) \\ Y_{ij} &= \max(X_{ij} + a_D, y_3) & Z_{ij} &= \max(X_{ij} + a_I, z_3) \end{aligned}$$



Equation 2-16. Alignment Scheme

$$\begin{aligned}
 S'_{uV} &= \exp\left(\frac{S_{uV}}{t}\right) = \frac{\Gamma}{\omega_D \omega_i^3} \cdot \frac{\pi_{uV}}{\phi_u \psi_V} \\
 a'_I &= \exp\left(\frac{a_I}{t}\right) = \frac{\alpha_I(1 - \beta_I)}{\beta_I} \\
 a'_D &= \exp\left(\frac{a_D}{t}\right) = \frac{\alpha_D(1 - \beta_D)}{\beta_D} \\
 b'_I &= \exp\left(\frac{b_I}{t}\right) = \frac{\sqrt[3]{\beta_I \delta_I \epsilon_I}}{\omega_i} \\
 b'_D &= \exp\left(\frac{b_D}{t}\right) = \sqrt[3]{\frac{\beta_D \delta_D \epsilon_D}{\omega_D}} \\
 f'_I &= \exp\left(\frac{f_I}{t}\right) = \frac{1 - \delta_I}{1 - \beta_I} \sqrt[3]{\frac{\beta_I^2}{\delta_I \epsilon_I}} \\
 f'_D &= \exp\left(\frac{f_D}{t}\right) = \frac{1 - \delta_D}{1 - \beta_D} \sqrt[3]{\frac{\beta_D^2}{\delta_D \epsilon_D \omega_D^2}} / \omega_i^2 \\
 g'_I &= \exp\left(\frac{g_I}{t}\right) = \frac{1 - \epsilon_I}{1 - \beta_I} \sqrt[3]{\frac{\beta_I \delta_I}{\epsilon_I^2}} \\
 g'_D &= \exp\left(\frac{g_D}{t}\right) = \frac{1 - \epsilon_D}{1 - \beta_D} \sqrt[3]{\frac{\beta_D \delta_D}{\epsilon_D^2 \omega_D}} / \omega_i
 \end{aligned}$$

Equation 2-17. Score and Model Parameters

Gene Analysis. Another application of DNA-Protein Alignment is for identifying frameshift mutations, which are shifts in the DNA sequences that can lead to faulty proteins. Although frameshifts occur naturally over time through evolution, it can also result from genetic mutations, sometimes leading to diseases. Hence, detecting these mutations is important for fields like evolutionary biology and medical genetics. For example, in Melanoma Detection [7], it is used to identify and convert mutated genes to their protein equivalents. Figure 2-7 showcases the approach outline used in the study. The process is divided into 2 processes, DNA Mutation Checking, where the first module checks for the mutations from the DNA sequence, and PPS Analysis, where the second module uses the protein primary structure (PPS) to check for mutation impact on Melanoma. DNA mutation checking starts by checking if the sequence is fragmented or not. If yes, the sequence will undergo defragmentation. If not, it will proceed to the next step. The next step uses Smith-Waterman Alignment. This will be used to identify the kind of mutation and the causes. These outputs will then be passed to the next module, as seen in Figure 2-8, which will detect if Melanoma is present or not.

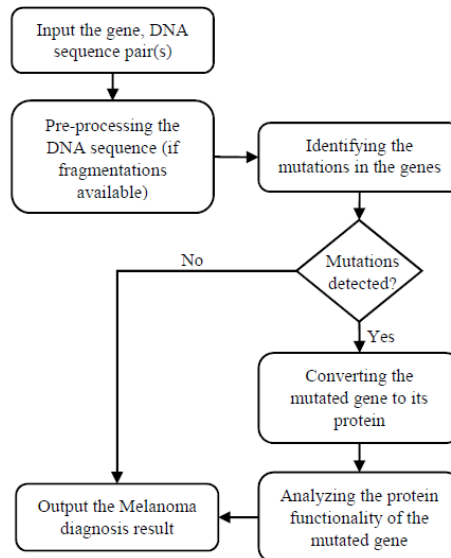


Figure 2-7. System Overview of Melanoma Detection System

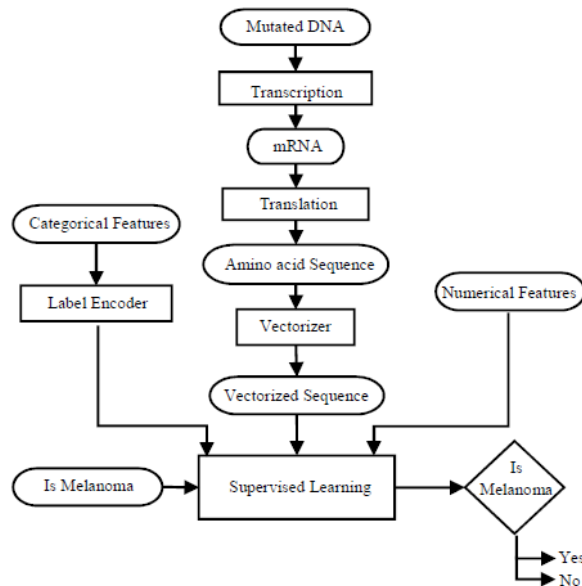


Figure 2-8. System Overview of PPS Analysis



Gene Expression. DNA-protein alignment can be an approach for differential gene expression (DGE) analysis in non-model organisms, where there may be limited or no well-annotated reference genomes available. A study [8] presents an alternative approach for differential gene expression analysis for non-model organisms by using a method called quasi-mapping. Quasi-mapping is a technique that aligns RNA-seq reads to protein sequences without full alignment. Figure 2-9 showcases the quasi-mapping of one translation. For each sequence, the algorithm matches by examining the k-mers starting at different positions. This is repeated for all six reading frames. For each position in the translation, if the k-mer starting at this position is found in a hash table, the algorithm retrieves its associated suffix array interval, indicating potential matches. The match is extended by adding subsequent amino acids and performing binary searches within the suffix array until no more matches are found. This method achieved over 1000x speed improvement over assembly-based methods.

Input: Translation T of a read, concatenated reference C , suffix array SA , hash table HT , bit-vector B , k-mer size k , and coverage threshold t .

Output: Set of protein sequences that T maps to.

```

1:  $pos \leftarrow 0$ 
2:  $coverage_P \leftarrow 0$  for each protein  $P$  in reference
3: while  $pos + k < |T|$  do
4:   if  $T[pos : pos + k - 1]$  is a key in  $HT$  then
5:      $sai \leftarrow HT(T[pos : pos + k - 1])$ 
6:      $k' \leftarrow k + 1$ 
7:     while  $(sai \neq \emptyset)$  and  $(pos + k' < |T|)$  do
8:        $sai \leftarrow SA$  interval for  $T[pos : pos + k' - 1]$  by binary-searching  $sai$ 
9:        $k' \leftarrow k' + 1$ 
10:    end while
11:    for  $P \in rank_B(sai')$  do
12:       $coverage_P \leftarrow coverage_P + k'$ 
13:    end for
14:     $pos \leftarrow pos + k'$ 
15:  else
16:     $pos \leftarrow pos + 1$ 
17:  end if
18: end while
19: return  $\{P \mid coverage_P > t\}$ 

```

Figure 2-9. Quasi-mapping Algorithm

Virology. DNA-protein alignment is also used to analyze DNA from diverse microorganisms in environments. By comparing these DNA sequences to known protein databases, researchers can classify unknown microbes and learn about their biological functions, even if the organisms can't be cultured in a laboratory. A study [9] examines the diversity of single-stranded DNA (ssDNA) viruses found in the fecal matter of the South Island robin, an endangered bird species native to New Zealand. The sequences were able to be classified into recognized viral families (Circoviridae, Genomoviridae,



and Microviridae) based on protein homology. In this study, BLASTX [11] is used to analyze the viral sequences identified from South Island robin fecal samples. BLASTX is an algorithm used to annotate coding regions on a nucleotide sequence while also detecting frameshifts in these coding regions. The viral DNA sequences were compared to a protein database using BLASTX. This translation was necessary because many viral sequences do not directly match known nucleotide sequences in GenBank. By translating the DNA sequences into protein sequences, BLASTX allowed the researchers to capture evolutionary relationships and functional similarities at the protein level. This resulted in the identification of specific viral genes, such as those encoding replication-associated proteins and capsid proteins.

Another alternative technique is DIAMOND (double index alignment of next-generation sequencing data) [12], an open-source program that is ideally suited for replacing BLASTX. Unlike BLASTX, which uses a single index to locate seeds in the reference database, DIAMOND uses double indexing. It indexes both the query and reference sequences, creating two lists of seed locations that are matched in a way that maximizes data locality and minimizes memory bandwidth requirements. This technique enhances DIAMOND's speed by reducing the need to load data from main memory frequently. This results in DIAMOND being significantly faster than BLASTX, achieving up to 20,000 times the speed on short reads, making it suitable for large metagenomic studies.



3. Theoretical Framework

3.1 Modified Zhang's Three-Frame Algorithm

DNA Sequence	A	T	G	C	A	C	T	G	C	A	C	T	G	C	A	G	T	G	C	A	C
Frame 1		M			H			C			T			A		V			T		
Frame 2			C			T			A			L			Q			C			T
Frame 3				A			L			H			C			S			A		
Protein Sequence		M			H			S		H			C			H			C		T
									*							*					

Figure 3-1 Three-Frame DNA-Protein Alignment. Asterisks mean frameshift

In DNA-to-protein sequence alignment, the DNA sequence is split into codons, which are groups of three nucleotides corresponding to a distinct amino acid. However, because the DNA sequence can be read in several frames, alignment must account for frameshift errors caused by insertions or deletions in the sequence. While many algorithms have been developed for DNA-protein alignment, Zhang's three-frame alignment approach stands out as a simple yet effective method [5].

Zhang's Three Frame Alignment Algorithm [13] is a sequencing alignment that matches a DNA sequence with a protein sequence by translating the DNA sequence into three reading frames. Each frame represents a separate codon translation of different starting locations. The algorithm compares all three frames to the reference protein sequence to ensure that any frameshift faults (insertions or deletions) are captured in the alignment.

To compute alignments, three matrices are generated (*C*, *I*, and *D*): the *C* matrix, which has the highest alignment scores; the *I* matrix, which handles insertions; and the *D* matrix, which manages deletions. The algorithm builds scoring matrices for each frame, which are used to determine the best alignment by comparing potential matches, mismatches, insertions, and deletions. Each cell in the matrix represents a score based on the optimal path from the beginning of the sequence to that position. The algorithm uses a recurrence relation to update these scores and rewards for matches based on the BLOSUM62 matrix. Additionally, the algorithm uses gap penalties to account for insertions and deletions in the sequences. A gap-opening penalty (*gop*) is applied whenever a new gap is introduced, while a gap-extension penalty (*gep*) is charged for extending an existing gap. This distinction helps the algorithm better reflect the biological cost of different types of gaps. Finally, the algorithm assigns a frameshift



penalty (π) to each instance where a nucleotide insertion or deletion causes a shift in the reading frame. This penalty ensures that the algorithm accurately accounts for disruptions in the sequence that might otherwise lead to misalignments. This method allows the algorithm to efficiently determine the best path through the matrix, which corresponds to the best possible alignment.

However, unlike the original Zhang's three-frame alignment algorithm [21], which utilizes global alignment, the modified version utilizes a local approach that allows partial alignment. This was done by initializing the first row of the C matrix to zero, whereas in the original implementation, it would have contained the gap penalties. This modification allows the first column to be completely unmodified to ensure that the DNA sequence aligns completely; in other words, the DNA sequence does not have to match the reference protein from beginning to end.

3.2 CUDA and SIMT Paradigm

Many applications, particularly sequencing alignment, are required to process large datasets or to execute complex algorithms. To fulfill these requirements, parallel computing is one of the most commonly used solutions, which utilizes multiple processing units to perform tasks simultaneously. NVIDIA's Compute Unified Device Architecture (CUDA) is one of the tools used in parallel computing. CUDA allows users to utilize the extensive parallelism embedded within the Graphics Processing Units (GPUs) using standard programming languages like C, C++, and Python [29].

CUDA's interface enables high-performance computing tasks that would otherwise be inefficient in traditional CPU implementations. CUDA's design is based on the Single Instruction, Multiple Threads (SIMT) approach, which allows several threads to execute identical instructions on different data streams, resulting in a massively parallel computation. CUDA utilizes two forms of parallelism: data parallelism and task parallelism. Data parallelism is concurrently running the same operation across multiple data streams, which is typically used in large-scale matrix or vector operations. On the other hand, task parallelism is concurrently running the various operations on different threads [30]. A thread block's threads can run concurrently, and a thread ID is used to identify each thread. The formula [31] in Equation 3-1 is used to deduce the thread ID used:

$$\begin{aligned} ID = & (blockIdx.y * gridDim.x + blockIdx.x) \\ & * (blockDim.x * blockDim.y) \\ & + threadIdx.y * blockDim.x + threadIdx.x \end{aligned} \quad \text{Equation 3-1}$$



In order to harness the parallelism capabilities of GPUs, the problem must be broken down into independent tasks that can be performed concurrently. This breakdown makes it possible for many threads to be executed at the same time, which is the main benefit of CUDA. For example, the code snippet below shows how CUDA's parallel architecture can be used on element-wise operations on arrays, similar to how operations can be parallelized in Zhang's six-frame algorithm.

```
void ElementwiseOp (size_t n, int* out, int* x, int* y, int* a){
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = index; i < n; i += stride)
        out[i] = a[i] + x[i] * y[i];
}
```

Figure 3-2 Sample CUDA code (ElementwiseOp Function)

In Figure 3-2, the function, *ElementwiseOp*, runs on the GPU, where each thread processes a segment of data independently. In the function, the variable *index* represents the starting point of the threads, while the variable *stride* makes sure the threads can handle multiple data points. In this example, parallelization is implemented by distributing the workload across multiple threads. This allows operations to be performed concurrently. In addition to this, memory management and prefetching can be handled using CUDA's unified memory and prefetching functions, as seen below.

```
int *x, *y, *a, *out;
cudaMallocManaged(&x, ARRAY_BYTES);
cudaMallocManaged(&y, ARRAY_BYTES);
cudaMallocManaged(&a, ARRAY_BYTES);
cudaMallocManaged(&out, ARRAY_BYTES);
// Prefetch data to the GPU
cudaMemPrefetchAsync(x, ARRAY_BYTES, device, NULL);
cudaMemPrefetchAsync(y, ARRAY_BYTES, device, NULL);
cudaMemPrefetchAsync(a, ARRAY_BYTES, device, NULL);
```

Figure 3-3 Sample CUDA code (CUDA's unified memory and prefetching functions)

In Zhang's six-frame algorithm, large datasets and inputs, such as the reference proteome and the DNA sequence, are handled to be processed. Traditional implementation would involve manually transferring data between the CPU and GPU, which can lead to high execution times, especially when dealing with massive sequences. Hence, using CUDA's unified memory *cudaMallocManaged*, the system can allocate memory that can be accessed by the CPU and GPU, effectively eliminating the need for explicit data transfers.



This ensures that the GPU can fetch the data efficiently. Additionally, CUDA's prefetching, *cudaMemPrefetchAsync*, ensures that the DNA sequence and other necessary data are loaded into the GPU before the alignment computation begins. This way, when the kernel starts processing, the data is already available in the GPU, effectively reducing latency.

However, to fully maximize performance, issues like occupancy, warp divergence, and memory access must be carefully considered. Occupancy relates to the usage of available resources; to be precise, it refers to the number of threads that operate concurrently. Higher occupancy ensures that the GPU is always working rather than waiting for data to be retrieved, successfully hiding memory latency. Warp divergence happens when threads inside a warp follow a separate execution path. This reduces performance since the GPU has to run these paths sequentially rather than in parallel. To reduce warp divergence, conditional branches must be minimized within warps.

3.3 Wavefront Paradigm

The foundation of Zhang's six-frame algorithm is centered around the concept of Dynamic programming, which utilizes matrices to score the alignment between the DNA and protein sequences. The challenge in implementing the algorithm in parallel is the fact that each cell in the matrix depends on its neighboring cells. This creates data dependencies that make it difficult to implement in parallel, as you can't compute the value of a cell until its neighbor's value is determined.

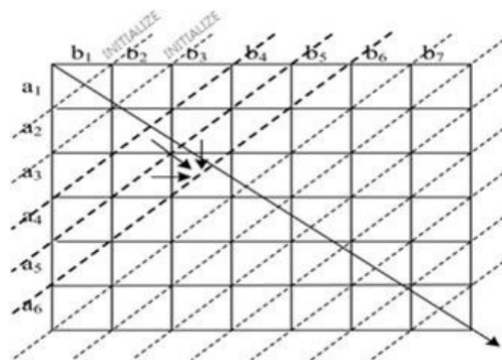


Figure 3-5: Anti-Diagonal Technique

The anti-diagonal technique shown in Figure 4-1 is a technique used for filling in a matrix by processing its elements along diagonals that run from the top left to the bottom right instead of the more traditional row-by-row or column-by-column

The anti-diagonal approach is often employed in DNA, RNA, or protein sequence alignment scenarios (See more in Chapter 2.1.2). It can efficiently handle large datasets and allow for faster computations, making it a preferred method for algorithms implemented on parallel processing systems like GPUs.

The BLOSUM62 matrix is one of the most commonly used scoring systems in bioinformatics, in particular for protein sequence alignment. It is part of the BLOSUM (Blocks Substitution Matrix) series, which was developed by analyzing patterns of amino acid substitutions in blocks of ungapped alignment from related proteins. The BLOSUM62 matrix includes data from proteins that are up to 62% identical.

[illegible]

32



For instance, amino acids with similar side chains (like aspartate (D) and glutamate (E)) receive positive scores because they often replace each other without drastically affecting the protein's structure. While substituting two very different amino acids (e.g., phenylalanine (F) and glutamine (Q)) is less common and, hence, receives negative scores. Additionally, the diagonal of the matrix contains self-substitution scores, which partly reflect the natural abundance of each amino acid. For example, rarer amino acids such as tryptophan (W) have relatively high scores, while more common amino acids like valine (V), leucine (L), and isoleucine (I) have lower scores. Unique cases like proline (P) and glycine (G) have higher self-scores due to their specific roles in maintaining protein structure.



4. System Design

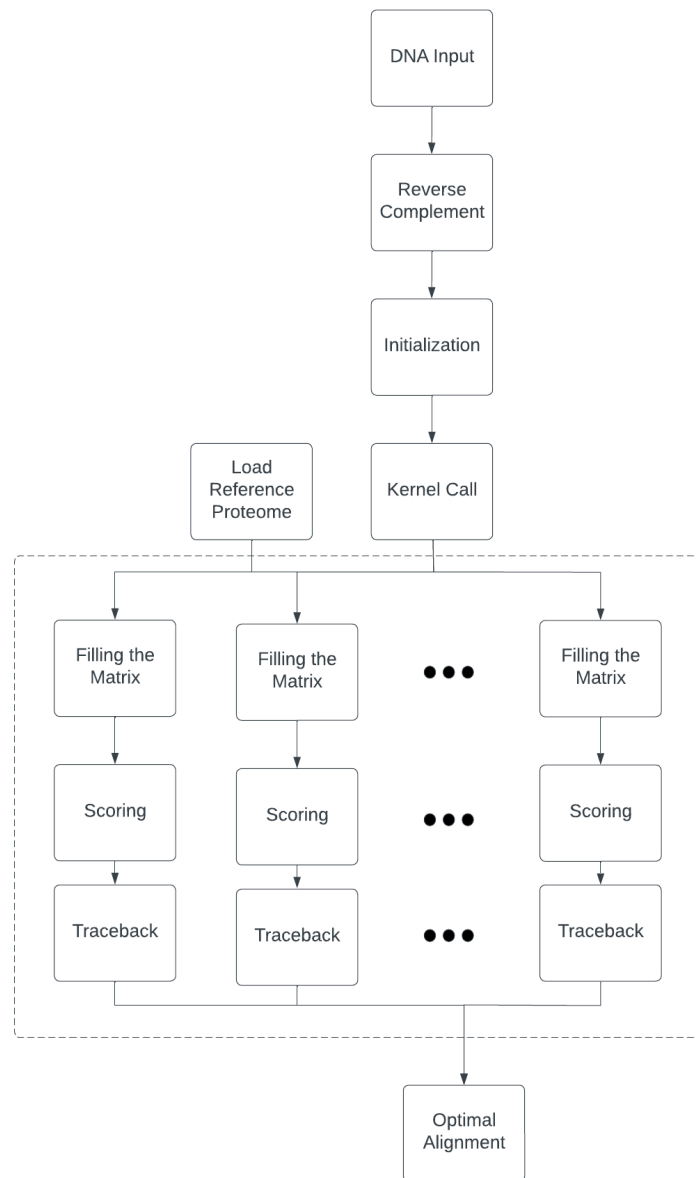


Figure 4: System Overview



4.1 Zhang's Six-Frame Algorithm

Before implementing the proposed parallel implementation, it's essential to establish a sequential implementation. This will serve as a baseline for comparison and ensure the correctness of the parallelized version.

DNA Sequence	A	T	G	C	A	C	T	G	C	A	C	T	G	C	A	G	T	G	C	A	C
R. Complement	A	G	T	G	T	G	C	A	C	T	G	C	A	G	T	G	C	A	G	T	G
Frame 4		S			V			H			C			S			A			V	
Frame 5			V			C			T			A			V			Q			C
Frame 6				C			A			L			Q			C			S		
Protein Sequence	M				H		S	H			C			H		C			T		
																*					

Figure 4-1 Six-Frame DNA-Protein Alignment

Zhang's Six-Frame Algorithm begins by reading an input DNA sequence and a reference protein sequence. The DNA sequence is then processed to generate all possible reading frames (three frames from the forward strand and another three frames from the reverse complement strand). Three reading frames from each strand are generated by starting translation at the first, second, and third nucleotide positions of the DNA sequence. Using the same example stated above (Figure 3-1), Figure 4-1 shows the next three frames from the reverse complement strand.



$$\begin{aligned}
 I(j, 0) &= -\infty \\
 D(0, j) &= D(2, j) = D(3, j) = -\infty \\
 D(1, j) &= C(0, j) - \text{gop} - \text{gep} \\
 C(0, 0) &= 0 \\
 C(0, j) &= I(0, j) \\
 C(j, 0) &= D(j, 0) \\
 C(1, j) &= \max \begin{cases} I(1, j) \\ D(1, j) \\ C(0, j-1) + S[b_1, a_j] \end{cases} \\
 C(2, j) &= \max \begin{cases} I(2, j) \\ C(0, j-1) + S[b_1, a_j] - \pi \end{cases} \\
 C(3, j) &= \max \begin{cases} I(3, j) \\ C(1, j-1) + S[b_2, a_j] - \pi \end{cases} \\
 C(4, j) &= \max \begin{cases} I(4, j) \\ D(4, j) \\ C(1, j-1) + S[b_4, a_j] \\ C(2, j-1) + S[b_4, a_j] - \pi \end{cases}
 \end{aligned}$$

Equation 4-1 Matrix Initialization Formula

The algorithm then starts by initializing the three main matrices. The C Matrix holds the alignment scores. The I Matrix holds the insertion scores. The D Matrix holds the deletion scores. During this initialization, the algorithm sets scores assuming a local alignment. Local alignment focuses on identifying the most similar subsequences rather than forcing the entire sequence to match. The equation below shows the initialization of the matrices.

The scores are obtained from the BLOSUM62 matrix, which is commonly used to evaluate how likely it is for one amino acid to be substituted for another. While *gop*, *gep* and π are initialized below.

$$\text{gop} = 2; \text{gep} = 3; \pi = 4 \quad \text{Equation 4-2}$$

Once the initialization is finished, the algorithm begins processing the sequences. For each cell, the algorithm uses the recurrence relation found in Equation 4-3 to decide the best possible scores by comparing the different alignment paths.



$$\begin{aligned}
 I(i, j) &= \max \begin{cases} I(i, j-1) - gep \\ C(i, j-1) - gep - gop \end{cases} \\
 D(i, j) &= \max \begin{cases} D(i-3, j) - gep \\ C(i-3, j) - gep - gop \end{cases} \\
 C(i, j) &= \max \begin{cases} I(i, j) \\ D(i, j) \\ C(i-2, j-1) + S[b_i, a_j] - \pi \\ C(i-3, j-1) + S[b_i, a_j] \\ C(i-4, j-1) + S[b_i, a_j] - \pi \end{cases}
 \end{aligned}$$

Equation 4-3

The algorithm will evaluate the scores from the I and D matrix for each cell in the C matrix. If starting or continuing a gap generates a higher score for the alignment, the algorithm will choose the values from the I and D matrices. On the other hand, if aligning the sequences directly provides a higher score, the matches from the values *sc_1*, *sc_2*, or *sc_3* will be selected. This approach considers all possible variations, providing the best score for that cell based on the current sequence.

A key feature of Zhang's six-frame alignment is its ability to handle frameshifts. Frameshifts occur when insertions or deletions cause the reading frame to change, which can significantly alter how the DNA sequence translates into proteins. The algorithm accounts for this by using *sc_1*, *sc_2*, and *sc_3*, which allows different matching possibilities that take into consideration potential shifts in the reading frame. When the algorithm encounters a situation where a frameshift might have occurred, it applies a shift penalty to guide the alignment. This ensures the algorithm correctly identifies and adjusts for frameshifts rather than forcing a faulty alignment.

In summary, the sequential implementation processes each cell of the matrix row by row, filling in scores based on the alignment scoring mechanisms, including match rewards, mismatch penalties, gap penalties, and frameshift penalties. After the matrices are filled, a traceback is performed to find the best alignment path across all frames, starting from the highest score.

4.2 Dataset

The dataset utilized in this research comprises protein and DNA sequences for *Drosophila melanogaster* (fruit fly) in FASTA format. The dataset is obtained from UniProt [34] under the proteome ID UP000000803. The dataset contains a total of 22,032



proteomes of this species, which can be conveniently accessed via direct download or through API calls.

However, to get a single proteome's DNA translation, all DNA sequences available for a given proteome must be examined on the European Molecular Biology Laboratory (EMBL) website [35]. This process involves retrieving the FASTA files for all potential DNA translations of the proteome and then translating them using a translation tool, "Expasy [36]," to get its protein sequence. The translated sequences are then compared to identify the DNA sequence that matches the reference proteome

This method of data gathering is both time-consuming and resource-intensive to perform. For an efficient method of data gathering, the same process was automated by a web-crawler script developed in Python using the Selenium package. This automation makes it convenient for rapid data extraction, significantly reducing the time required for data collection. Furthermore, the data extraction method is further sped up with the implementation of multithreading within the script, allowing for concurrent data extraction operations.

4.3 Preparation and Preprocessing

This section discusses the preparatory steps necessary before executing Zhang's Six-Frame Alignment Algorithm using CUDA. These steps include initializing the data structures and constants that support dynamic programming, translating DNA input into amino acid sequences, and initializing the CUDA kernel's grid and block structure to optimize GPU performance.

4.3.1 Data Initialization

Since dynamic programming is the groundwork for Zhang's six-frame algorithm, each matrix value is determined based on the previous operation values. This results in data dependencies across rows, columns, and tables, which need to be resolved before implementing the algorithm in parallel.

To handle this, the matrices' first four rows and first column are initialized beforehand. This ensures that the data dependencies are resolved before the alignment computation occurs. The algorithm utilizes the SIMT model to process the remaining cells without the risk of missing or outdated values.



Furthermore, constants such as gap opening penalty, gap extension penalty, and frameshift penalty are initialized before the matrix operation. Meanwhile, constants like the BLOSUM62 matrix are stored in constant memory.

4.3.2 Translation and BLOSUM62

A hash table will be used for the conversion of codon to amino acid since hash tables are far more efficient than a sequence of if-else statements. This helps prepare the data for comparison with the reference protein sequence. After conversion, these amino acids can be mapped into indices corresponding to their BLOSUM62 score. This allows the GPU to quickly fetch the match or mismatch score during the alignment process.

4.3.3 Grid and Block Configuration

Initialization of the grid and block size is crucial in maximizing GPU performance. A block is a group of threads that is assigned to execute a portion of an algorithm. The number of threads in the block, called the block size, directly affects how efficiently the GPU can process the operation. On the other hand, the grid is made up of blocks and is used to determine how many blocks will be launched in a given kernel. The grid size must be chosen carefully to ensure that all parts of the operation are processed and the GPU remains fully occupied.

The CUDA Driver API [32] provides a convenient way to compute the optimal block size and the grid size based on the specific kernel and the system's available resources using the *cuOccupancyMaxPotentialBlockSize* function. It takes into account how much shared memory the kernel will utilize and returns the best block size and grid size that maximizes occupancy.

4.4 Matrix Parallelization

After the necessary preparation, a CUDA kernel call will be made to pass the necessary matrices, sequences, and variables to the CUDA kernel and the GPU for computation.

Since the algorithm to be parallelized contains many dependencies, there is a challenge to efficiently parallelize the matrix calculations. Inter-matrix dependencies, previous values, and previous row/column dependencies are present in the algorithm. This means that row-wise or column-wise parallelization is not feasible due to the needed values being uncomputed. Thus, the algorithm will be implemented through an anti-diagonal wavefront propagation-like method, beginning from the upper-left-most cell.



4	-5	-5	-5	-5	-5	-5	-5
5	-5	0					
6	-5						
7	-5						
8	-5						
9	-5						
Matrix I							

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4					
6	-5						
7	-4						
8	0						
9	-3						
Matrix D							

4	0	3	3	3	3	3	3
5	0	3					
6	0						
7	0						
8	0						
9	0						
Matrix C							

Figure 4-2: Concept of Parallelization, Cycle 1

4	-5	-5	-5	-5	-5	-5	-5
5	-5	0	1				
6	-5	-3					
7	-5						
8	-5						
9	-5						
Matrix I							

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4	-4				
6	-5	-5					
7	-4						
8	0						
9	-3						
Matrix D							

4	0	3	3	3	3	3	3
5	0	3	3				
6	0	3					
7	0						
8	0						
9	0						
Matrix C							

Figure 4-3: Concept of Parallelization, Cycle 2

4	-5	-5	-5	-5	-5	-5	-5
5	-5	0	1	1			
6	-5	-3	-3				
7	-5	-4					
8	-5						
9	-5						
Matrix I							

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4	-4	-4			
6	-5	-5	-5				
7	-4	-4					
8	0						
9	-3						
Matrix D							

4	0	3	3	3	3	3	3
5	0	3	3	3			
6	0	3	3				
7	0	3					
8	0						
9	0						
Matrix C							

Figure 4-4: Concept of Parallelization, Cycle 3

4	-5	-5	-5	-5	-5	-5	-5
5	-5	0	1	1	1		
6	-5	-3	-3	-3			
7	-5	-4	-3				
8	-5	0					
9	-5						
Matrix I							

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4	-4	-4	-4		
6	-5	-5	-5	-5			
7	-4	-4	-4				
8	0	1					
9	-3						
Matrix D							

4	0	3	3	3	3	3	3
5	0	3	3	3	3		
6	0	3	3	3			
7	0	3	3				
8	0	3					
9	0						
Matrix C							

Figure 4-5: Concept of Parallelization, Cycle 4

4	-5	-5	-5	-5	-5	-5	-5
5	-5	0	1	1	1	1	1
6	-5	-3	-3	-3	-3	-3	-3
7	-5	-4	-3	-2	-2	-2	-2
8	-5	0	5	6	6	6	
9	-5	-3	-1	-1	-1		
Matrix I							

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4	-4	-4	-4	-4	-4
6	-5	-5	-5	-5	-5	-5	-5
7	-4	-4	-4	-4	-4	-4	-4
8	0	1	1	1	1	1	
9	-3	-3	-3	-3	-3		
Matrix D							

4	0	3	3	3	3	3	3
5	0	3	3	3	3	3	3
6	0	3	3	3	3	3	3
7	0	3	3	3	3	3	3
8	0	3	2	2	2	2	
9	0	3	3	3	3		
Matrix C							

Figure 4-6: Concept of Parallelization, Cycle 8



4	-5	-5	-5	-5	-5	-5	-5
5	-5	0	1	1	1	1	1
6	-5	-3	-3	-3	-3	-3	-3
7	-5	-4	-3	-2	-2	-2	-2
8	-5	0	5	6	6	6	6
9	-5	-3	-1	-1	-1	-1	-1

Matrix I

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4	-4	-4	-4	-4	-4
6	-5	-5	-5	-5	-5	-5	-5
7	-4	-4	-4	-4	-4	-4	-4
8	0	1	1	1	1	1	1
9	-3	-3	-3	-3	-3	-3	-3

Matrix D

4	0	3	3	3	3	3	3
5	0	3	3	3	3	3	3
6	0	3	3	3	3	3	3
7	0	3	3	3	3	3	3
8	0	3	2	2	2	2	2
9	0	3	3	3	3	3	3

Matrix C

Figure 4-7: Concept of Parallelization, Cycle 9

4	-5	-5	-5	-5	-5	-5	-5
5	-5	0	1	1	1	1	1
6	-5	-3	-3	-3	-3	-3	-3
7	-5	-4	-3	-2	-2	-2	-2
8	-5	0	5	6	6	6	6
9	-5	-3	-1	-1	-1	-1	-1

Matrix I

4	-5	-5	-5	-5	-5	-5	-5
5	-4	-4	-4	-4	-4	-4	-4
6	-5	-5	-5	-5	-5	-5	-5
7	-4	-4	-4	-4	-4	-4	-4
8	0	1	1	1	1	1	1
9	-3	-3	-3	-3	-3	-3	-3

Matrix D

4	0	3	3	3	3	3	3
5	0	3	3	3	3	3	3
6	0	3	3	3	3	3	3
7	0	3	3	3	3	3	3
8	0	3	2	2	2	2	2
9	0	3	3	3	3	3	3

Matrix C

Figure 4-8: Concept of Parallelization, Cycle 10

The proposed method allows for some level of parallelism while not violating the data dependencies. However, the inter-matrix dependencies are still present. Thus, it is expected that for each iteration, the I and D matrices are evaluated before the C matrix.

4.5 Traceback

Before traceback, the scores from the C matrix need to be separated into three matrices. This makes sure that the algorithm can track where the sequence aligns and where the gaps exist due to either insertions or deletions. The traceback starts by identifying the highest score across the three matrices. This score represents the best possible endpoint for the alignment. From here, the traceback will work backward through the matrices. The algorithm will check the diagonal, left, and up scores to determine the next step. Diagonal matches indicate a match or mismatch, left matches indicate an insertion, and up matches indicate a deletion. The diagonal scores will be from the three matrices which will indicate if a frameshift is needed to match or get the highest score among the frames.

		M	A	S	H	C	H	C	T
M	0	0	0	0	0	0	0	0	0
H	0	5	0	0	0	0	0	0	0
C	0	0	3	0	8	3	9	4	2
A	0	0	0	2	3	17	12	18	13
T	0	0	0	1	1	12	15	13	23
S	0	0	4	1	1	12	15	13	18
V	0	1	0	2	0	8	13	12	26
H	0	0	0	0	10	6	19	14	21
T	0	0	0	0	1	5	9	14	18

FRAME 1

		M	A	S	H	C	H	C	T
C	0	0	0	0	0	0	0	0	0
T	0	5	0	0	0	5	0	5	0
A	0	0	1	2	0	0	3	0	10
L	0	0	4	2	0	4	0	5	5
Q	0	2	0	2	0	6	10	16	13
C	0	0	1	0	2	1	16	11	25
T	0	0	0	0	0	11	11	25	20
	0	0	0	1	0	6	9	20	30
	0	0	0	0	0	0	0	0	0

FRAME 2

		M	A	S	H	C	H	C	T
A	0	0	0	0	0	0	0	0	0
L	0	0	5	0	0	0	0	0	0
H	0	2	0	3	0	3	0	4	2
C	0	0	0	0	0	11	6	21	16
S	0	0	0	0	6	20	16	30	25
A	0	0	1	4	4	15	19	25	31
H	0	0	4	2	2	13	14	23	25
	0	0	0	3	10	11	21	21	24
	0	0	0	0	0	0	0	0	0

FRAME 3

MAX SCORE = 30



Figure 4-9: Finding the Max Score from three matrices

	M	A	S	H	C	H	C	T
M	0	0	0	0	0	0	0	0
H	0	5	0	0	0	0	0	0
C	0	0	0	2	3	17	12	18
T	0	0	0	1	1	12	15	13
A	0	0	4	1	1	12	15	13
V	0	1	0	2	0	8	13	12
H	0	0	0	0	10	6	19	14
T	0	0	0	1	5	9	14	18

FRAME 1

	M	A	S	H	C	H	C	T
C	0	0	0	0	0	0	0	0
T	0	5	0	0	0	5	0	5
A	0	0	1	2	0	0	3	0
L	0	0	4	2	0	4	0	5
Q	0	2	0	2	0	6	10	16
C	0	0	1	0	2	1	16	11
T	0	0	0	0	0	11	11	25
C	0	0	0	1	0	6	9	20
T	0	0	0	0	0	0	0	0

FRAME 2

	M	A	S	H	C	H	C	T
A	0	0	0	0	0	0	0	0
L	0	0	5	0	0	0	0	0
H	0	2	0	3	0	3	0	4
C	0	0	0	0	6	20	16	30
S	0	0	1	4	4	15	19	25
A	0	0	4	2	2	13	14	23
H	0	0	0	3	10	11	21	21
T	0	0	0	0	0	0	0	0

FRAME 3

Figure 4-10: Traceback Process Cycle 1-2

	M	A	S	H	C	H	C	T
M	0	0	0	0	0	0	0	0
H	0	5	0	0	0	0	0	0
C	0	0	3	0	8	3	9	4
T	0	0	0	2	3	17	12	18
A	0	0	0	1	1	12	15	13
V	0	0	4	1	1	12	15	13
H	0	1	0	2	0	8	13	12
T	0	0	0	0	10	6	19	14
T	0	0	0	1	5	9	14	18

FRAME 1

	M	A	S	H	C	H	C	T
C	0	0	0	0	0	0	0	0
T	0	5	0	0	0	5	0	5
A	0	0	1	2	0	0	3	0
L	0	0	4	2	0	4	0	5
Q	0	2	0	2	0	6	10	16
C	0	0	1	0	2	1	16	11
T	0	0	0	0	0	11	11	25
T	0	0	0	1	0	6	9	20
T	0	0	0	0	0	0	0	0

FRAME 2

	M	A	S	H	C	H	C	T
A	0	0	0	0	0	0	0	0
L	0	0	5	0	0	0	0	0
H	0	2	0	3	0	3	0	4
C	0	0	0	0	6	20	16	30
S	0	0	1	4	4	15	19	25
A	0	0	4	2	2	13	14	23
H	0	0	0	3	10	11	21	21
T	0	0	0	0	0	0	0	0

FRAME 3

Figure 4-11: Traceback Process Cycle 3-7. Moved to the other matrix indicating a shift

	M	A	S	H	C	H	C	T
M	0	0	0	0	0	0	0	0
H	0	5	0	0	0	0	0	0
C	0	0	3	0	8	3	9	4
T	0	0	0	2	3	17	12	18
A	0	0	0	1	1	12	15	13
V	0	0	4	1	1	12	15	13
H	0	1	0	2	0	8	13	12
T	0	0	0	0	10	6	19	14
T	0	0	0	1	5	9	14	18

FRAME 1

	M	A	S	H	C	H	C	T
C	0	0	0	0	0	0	0	0
T	0	5	0	0	0	5	0	5
A	0	0	1	2	0	0	3	0
L	0	0	4	2	0	4	0	5
Q	0	2	0	2	0	6	10	16
C	0	0	1	0	2	1	16	11
T	0	0	0	0	0	11	11	25
T	0	0	0	1	0	6	9	20
T	0	0	0	0	0	0	0	0

FRAME 2

	M	A	S	H	C	H	C	T
A	0	0	0	0	0	0	0	0
L	0	0	5	0	0	0	0	0
H	0	2	0	3	0	3	0	4
C	0	0	0	0	6	20	16	30
S	0	0	1	4	4	15	19	25
A	0	0	4	2	2	13	14	23
H	0	0	0	3	10	11	21	21
T	0	0	0	0	0	0	0	0

FRAME 3

Figure 4-12: Traceback Process Cycle END

The algorithm can also detect frameshifts by checking if the path moves from one frame to another. An asterisk '*' is used to indicate a frameshift, while a plus sign '+' is used when the alignment continues in the same frame. As the traceback continues, it builds two aligned sequences (*seq1* and *seq2*). The first sequence (*seq1*) represents the DNA sequence translated to amino acids, with gaps added. On the other hand, the second sequence (*seq2*) represents the reference protein, with gaps to align properly with *seq1*. The traceback will continue until it encounters a cell with a score of zero, indicating the end of that region. This means that the best possible alignment has been identified.



5. System Implementation

This section showcases the complete system implementation of TARA CUDA. It discusses the design and development process for reading input data, configuring GPU execution, managing memory allocation, computing alignment matrices, and executing traceback to get the optimal alignment.

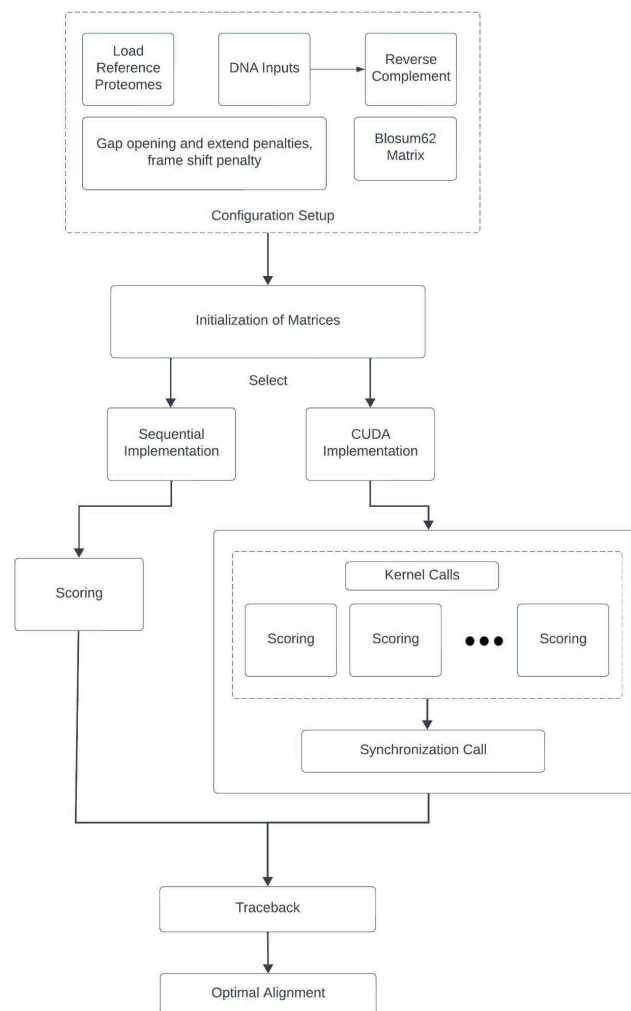


Figure 5-1: Complete System Diagram of TARA CUDA



5.1 Data Gathering & Dataset

The datasets used in this research consist of protein and DNA sequences for *Drosophila melanogaster* (fruit fly) and an additional dataset, which is *Arabidopsis thaliana* (mouse-ear cress), provided in FASTA format. The protein sequences were obtained from UniProt under the proteome IDs UP000000803 and UP000006548.

The protein sequences were obtained through direct download from the UniProt database. However, unlike the protein sequences, to obtain the corresponding DNA sequences for each protein entry, a web-crawling script was developed using Python with the Selenium library to hasten the data gathering. This data gathering automation enabled easier and faster retrieval of DNA sequences and their corresponding headers. Additionally, to further improve efficiency, the script was optimized with multithreading capabilities, allowing multiple data extraction tasks to be performed concurrently. As a result, the data collection process was accelerated and made more scalable for larger datasets.

Furthermore, a preprocessing phase was implemented. During this phase, a test was conducted to validate the alignment between DNA and protein sequences, verifying that:

- Each DNA sequence is aligned to its corresponding protein sequence,
- No missing or null values existed within the datasets.
- All sequence pairs were complete and valid.

Index	Protein ID	DNA ID	
0	A0A0B4K692	A0A0B4K692	TRUE
1	A0A0B4K753	A0A0B4K753	TRUE
2	A0A0B4K7J2	A0A0B4K7J2	TRUE
3	A0A0B4K859	A0A0B4K859	TRUE
4	A0A0B4KEE4	A0A0B4KEE4	TRUE
5	A0A0B4KGY6	A0A0B4KGY6	TRUE
6	A0A0B4LFY9	A0A0B4LFY9	TRUE
7	A0A0B7P9G0	A0A0B7P9G0	TRUE
8	A0A0U1QT59	A0A0U1QT59	TRUE
9	A0A126GUP6	A0A126GUP6	TRUE
10	A0A1F4	A0A1F4	TRUE

Figure 5-2: Data Preprocessing: Validation and Completeness Check in Fruit Fly Dataset



Index	Protein ID	DNA ID	
0	A0A0A7EPL0	A0A0A7EPL0	TRUE
1	A0A140JWM8	A0A140JWM8	TRUE
2	A0A178U9T0	A0A178U9T0	TRUE
3	A0A178VBJ0	A0A178VBJ0	TRUE
4	A0A178VE74	A0A178VE74	TRUE
5	A0A178VEK7	A0A178VEK7	TRUE
6	A0A178WF56	A0A178WF56	TRUE
7	A0A1I9LM04	A0A1I9LM04	TRUE
8	A0A1I9LMX5	A0A1I9LMX5	TRUE
9	A0A1I9LN01	A0A1I9LN01	TRUE
10	A0A1I9LQ12	A0A1I9LQ12	TRUE

Figure 5-3: Data Preprocessing: Validation and Completeness Check in Mouse Ear Cress Dataset

Any sequences found without a matching pair or containing invalid or missing data were dropped from the final dataset. This preprocessing ensured that only aligned and valid sequence pairs were used for this research.

After preprocessing and validation, the final datasets that will be used consist of valid and fully aligned protein and DNA sequence pairs for *Drosophila melanogaster* (fruit fly) and *Arabidopsis thaliana* (mouse-ear cress). The fruit fly dataset contained a total of 21,449 protein sequences, each paired with its corresponding DNA translation. This dataset displayed a wide range in base pair lengths, with the shortest protein consisting of just 11 bp and the longest extending up to 22,949 bp. The average protein length in this dataset was approximately 451 bp. On the other hand, the mouse-ear cress dataset consisted of 39,067 protein sequences. Though it had a higher number of entries than the fruit fly dataset, the protein sequences in the mouse-ear cress dataset were generally shorter in comparison. The lengths ranged from as short as 5 bp to a maximum of 5,400 bp, with an average protein length of roughly 396 bp.

5.2 Input Processing

The program will take two inputs: the DNA read sequences and the reference proteome dataset. Both inputs are provided in FASTA format, a standard text-based format for representing nucleotide or protein sequences. Upon reading, the implementation parses the FASTA files to extract the sequence data and corresponding headers.



```
Input file name for protein(exclude .fasta/.fastq):  
--> testProt  
Input file name for DNA (exclude .fasta/.fastq):  
--> testDNA
```

Figure 5-4: System Prompt for DNA Sequence and Proteome Input

```
vector<string> readFastaSequences(const string& filename) {  
    ifstream file("./Resource Files/" + filename + ".fasta");  
    vector<string> sequences;  
    string line, sequence;  
  
    if (!file) {  
        cerr << "Error: Unable to open file " << filename << endl;  
        return sequences;  
    }  
  
    while (getline(file, line)) {  
        if (line.empty()) continue;  
        if (line[0] == '>') {  
            if (!sequence.empty()) {  
                sequences.push_back(sequence);  
                sequence.clear();  
            }  
        }  
        else {  
            sequence += line;  
        }  
    }  
  
    if (!sequence.empty()) {  
        sequences.push_back(sequence);  
    }  
  
    file.close();  
  
    return sequences;  
}
```

Figure 5-5: Code Snippet of FASTA Sequence Read Function

```
vector<string> readFastaIDs(const string& filename) {  
    ifstream file("./Resource Files/" + filename + ".fasta");  
    vector<string> sequenceIDs;  
    string line;
```



```
if (!file) {
    cerr << "Error: Unable to open file " << filename << endl;
    return sequenceIDs;
}

while (getline(file, line)) {
    if (!line.empty() && line[0] == '>') {
        sequenceIDs.push_back(line.substr(1)); // Remove '>' and
store only the ID
    }
}
return sequenceIDs;
}
```

Figure 5-6: Code Snippet: FASTA Header (ID) Extraction Function

After, the program prompts the user to choose between the Sequential or CUDA (Parallel) implementation of the selected algorithm.

```
Input file name for protein(exclude .fasta/.fastq):
--> testProt
Input file name for DNA (exclude .fasta/.fastq):
--> testDNA
Choose mode [0 - Sequential] [1 - CUDA]:
--> 1
```

Figure 5-7: System Prompt for Algorithm Implementation Type

After choosing whether the sequential or parallel version will be used, the implementation will prompt the user to choose the frame count (three frame or six frame) that the system will use.

```
Input file name for protein(exclude .fasta/.fastq):
--> testProt
Input file name for DNA (exclude .fasta/.fastq):
--> testDNA
Choose mode [0 - Sequential] [1 - CUDA]:
--> 1
Choose frame count [3 - 3 Frame] [6 - 6 Frame]:
--> 6
```

Figure 5-8: System Prompts for Three-Frame or Six-Frame Selection



If the user chooses the three frame option, three frame alignment is performed on the forward strand of the input. On the other hand, selecting the six frame option prompts the program to also perform a three frame alignment on the reverse complement of the input. The reference proteome is read as an array of protein sequences. Each DNA read sequence (and its reverse complement if six frame is selected) is aligned to each of the protein sequences to identify matches or similarities.

5.3 Translation & BLOSUM62

Additionally, instead of using hash tables for codon-to-amino acid conversion and BLOSUM62 indexing (as discussed in Chapter 4.2), the system now uses if-else statements to perform codon-to-protein translation, and a char-to-int array lookup table for mapping amino acids to their corresponding BLOSUM62 matrix indices. This approach reduces the overhead associated with hash table access, especially during matrix scoring, where minimizing memory latency and branching is crucial. These modifications contribute to faster score computation and ensure consistent behavior between the CUDA and sequential implementations of the algorithm.

```
__host__ __device__ char d_DNA_to_Protein(const char* dna_seq, int
dna_index_1, int dna_index_2, int dna_index_3) {
    char codon[4] = " ";
    codon[0] = dna_seq[dna_index_1];
    codon[1] = dna_seq[dna_index_2];
    codon[2] = dna_seq[dna_index_3];
    codon[3] = '\0';

    if (codon[0] == 'T') {
        if (codon[1] == 'T') {
            if (codon[2] == 'T' || codon[2] == 'C') return 'F';
            if (codon[2] == 'A' || codon[2] == 'G') return 'L';
        }
        else if (codon[1] == 'C') {
            return 'S';
        }
        else if (codon[1] == 'A') {
            if (codon[2] == 'T' || codon[2] == 'C') return 'Y';
            if (codon[2] == 'A' || codon[2] == 'G') return 'X';
        }
        else if (codon[1] == 'G') {
            if (codon[2] == 'T' || codon[2] == 'C') return 'C';
            if (codon[2] == 'G') return 'W';
            if (codon[2] == 'A') return 'X';
        }
    }
}
```



```
else if (codon[0] == 'C') {
    if (codon[1] == 'T') return 'L';
    if (codon[1] == 'C') return 'P';
    if (codon[1] == 'A') {
        if (codon[2] == 'T' || codon[2] == 'C') return 'H';
        if (codon[2] == 'A' || codon[2] == 'G') return 'Q';
    }
    else if (codon[1] == 'G') return 'R';
}
else if (codon[0] == 'A') {
    if (codon[1] == 'T') {
        if (codon[2] == 'A' || codon[2] == 'C' || codon[2] ==
'T') return 'I';
        if (codon[2] == 'G') return 'M';
    }
    if (codon[1] == 'C') return 'T';

    if (codon[1] == 'A') {
        if (codon[2] == 'T' || codon[2] == 'C') return 'N';
        if (codon[2] == 'G' || codon[2] == 'A') return 'K';
    }

    if (codon[1] == 'G') {
        if (codon[2] == 'T' || codon[2] == 'C') return 'S';
        if (codon[2] == 'A' || codon[2] == 'G') return 'R';
    }
}
else if (codon[0] == 'G') {
    if (codon[1] == 'T') return 'V';
    if (codon[1] == 'C') return 'A';
    if (codon[1] == 'A') {
        if (codon[2] == 'T' || codon[2] == 'C') return 'D';
        if (codon[2] == 'G' || codon[2] == 'A') return 'E';
    }
    if (codon[1] == 'G') return 'G';
}
return ' ';
}
```

Figure 5-9: Code Snippet of a Series of if-else Statements for Codon-to-Protein Translation

```
void initialize_protein_to_int_map() {
    vector<int> host_map(128, -1);
}
```



```
host_map['A'] = 0; host_map['R'] = 1; host_map['N'] = 2;
host_map['D'] = 3; host_map['C'] = 4; host_map['Q'] = 5;
host_map['E'] = 6; host_map['G'] = 7; host_map['H'] = 8;
host_map['I'] = 9; host_map['L'] = 10; host_map['K'] = 11;
host_map['M'] = 12; host_map['F'] = 13; host_map['P'] = 14;
host_map['S'] = 15; host_map['T'] = 16; host_map['W'] = 17;
host_map['Y'] = 18; host_map['V'] = 19; host_map['B'] = 20;
host_map['Z'] = 21; host_map['X'] = 22; host_map['U'] = 23;

for (int i = 0; i < 128; ++i)
    prot_to_idx[i] = host_map[i];

checkCudaErrors(cudaMemcpyToSymbol(d_prot_to_idx,
host_map.data(), sizeof(int) * host_map.size(), 0,
cudaMemcpyHostToDevice));
}
```

Figure 5-10: Code Snippet of Array Lookup of BLOSUM62

5.4 Matrix Initialization

Before matrix scoring can be done, the scoring (*sc_mat*), insertion (*ins_mat*), and deletion (*del_mat*) matrices, along with their corresponding traceback matrices (*t_sc_mat*, *t_ins_mat*, *t_del_mat*), must be initialized on the host side. The first few rows and columns of the matrices are initialized as shown in Equation 4.1. This preparation ensures that the algorithm's data dependencies are fulfilled before matrix scoring is performed.

```
void init_local_v2(...) {
    for (size_t i = 0; i < N; i++) {
        ins_mat[i * M + 0] = infn;
        t_ins_mat[i * M + 0] = infn;
    }

    for (size_t i = 0; i < N; i++) {
        for (size_t j = 0; j < M; j++) {
            if (i == 0 || j == 0) {
                sc_mat[i * M + j] = 0;
                t_sc_mat[i * M + j] = 0;
            }
        }
    }

    for (size_t j = 0; j < M; j++) {
        del_mat[0 * M + j] = infn;
        del_mat[2 * M + j] = infn;
    }
}
```



```
del_mat[3 * M + j] = infn;
del_mat[1 * M + j] = sc_mat[0 * M + j] - gop - gep;

t_del_mat[0 * M + j] = infn;
t_del_mat[2 * M + j] = infn;
t_del_mat[3 * M + j] = infn;
t_del_mat[1 * M + j] = 1;
}

for (size_t i = 4; i < N; i++) {
    del = del_mat[(i - 3) * M + 0] - gep;
    xscore = sc_mat[(i - 3) * M + 0] - gop - gep;
    del_mat[i * M + 0] = (del > xscore) ? del : xscore;
}

for (size_t i = 0; i < 4; i++) {
    for (size_t j = 1; j < M; j++) {
        insert = ins_mat[i * M + (j - 1)] - gep;
        xscore = sc_mat[i * M + (j - 1)] - gop - gep;
        prot_seq = d_DNA_to_Protein(input_seq, i - 1, i, i + 1);

        if (insert > xscore) {
            ins_mat[i * M + j] = insert;
            t_ins_mat[i * M + j] = 0;
        }
        else {
            ins_mat[i * M + j] = xscore;
            t_ins_mat[i * M + j] = 1;
        }

        insert = ins_mat[i * M + j];
        del = del_mat[i * M + j];

        if (i == 1) {
            xscore = sc_mat[0 * M + (j - 1)] + score(prot_seq,
ref_seq[j - 1]);
            if (insert >= del && insert >= xscore) {
                sc_mat[i * M + j] = insert;
                t_sc_mat[i * M + j] = -2;
            }
            else if (del >= insert && del >= xscore) {
                sc_mat[i * M + j] = del;
                t_sc_mat[i * M + j] = -1;
            }
            else {
                sc_mat[i * M + j] = xscore;
                t_sc_mat[i * M + j] = 1;
            }
        }
    }
}
```




```
    }
    else if (i == 2) {
        xscore = sc_mat[0 * M + (j - 1)] + score(prot_seq,
ref_seq[j - 1]) - shift;
        if (insert > xscore) {
            sc_mat[i * M + j] = insert;
            t_sc_mat[i * M + j] = -2;
        }
        else {
            sc_mat[i * M + j] = xscore;
            t_sc_mat[i * M + j] = 2;
        }
    }
    else if (i == 3) {
        xscore = sc_mat[1 * M + (j - 1)] + score(prot_seq,
ref_seq[j - 1]) - shift;
        if (insert > xscore) {
            sc_mat[i * M + j] = insert;
            t_sc_mat[i * M + j] = -2;
        }
        else {
            sc_mat[i * M + j] = xscore;
            t_sc_mat[i * M + j] = 2;
        }
    }

    if (sc_mat[i * M + j] < 0)
        sc_mat[i * M + j] = 0;
}
}
```

Figure 5-11: Code Snippet of Matrix Initialization Process

5.5 Sequential Implementation

In preparation for the implementation of the CUDA parallelization of Zhang's Six Frame Algorithm, a sequential version was implemented. This version served two purposes: to validate the algorithm logic and visualize the data dependencies, and to provide a base implementation to compare the performance gains (speed-up) achieved by the parallel implementation.

After the matrix initialization discussed in Chapter 5.4, the matrices are filled using recurrence relations defined in Equation 4-3. The algorithm starts from the 4th position ($i=4$) up to the end of the DNA sequence. This process is repeated across all reading



frames. For three frame alignment, the forward strand of the DNA input is used, and the scoring function is executed. Six frame alignment, however, uses both the forward strand and reverse complement of the DNA input, which means the scoring function is executed twice sequentially.

```
void scoring_local_v2(...) {
    for (int i = 4; i < N; i++) {
        for (int j = 1; j < M; j++) {
            prot_seq = d_DNA_to_Protein(input_seq, i - 1, i, i + 1);
            scoring = score(prot_seq, ref_seq[j - 1]);
            insert = ins_mat[i * M + (j - 1)] - gep;
            xscore = sc_mat[i * M + (j - 1)] - gop - gep;

            ins_mat[i * M + j] = (insert > xscore) ? insert :
xscore;
            t_ins_mat[i * M + j] = (insert > xscore) ? 0 : 1;

            del = del_mat[(i - 3) * M + j] - gep;
            xscore = sc_mat[(i - 3) * M + j] - gop - gep;

            del_mat[i * M + j] = (del > xscore) ? del : xscore;
            t_del_mat[i * M + j] = (del > xscore) ? 0 : 1;

            if (i < N - 1) {
                insert = ins_mat[i * M + j];
                del = del_mat[i * M + j];
                sc_1 = sc_mat[(i - 2) * M + (j - 1)] + scoring -
shift;
                sc_2 = sc_mat[(i - 3) * M + (j - 1)] + scoring;
                sc_3 = sc_mat[(i - 4) * M + (j - 1)] + scoring -
shift;

                if (insert >= del && insert >= sc_1 && insert >=
sc_2 && insert >= sc_3) {
                    sc_mat[i * M + j] = insert;
                    t_sc_mat[i * M + j] = -2;
                }
                else if (del >= insert && del >= sc_1 && del >= sc_2
&& del >= sc_3) {
                    sc_mat[i * M + j] = del;
                    t_sc_mat[i * M + j] = -1;
                }
                else if (sc_2 >= insert && sc_2 >= del && sc_2 >=
sc_1 && sc_2 >= sc_3) {
                    sc_mat[i * M + j] = sc_2;
                    t_sc_mat[i * M + j] = 3;
                }
            }
        }
    }
}
```



```
        }
        else if (sc_1 >= insert && sc_1 >= del && sc_1 >=
sc_2 && sc_1 >= sc_3) {
            sc_mat[i * M + j] = sc_1;
            t_sc_mat[i * M + j] = 2;
        }
        else if (sc_3 >= insert && sc_3 >= del && sc_3 >=
sc_1 && sc_3 >= sc_2) {
            sc_mat[i * M + j] = sc_3;
            t_sc_mat[i * M + j] = 4;
        }
    }

    if (sc_mat[i * M + j] < 0) {
        sc_mat[i * M + j] = 0;
    }

    if (i == N - 1) {
        ins_mat[i * M + j] = infn;
        sc_mat[i * M + j] = 0;
        t_sc_mat[i * M + j] = infn;
    }
}
}
```

Figure 5-12: Code Snippet of Sequential Scoring Process

Furthermore, to ensure fair comparison between the sequential and CUDA implementations, the structure of the function used in scoring was kept as consistent as possible in both versions. This minimizes algorithm bias and allows the evaluation to highlight the effectiveness of the implementation scheme and parallelism of the execution model rather than differences in the base algorithm implementation.

5.6 CUDA Implementation

This subsection tackles how an implementation scheme for Zhang's Six Frame algorithm was created to fit the parallel framework of CUDA's SIMT model. It outlines how the scoring matrices were divided into multiple blocks, how memory management was handled, and how the scoring matrix computations were parallelized. The researchers utilized a hybrid multi-level anti-diagonal wavefront implementation, which combined a host-side loop with submatrix kernel launches that utilize in-kernel block synchronization.



5.6.1 CUDA Error Handling

In order to ensure proper CUDA programming, a CUDA error checking utility macro *checkCudaErrors(val)* was implemented, shown in Figure 5-13. The macro wraps individual CUDA API calls to catch errors and halt execution, avoiding incorrect CUDA calls.

```
#define checkCudaErrors(val) check( (val), #val, __FILE__, __LINE__ )
template<typename T>
void check(T err, const char* const func, const char* const file,
const int line)
{
    if (err != cudaSuccess) {
        fprintf(stderr, "CUDA error at: %s : %d\n", file, line);
        fprintf(stderr, "%s %s\n", cudaGetErrorString(err), func);
        exit(1);
    }
}
```

Figure 5-13: Code Snippet of CUDA Error Checking

5.6.2 Grid and Block Configuration

Given the data-dependent nature of the algorithm, as well as the hybrid approach taken towards the parallelization of the algorithm, grid and block size configuration cannot be determined by using the previously mentioned *cuOccupancyMaxPotentialBlockSize()* function. Instead, block size was maximized to 1024 (32 x 32) in order to utilize the implemented in-kernel block synchronization across the largest possible submatrix size. Grid size, on the other hand, is dynamically determined within each iteration of the host-side loop, calculating the number of submatrices in the current antidiagonal of the loop, as seen in Figure 5-14. The relative positions of the edge submatrices of the current anti-diagonal are used to determine the number of submatrices in each anti-diagonal, setting the grid size accordingly; otherwise, the current anti-diagonal is either the first or the last anti-diagonal and thus contains only one submatrix.

```
unsigned int curr_diag_blocks = 0;
if (last_sub_y_curr_diag >= first_sub_y_curr_diag)
    curr_diag_blocks = last_sub_y_curr_diag - first_sub_y_curr_diag + 1;
dim3 currGridDim(1, 1);

if (curr_diag_blocks > 0)
    currGridDim = dim3(curr_diag_blocks, 1);
```

Figure 5-14: Code Snippet of Dynamic Grid Size Allocation



5.6.3 Memory Management

To allow the alignment algorithm to be executed on the GPU, the data required needs to be accessible by the device. First, memory is allocated on the device using “*cudaMalloc*” for the DNA sequence (“*d_DNA_sequence*”), its reverse complement (“*d_DNA_sequence_r*”), and the reference protein sequence (“*d_protein_sequence*”). The input sequences are then transferred from the host to the device using “*cudaMemcpy*”, ensuring the device can access the required input sequence data.

```
checkCudaErrors(cudaMalloc(&d_DNA_sequence, N_size));
checkCudaErrors(cudaMalloc(&d_protein_sequence, M_size));
checkCudaErrors(cudaMalloc(&d_DNA_sequence_r, N_size));
checkCudaErrors(cudaMemcpy(d_DNA_sequence, c_DNA_sequence, N_size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_protein_sequence, c_protein_sequence,
M_size, cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_DNA_sequence_r, c_DNA_sequence_r,
N_size, cudaMemcpyHostToDevice));
```

Figure 5-15: Code Snippet of Input Sequence Initialization & Host-to-Device Memory Transfer

Additionally, the scoring (“*d_sc_mat*”), insertion (“*d_ins_mat*”), deletion (“*d_del_mat*”), and their corresponding traceback matrices (“*d_t_sc_mat*”, “*d_t_ins_mat*”, and “*d_t_del_mat*”) are also allocated using “*cudaMalloc*”.

```
checkCudaErrors(cudaMalloc(&d_sc_mat, size));
checkCudaErrors(cudaMalloc(&d_ins_mat, size));
checkCudaErrors(cudaMalloc(&d_del_mat, size));

checkCudaErrors(cudaMalloc(&d_t_sc_mat, size));
checkCudaErrors(cudaMalloc(&d_t_ins_mat, size));
checkCudaErrors(cudaMalloc(&d_t_del_mat, size));
...
```

Figure 5-16: Code Snippet of CUDA Memory Allocation for Alignment Matrices

Unlike in Chapter 3.2, “*cudaMalloc*” was chosen over “*cudaMallocManaged*” to give the programmers explicit control over the management of data transfers, increasing performance of the program, since unified memory relies on the CUDA runtime to handle data migration between host and device automatically, introducing overhead, particularly when it comes to page faulting. Given the straightforward nature of the program flow; that is, initializing data on the host, transferring the data to the device, executing the scoring kernels, and finally returning the data to the host for printing and



traceback, using “*cudaMalloc*” offers better performance in such cases where data access patterns are simple and predictable, as it avoids the overhead associated with unified memory management and allows the GPU to access the data directly from device memory without additional implicit migration steps.

To prevent garbage values, all matrices are initialized to zero using “*cudaMemset*” to avoid any undefined behavior during computation. “*cudaMemcpy*” calls follow, allowing the data on the previously initialized host matrices to be present on the device for correct scoring.

```
checkCudaErrors(cudaMemset(d_sc_mat, 0, size));
checkCudaErrors(cudaMemset(d_ins_mat, 0, size));
checkCudaErrors(cudaMemset(d_del_mat, 0, size));
checkCudaErrors(cudaMemset(d_t_sc_mat, 0, size));
checkCudaErrors(cudaMemset(d_t_ins_mat, 0, size));
checkCudaErrors(cudaMemset(d_t_del_mat, 0, size));
...

checkCudaErrors(cudaMemcpy(d_sc_mat, sc_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_ins_mat, ins_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_del_mat, del_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_t_sc_mat, t_sc_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_t_ins_mat, t_ins_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_t_del_mat, t_del_mat, size,
cudaMemcpyHostToDevice));
...
```

Figure 5-17: Code Snippet of Device Matrix Initialization

5.6.4 Matrix Parallelization

Once memory is allocated and the data is initialized on the device, scoring can now be performed. The scoring phase of the alignment is parallelized using CUDA by implementing a wavefront anti-diagonal wavefront strategy, as discussed in Section 4.4. This method enables fine-grained parallelism, where multiple threads operate concurrently on independent cells across the scoring matrix. Since each matrix cell depends only on previously computed values along earlier anti-diagonals, this strategy ensures correctness while maximizing parallel execution. The actual implementation combines a host-side loop with in-kernel block synchronization, dividing the overall matrices into tiles of sizes equal to the block size (32 x 32) as seen in Figure 5-18, due to



the block synchronization limitations of the kernel implementation (further discussed in Section 5.6.4.2).

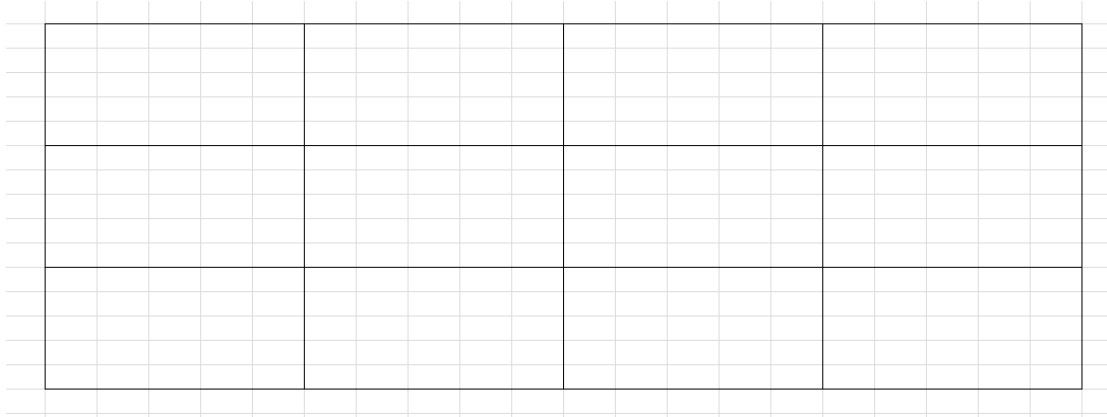


Figure 5-18: Sample Visualization of Submatrix Division

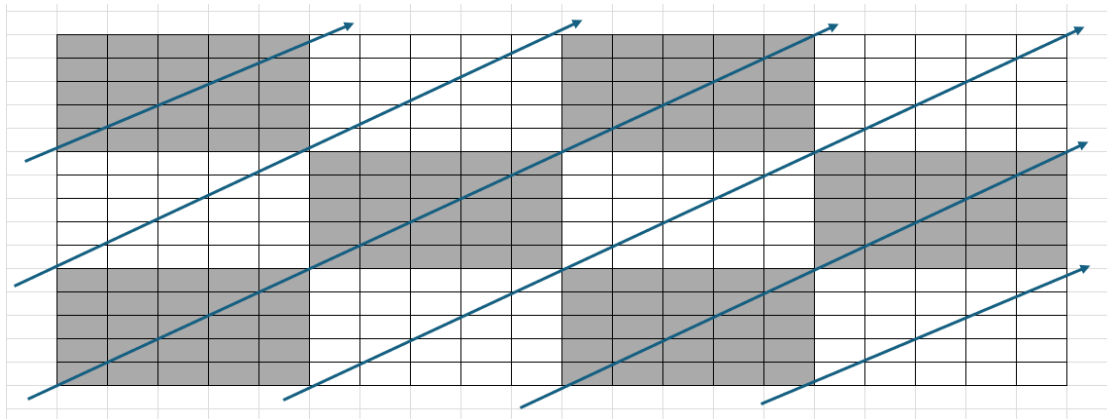


Figure 5-19: Sample Visualization of Macro-Level Matrix Anti-Diagonals

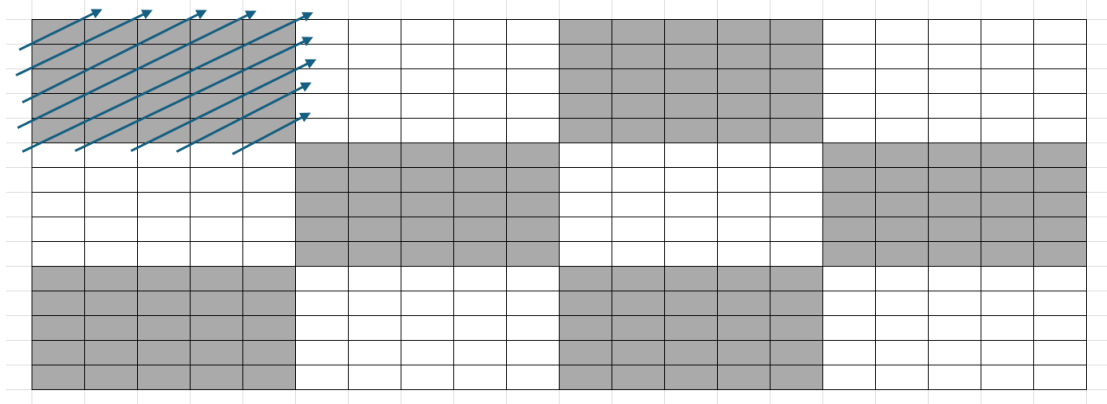


Figure 5-20: Sample Visualization of Submatrix-Level Anti-Diagonals

5.6.4.1 Host-Side Loop

The implementation utilizes a host-side loop that iterates through the tile-level anti-diagonals to manage the launching of kernels. This is seen in Figure 5-21. In each iteration of the loop, a CUDA kernel (*scoring_local_v2_cuda*) is launched to compute one full anti-diagonal of tiles across the matrices. For six frame alignment, the implementation utilized CUDA streams to enable the concurrent launching of the forward strand and reverse complement alignment kernel.

```
for (unsigned int diag = 0; diag < numSubmatrixRows + numSubmatrixCols - 1; ++diag) {
    unsigned int first_sub_y_curr_diag = std::max(0, (int)diag - (int)(numSubmatrixCols - 1));
    unsigned int last_sub_y_curr_diag = std::min((int)diag, (int)numSubmatrixRows - 1);

    //Grid dimension calculation

    scoring_local_v2_cuda <<<GridDim, blockDim, 0, stream1 >>> (...);
    scoring_local_v2_cuda <<<GridDim, blockDim, 0, stream2 >>> (...);
}
```

Figure 5-21: Code Snippet of Host Side Loop for Macro Level Anti-diagonals

5.6.4.2 CUDA Kernels

The CUDA kernel is responsible for calculating the insertion, deletion, and substitution scores and updating the respective matrices in place. In addition to the scores, traversal



values that will be used in the traceback are also computed and stored. The kernel also utilizes a conditional anti-diagonal loop to ensure that only cells within the current anti-diagonal loop are calculated. Block synchronization calls are performed in every iteration of the loop to guarantee that threads within the anti-diagonal wait for each other and respect the data dependencies, preventing them from performing calculations with values that are not yet available. Grid-stride calculations are performed within the kernel to allow a single kernel to perform the scoring for all tiles in the current anti-diagonal. A visualization of this implementation is seen in Figure 5-22, and the implementation of this kernel is seen in Figure 5-23.

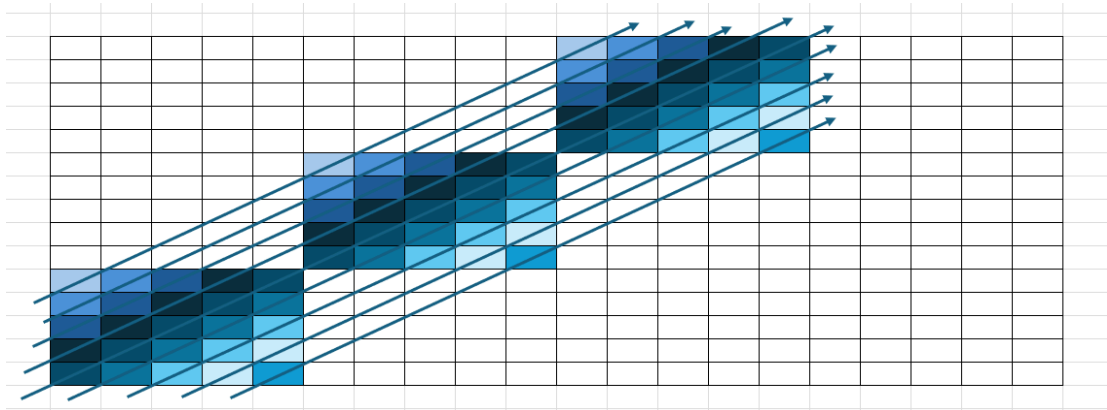


Figure 5-22: Sample Visualization of Grid-Stride Anti-Diagonal Computation in CUDA Kernel

```
__global__ void scoring_local_v2_cuda(...) {
    cg::thread_block block = cg::this_thread_block();

    unsigned int diagBlockIdx = blockIdx.x;

    unsigned int submatrixStartY = currDiagFirstSubY +
diagBlockIdx;
    unsigned int submatrixStartX = currDiag - submatrixStartY;

    int i = submatrixStartY * blockDim.y + threadIdx.y;
    int j = submatrixStartX * blockDim.x + threadIdx.x;

    int insert = 0;
    int del = 0;
    int xscore = 0;
    int sc_1 = 0, sc_2 = 0, sc_3 = 0;
    int scoring = 0;
    char prot_seq;
```



```
        if (i >= 4 && i < N && j >= 1 && j < M) {
            for (unsigned int sub_diag_in_tile = 0; sub_diag_in_tile
< 2 * submatrixSide - 1; ++sub_diag_in_tile) {
                if (((i - (submatrixStartY * blockDim.y)) + (j -
(submatrixStartX * blockDim.x))) == sub_diag_in_tile) {
                    if (i == N - 1) {
                        u_ins_mat[i * M + j] = infn;
                        u_sc_mat[i * M + j] = 0;
                        u_t_sc_mat[i * M + j] = infn;
                    }
                    else {
                        prot_seq = d_DNA_to_Protein(input_seq, i -
1, i, i + 1);

                        scoring = d_score(prot_seq, ref_seq[j - 1]);
                        insert = u_ins_mat[i * M + (j - 1)] - c_gep;
                        xscore = u_sc_mat[i * M + (j - 1)] - c_gop -
c_gep;

                        u_ins_mat[i * M + j] = (insert > xscore) ?
insert : xscore;
                        u_t_ins_mat[i * M + j] = (insert > xscore) ?
0 : 1;

                        del = u_del_mat[(i - 3) * M + j] - c_gep;
                        xscore = u_sc_mat[(i - 3) * M + j] - c_gop -
c_gep;

                        u_del_mat[i * M + j] = (del > xscore) ? del
: xscore;
                        u_t_del_mat[i * M + j] = (del > xscore) ? 0
: 1;

                        insert = u_ins_mat[i * M + j];
                        del = u_del_mat[i * M + j];
                        sc_1 = u_sc_mat[(i - 2) * M + (j - 1)] +
scoring - c_shift;
                        sc_2 = u_sc_mat[(i - 3) * M + (j - 1)] +
scoring;
                        sc_3 = u_sc_mat[(i - 4) * M + (j - 1)] +
scoring - c_shift;

                        int max_current_score;
                        int trace_value;

                        max_current_score = (insert >= del) ? insert
: del;
                        trace_value = (insert >= del) ? -2 : -1;
```



```
        if (sc_2 > max_current_score) {
            max_current_score = sc_2;
            trace_value = 3;
        }
        if (sc_1 > max_current_score) {
            max_current_score = sc_1;
            trace_value = 2;
        }
        if (sc_3 > max_current_score) {
            max_current_score = sc_3;
            trace_value = 4;
        }

        u_sc_mat[i * M + j] = (max_current_score <
0) ? 0 : max_current_score;
        u_t_sc_mat[i * M + j] = trace_value;
    }
    cg::sync(block);
}
}
```

Figure 5-23: Code Snippet of Device Matrix Initialization

5.7 Traceback

After scoring, the traceback is performed to recover the best alignment path sequentially. However, unlike the approach discussed in Section 4.5 that infers the path based on score comparison alone, this updated implementation uses traversal values found in the traceback matrix (t_sc_mat) during the scoring phase to guide the traceback. This method improves efficiency and ensures accurate path alignment. It will start at the index where the highest score was found in the scoring matrix (sc_mat).



-	-	-	-	-	-	-	-
-1	-1	-1	-2	-2	-2	-2	-2
-1	-1	-1	-1	1	-2	-2	-2
-1	-1	-1	4	2	-2	-2	-2
-1	-1	-1	-1	3	2	-2	3
3	-1	-1	3	4	2	-2	4
-1	-1	-1	-1	-1	3	2	2
-1	-1	-1	-1	3	3	3	-2
-1	-1	-1	3	-1	3	4	2
-1	2	-1	-1	-1	2	-2	2
-1	3	-1	-1	-1	3	-2	3
-1	-1	-1	3	-1	-1	2	4

Figure 5-24: Visualization of the Traceback Traversal Logic

The values in the traceback matrix indicate the direction and number of steps for the next traversal path.

- If the value is -2, it means that there is an insertion in the read sequence, so move 1 cell to the left.
- If the value is -1, it means that there is a deletion in the read sequence, so move 3 cells upward.
- If the value is 1, it indicates that it is at the last part of the traceback; therefore, move 1 cell to the left and 1 cell upward.
- If the value is 2, it means that there is a frameshift that occurs, so move 1 cell to the left and 2 cells upward.
- If the value is 3, it means that the traversal continues on the current frame, so move 1 cell to the left and 3 cells upward.
- If the value is 4, it means that there is a frameshift that occurs, so move 1 cell to the left and 4 cells upward.

```
void routine(int trace, int& i, int& j, char* str, char* ref_seq,
string& final_seq1, string& final_seq2, string& frameshift) {
    int k = 0;

    if (trace == 3) {
        i -= 3;
        j--;
        k = i + 3;
    }
}
```



```
        final_seq1 += d_DNA_to_Protein(str, k - 1, k, k + 1);
        final_seq2 += ref_seq[j];
        frameshift += " ";
    }
    else if (trace == -2) {
        j--;
        final_seq1 += "-";
        final_seq2 += ref_seq[j];
        frameshift += " ";
    }
    else if (trace == -1) {
        i -= 3;
        k = i + 3;
        final_seq1 += d_DNA_to_Protein(str, k - 1, k, k + 1);
        final_seq2 += "-";
        frameshift += " ";
    }
    else if (trace == 4) {
        i -= 4;
        j--;
        k = i + 4;
        final_seq1 += d_DNA_to_Protein(str, k - 1, k, k + 1);
        final_seq2 += ref_seq[j];
        frameshift += "*";
    }

    else if (trace == 2) {
        i -= 2;
        j--;
        k = i + 2;
        final_seq1 += d_DNA_to_Protein(str, k - 1, k, k + 1);
        final_seq2 += ref_seq[j];
        frameshift += "*";
    }
    else if (trace == 1) {
        i--;
        j--;
        k = i + 1;
        final_seq1 += d_DNA_to_Protein(str, k - 1, k, k + 1);
        final_seq2 += ref_seq[j];
        frameshift += " ";
    }
}
```

Figure 5-25: Code Snippet of the Traceback Traversal Logic

```
void traceV2_1d_check(char* input_seq, char* ref_seq, int* sc_mat,
```



```
int* t_sc_mat, size_t N, size_t M, int index, int* indexes) {
    N = (int)N;
    M = (int)M;
    int i_max = 0, j_max = 0, max_score = 0, curr_score, i= 0, j =0;
    string seq_dna, seq_prot, frameshift;

    for (i = 0; i < N; i++) {
        for (j = 0; j < M; j++) {
            curr_score = sc_mat[i * M + j];
            if (curr_score > max_score) {
                max_score = curr_score;
                i_max = i;
                j_max = j;
            }
        }
    }

    indexes[0] = max_score;
    indexes[1] = i_max;
    indexes[2] = j_max;

    i = i_max;
    j = j_max;

    myArray[index][0] = to_string(max_score);
    myArray[index][2] = to_string(j_max);

    while (sc_mat[i * M + j] != 0) {
        myArray[index][1] = to_string(j);
        indexes[3] = j;
        routine(t_sc_mat[i * M + j], i, j, input_seq, ref_seq,
        seq_dna, seq_prot, frameshift);
    }
}
```

Figure 5-26: Code Snippet of Traceback Process



6. Results and Analysis

This section discusses the results gathered from a series of tests conducted on the implemented system. Four main categories of testing were conducted: correctness, discussed in Chapter 6.1, runtime performance, discussed in Chapter 6.2, scalability in Chapter 6.3, and cross-platform performance in Chapter 6.4.

The implementation was developed on the Visual Studio 2022 Integrated Development Environment and used NVIDIA CUDA Toolkit 12.6. Visual Studio was configured to Release mode, and the majority of the settings are unchanged. The implementation used the shared CUDA runtime library and the Multi-Threaded (/MT) runtime library for the host. Optimization was set to Full Optimization (/Ox), and no additional compiler optimizations were specified; however, the different optimization levels available in Visual Studio, the nvcc compiler, as well as ptxas were examined in Section 6.3.2.

Tests were conducted across varying hardware platforms. The primary testing platform is a desktop computer with an Intel i7-12700F, a 12-core 20-thread desktop processor with 8 performance cores that support Hyper-Threading, clocked at 2.10 GHz base and turbo up to 4.90 GHz, and 4 efficient cores clocked at 1.60 GHz and turbo up to 3.60 GHz. This is paired with an NVIDIA RTX 4070 Super, specifically the Zotac Twin Edge OC variant, a high-end graphics card with 7168 CUDA cores clocked at 1980 MHz and turbo 2490 MHz, and 12 GB of GDDR6X memory clocked at 1313 MHz. This platform has 32 GB of DDR4 RAM at 3200 MT/s. The secondary testing platform is an ROG Strix G15 513 laptop with a Ryzen 7 5800H, an 8-core 16-thread mobile processor clocked at 3.20 GHz base and turbo up to 4.40 GHz. This is paired with an NVIDIA RTX 3050 Ti Mobile, a mobile GPU with 2560 CUDA Cores clocked at 735 MHz and turbo 1035 MHz, and 4 GB of GDDR6 memory clocked at 1500 MHz. This platform has 16 GB of DDR4 RAM at 3200 MT/s. The third testing platform is CCSCloud, a cloud-based Jupyter Environment with a QEMU Virtual CPU version 2.5+, a virtualizer that emulates an Intel Xeon Silver 4210 with 16 cores clocked at 2.20GHz base and turbo up to 3.20 GHz. This is paired with an NVIDIA Tesla V100 PCIe 32GB, a professional graphics card with 5120 CUDA cores clocked at 1245 MHz base and turbo up to 1380 MHz, and 16 GB of HBM2 memory clocked at 876 MHz. This platform has 32 GB of RAM.

The Implementation will also be compared to Zhang's Six Frame Algorithm in SIMD [14], a predecessor of this thesis. The SIMD implementation used an Advanced Vector Extensions 2 (AVX2) instruction set architecture. It is an SIMD extension provided by Intel, which operates on 256-bit registers (ymm registers) and enables simultaneous computation of multiple data elements. This allowed the implementation to process up



to 8 integers concurrently within each register, hence improving throughput compared to scalar execution. By leveraging AVX2 instructions and 256-bit registers, the SIMD version implemented a staircase-like processing strategy to maintain correctness while achieving speedups of approximately $3\times$ on average, as shown in their performance evaluations.

For additional testing, the researchers also used a fourth testing platform to test the SIMD program. This is due to platform incompatibility, where the researchers were unable to execute the program on systems with Windows 11, and only on systems with Windows 10. It was done on a desktop computer with a Ryzen 5 7600, a 6-core 12-thread desktop processor clocked at 3.8GHz base and turbo up to 5.10 GHz. This platform has 32 GB of DDR5 RAM at 6000 MT/s.

6.1 Correctness Test

To evaluate the accuracy of the algorithm, a Correctness Test was used to verify that it produces the expected output for a given set of inputs. The test involved two aligned datasets: one DNA sequence and its corresponding protein sequence translation, where each amino acid is directly derived from the DNA codon. A substring was then extracted from the DNA sequence and passed through the implemented system to test if it aligns with the correct index in the protein sequence (i.e., $\text{DNA}[i] \rightarrow \text{Protein}[i]$).

To thoroughly simulate different biological conditions and possible sequencing errors, four versions of the DNA substring were created:

- A Match: No changes were made to the original substring from the DNA dataset.

```
>OFFread30_match  
ATGATCACCGACGTGCAATTGGCCATTTTCTCCAACGTTCTGGGCGTATTCTGTTCTCTGCT  
GGTGGTTGCCTATCATTATATTAATGCCAATACCGAAAGCCAGTGCCAAGGCAAAATGA
```

Figure 6-1: DNA Substring (read30) - Match Case

- With Insertion: A random nucleotide (highlighted in yellow) was inserted into the substring to simulate a frameshift.



```
Dna Input v1
>1FFread30ins1
ATGATCACTCGACGTGCAATTGGCCATTTTCTCCAACGTTCTGAAGGCGTATTTCTGTTCCCTG
CTGGTGGTTGCCTATCATATATATTAATGCCAATACCGGAAAGCCCAAGTCCCAAGGCAATG
AATGA

Dna Input v2
>2FFread30ins2
ATGATCACCGACGTGCAATTAGGGCCATTTTCTCCAACGTTCTGGGCGTATTTCTGTTCCCT
GCTGGTGGTTGCCTATCATTATATTAATGCCAATACCGGAAATAGCCCAAGTCCCAAGGCAAA
ATGA
```

Figure 6-2: DNA Substring - With Insertion

- With Deletion: A nucleotide was removed (in white) to simulate another form of frameshift.

```
Protein Start: MITDVQLAIFSNVLGVFLFLLVYHYINANTGKPSAKAK

Dna [30]v1:
ATGATCACCGACGTGCAATTGGCCATTTTCTCCAACGTTCTGGGCGTATTTCTGTTCCCTGCT
GGTGGTTGCCTATCATTATATTAATGCCAATACCGGAAAGCCCAAGTCCCAAGGCAAAATGA

Input: 108
>3FFread30del1
ATGATCACCGACGTGCAATTGGCCATTTTCTCCAACGTTCTGGTATTTTCTGCTGGTGGT
TGCCTATCATTATATTAATGCCAATACCGGAAAGCCCAAGTCCCAAGGCAAAATGA

Dna [30]v2:
ATGATACCGACGTGCAATTGGCCATTTTCTCCAACCTTCTGGGCGTATTTCTGTTCTGCTGCT
GGTGGTTGCCTATCATTATATTAATGCCAATACCGGAAAGCCCAAGTGCCAGGCAAAATGA

Input: 117
>4FFread30del2
ATGATACCGACGTGCAATTGGCCATTTTCTCCAACCTTCTGGGCGTATTTCTGTTCTGCTGGT
GGTGGTGCCTATCATTATATTAATGCCAATACCGGAAAGCCCAAGTGCCAGGCAAAATGA
```

Figure 6-3: DNA Substring - With Deletion

- With Insertion and Deletion: Both changes were applied to test the robustness of the algorithm in the presence of multiple variations.



```
Dna [30]
ATGATCACTACCGACGTCAATTGGCCATTTTCTCCAACGTTCTGGGCGATATTTCTGCTTC
CTGCTGGTGGTTGCCCTATCATTATATTAATGCCAATACTCGGAAGCCAGTGCCAAGGCAA
AATGA

Input - 123
ATGATCACTACCGACGTCAATTGGCCATTTTCTCCAACGTTCTGGGCGATATTTCTGCTTCCTG
CTGGTCGGTCTATACATTATATTAATGCCAATACTCGGAAGCCAGTAAGGCAAAATGA

>read30_indelv2
ATGATGGTCACCGACGTGCAATTGGCCATTTTCTCCAACGTTCTGGGCGTATTTCTGTTTCCT
GCTGGTGGTTGCCTATCATTATATTAATGCCAATACCGGAAGCCAGTGCCAAGGCAAAAT
GA

>read30_indelv2
ATGATGGTCACCGACGTGCAATTGGCCATTTTCTCCAACGTTCTGGGCGTATTTCTGTTTCCT
GCTGGTGGTTGCCTATCATTATATTAATGCCAATACCGGAAGCCGCCAAGGCAAAATGA
```

Figure 6-4: DNA Substring - With Insertion and Deletion

For each case, the output of the CUDA implementation was compared to the reference output of the sequential implementation.

6.1.1 Manual Checking

The first Correctness Test is a manual inspection via Excel. It was performed by aligning DNA reads against known reference proteins manually. The insertion (I), deletion (D), and scoring (C) matrices were computed and examined to ensure that the scoring followed the expected recurrence relations.

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=3 SV=2																																											
REF PROTEIN: MITDVQLAIFSNVLGVFLVLLVAYHYINANTGKPSAKAK																																											
DNA INPUT: GACGCTGAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTGATCCGCGTCAGTCAGAGGT																																											
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	V	V	A	Y	H	Y	I	N	A	N	T	G	K	P	S	A	K	A	K			
i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40		
A	0	-999	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
T	1	-999	-5	0	-2	-4	-5	-4	-5	-3	-5	-4	-5	-5	-4	-3	-5	-4	-3	-5	-3	-3	-4	-4	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
G	2	-999	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	3	-999	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
T	4	-999	-5	-4	4	2	0	-2	-4	-3	-4	-1	-3	-5	-2	-2	-4	-2	-4	-3	-3	-3	-1	0	-1	-3	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
C	5	-999	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-4	-4	-4	-4	-5	-5	-5	-1	-3	-5	-4
A	6	-999	-5	-5	-5	-2	-4	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-3	3	1	-1	-3	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5
C	7	-999	-5	-5	-1	9	7	5	3	1	-1	-3	-3	-3	-5	-3	-4	-5	-4	-5	-5	-4	-4	-1	0	-1	-3	-5	-5	-1	-3	-5	0	-2	-4	-5	-4	-5	-4	-5	-5	-5	
C	8	-999	-5	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	2	0	-2	-4	-5	
G	9	-999	-5	-5	-5	-5	3	1	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-2	1	-1	-3	-5	-5	-5	-5	-5	-5	-3	-5	-5	-3	-5	
A	10	-999	-5	-5	-3	4	15	13	11	9	7	5	3	1	-1	-3	-5	-4	-5	-5	-5	-5	-5	-5	-4	-2	-4	-5	-4	-3	-4	-5	-1	-3	-5	-2	-4	-5	-5	-5	-5	-5	
C	11	-999	-5	-5	-5	0	4	2	0	-2	-4	-5	-5	-4	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-4	-5	-5	-5	0	-2	-4	-3	3	1	-1	-3	-5		

Figure 6-5: Excel Snapshot of Manual Computation of the I matrix (match case)



PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=3 SV=1																																												
REF PROTEIN: MITDVQLAIFSNVLGVFLLLVAYHYINANTGKPSAKAK																																												
DNA INPUT: GACGCGTGTAGGCCATAACATGATTGCCACTTGTCACAGTAGGCTGAACCGCCGGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTGATCCGCGTCAGTCAGAGGTTGTCATTGAC																																												
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	V	V	A	Y	H	Y	I	N	A	N	T	G	K	P	S	A	K	A	K				
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40		
A	0	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99		
T	1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
G	2	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99
A	3	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99
T	4	-5	0	-4	-5	-5	-4	-5	-3	-5	-4	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3	-3	-4	-4	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
C	5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
A	6	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
C	7	-5	-2	4	-1	-3	-2	-5	-3	-4	-1	-4	-5	-5	-2	-2	-5	-2	-4	-3	-3	-3	-1	0	-1	-5	-5	-5	-1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
C	8	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-4	-4	-4	-5	-5	-5	-5	-5	-1	-4	-5	-4	-5		
G	9	-5	-5	-5	-2	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-3	3	-2	-4	-4	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5		
A	10	-5	-4	2	9	4	2	0	-2	-3	-3	-3	-3	-4	-3	-4	-4	-5	-4	-3	-1	0	-1	-5	-5	-5	-3	-1	-5	0	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5		
C	11	-5	-5	-5	-1	-5	-5	-5	-5	-5	-5	-3	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
G	12	-5	-5	-5	-4	3	-2	-1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	1	1	-4	-4	-5	-5	-5	-5	-3	-5	-5	-5	-3	-5	-3	

Figure 6-6: Excel Snapshot of Manual Computation of the D matrix (match case)

PROTEIN HEADER [FF30] >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=3 SV=1																																												
REF PROTEIN: MITDVQLAIFSNVLGVFLLVAYHYINANTGKPSAKAK																																												
DNA INPUT: GACGCTGTAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTTGATCCGCGTCAGTCAGAGGTTGTCATTGAC																																												
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	A	Y	H	Y	I	N	A	N	T	G	K	P	S	A	K	A	K			
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40		
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
T	1	0	5	1	0	0	1	0	2	0	1	0	0	0	1	2	0	1	0	2	0	2	2	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
G	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
A	3	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
T	4	0	1	9	4	2	3	0	2	1	4	1	0	0	3	3	0	3	1	2	2	2	4	5	4	0	0	0	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
C	5	0	0	0	1	0	0	0	1	0	0	4	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	1	1	1	0	0	0	4	1	0	1	0	1	
A	6	0	0	0	3	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	8	3	1	1	0	1	0	0	0	0	0	0	0	0	0	
C	7	0	0	4	14	9	7	5	3	2	0	2	2	0	0	2	1	0	1	0	0	1	1	4	5	4	0	0	0	4	0	0	5	0	0	0	0	1	0	0	0	0	0	
C	8	0	0	0	4	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	7	2	3	0	0	0		
G	9	0	0	0	0	8	3	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	3	6	1	1	0	0	0	0	2	0	0	0	2	0	2	
A	10	0	0	2	9	20	15	13	11	9	7	5	3	3	0	0	1	0	0	0	0	0	0	0	0	0	1	3	1	0	1	0	1	2	1	0	4	0	0	3	0	0	0	
C	11	0	0	0	5	9	5	2	0	0	0	0	1	0	2	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	5	0	0	2	8	3	2	0	0
G	12	0	0	0	0	3	13	12	7	6	3	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	3	1	0	0	0	2	0	2	0	2	1	2	0	

Figure 6-7: Excel Snapshot of Manual Computation of the C matrix (match case)



PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=:																						
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVYHYINANTGKPSAKAK																						
DNA INPUT: GACGCTGTAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTGATCCGCGTCAGTCAG																						
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	
	ij	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	0	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
T	1	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	2	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
A	3	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
T	4	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	5	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
A	6	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	7	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	8	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	9	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
A	10	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	11	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	12	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
T	13	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	14	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	15	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

Figure 6-8: Excel Snapshot of Program Output vs. Manual Calculation Comparison (match case)

The traceback path was also validated to confirm that the alignments followed logical biological interpretations.

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=3 SV=1																																										
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVYHYINANTGKPSAKAK																																										
DNA INPUT: GACGCTGTAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAAGCTTCTATTCAAATTGATCCGCGTCAGTCAGAGGTTGTCATTGAC																																										
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	L	V	V	A	Y	H	Y	I	N	A	N	T	G	K	P	S	A	K	A	K			
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
T	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	M	
G	2	0	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	
A	3	0	-2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	-2	
T	4	0	3	3	-2	-2	3	-2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	I	
C	5	0	3	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	
A	6	0	3	3	2	2	3	3	3	3	3	2	3	3	3	3	3	3	3	3	3	3	3	3	2	2	2	3	3	-2	2	3	3	3	3	3	3	3	3	3	3	
C	7	0	3	-1	3	-2	-2	-2	3	3	3	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	T	
C	8	0	3	3	4	3	3	3	3	3	3	-1	3	3	3	3	3	3	3	3	3	3	3	3	4	3	3	3	2	3	3	3	3	3	3	3	3	3	-2	3	3	
G	9	0	3	3	2	2	-2	2	-2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	2	2	-1	3	-2	3	3	3	3	2	3	3	3	3	3	3	3	
A	10	0	3	-1	-1	3	-2	-2	-2	-2	-2	3	-2	-2	3	3	3	3	3	3	3	3	3	3	3	3	4	-1	3	3	3	3	-1	3	-2	3	2	3	3	3	D	
C	11	0	3	3	3	4	4	-2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	4	3	3	2	3	3	3	3	3	-2	3	-1	3	-2	3	3	
G	12	0	3	3	3	-1	2	2	2	2	2	2	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	

Figure 6-9: Excel Snapshot of Manual Traceback Process (match case)

Manual Traceback	MITDVQLAIFSNVLGVFLFLLVYHYINANTGKPSAKAK
Program Traceback	MITDVQLAIFSNVLGVFLFLLVYHYINANTGKPSAKAK



Table 6-1: Traceback Summary (match case)

Cases such as perfect matches, insertions, deletions, and combinations of both were tested, and results were also consistent with expected outputs. This verification ensures that the scoring scheme, frame handling, and traceback traversal were implemented correctly in code.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	AC	AD	
PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase																															
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																															
DNA INPUT: ATGATCACCGACGTGCAATTTAGGGCCATTTCTCCAACGTTCTGGGCGTATTCTGTTCTGCTGGTCT																															
				M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	A	Y	H	Y	
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27		
A	0	-99	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5		
T	1	-99	-5	0	-2	-4	-5	-4	-5	-3	-5	-4	-5	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3	-3	-4	-4	-5	-5	-5		
G	2	-99	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5		
A	3	-99	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5		
T	4	-99	-5	-4	4	2	0	-2	-4	-3	-4	-1	-3	-5	-5	-2	-2	-4	-2	-4	-3	-3	-3	-1	0	-1	-3	-5	-5		
C	5	-99	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	
A	6	-99	-5	-5	-5	-2	-4	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-3	3	
C	7	-99	-5	-5	-1	9	7	5	3	1	-1	-3	-3	-3	-5	-5	-3	-4	-5	-4	-5	-5	-4	-4	-1	0	-1	-3	-5		
C	8	-99	-5	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
G	9	-99	-5	-5	-5	-5	3	1	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-2	
A	10	-99	-5	-5	-3	4	15	13	11	9	7	5	3	1	-1	-3	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-4	-2	-4	-5	
C	11	-99	-5	-5	-5	0	4	2	0	-2	-4	-5	-5	-4	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	

Figure 6-10: Excel Snapshot of Manual Computation of the I matrix (With Insertion)



PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subu																												
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																												
DNA INPUT: ATGATCACCGACGTGCAATTTAGGGCCATTTTCTCCAACGTTCTGGGCGTATTCTGTTCCTGCTGGTGGTTG																												
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	A	Y	H	
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
A	0	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99
T	1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
G	2	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99
A	3	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99	-99
T	4	-5	0	-4	-5	-5	-4	-5	-3	-5	-4	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3	-3	-4	-4	-5	-5	-5	-5
C	5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	6	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
C	7	-5	-2	4	-1	-3	-2	-5	-3	-4	-1	-4	-5	-5	-2	-2	-5	-2	-4	-3	-3	-3	-1	0	-1	-5	-5	-5
C	8	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5
G	9	-5	-5	-5	-2	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-3	3	
A	10	-5	-4	2	9	4	2	0	-2	-3	-3	-3	-3	-5	-4	-3	-4	-4	-4	-5	-5	-4	-3	-1	0	-1	-5	-5
C	11	-5	-5	-5	-1	-5	-5	-5	-5	-5	-5	-5	-3	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
G	12	-5	-5	-5	-4	3	-2	-1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	1
T	13	-5	-5	0	7	15	10	8	6	4	2	0	-2	-2	-5	-5	-4	-5	-5	-5	-5	-5	-3	-2	-2	-4	-5	

Figure 6-11: Excel Snapshot of Manual Computation of the D matrix (With Insertion)

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltr																											
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																											
DNA INPUT: ATGATCACCGACGTGCAATTTAGGGCCATTTTCTCCAACGTTCTGGGCGTATTCTGTTCCTGCG																											
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V			
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23		
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	1	0	5	1	0	0	1	0	2	0	1	0	0	0	1	2	0	1	0	2	0	2	2	1	1		
G	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
A	3	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	4	0	1	9	4	2	3	0	2	1	4	1	0	0	3	3	0	3	1	2	2	2	4	5	4		
C	5	0	0	0	1	0	0	0	0	1	0	0	4	1	0	0	0	0	0	0	0	0	0	0	0	0	0
A	6	0	0	0	3	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
C	7	0	0	4	14	9	7	5	3	2	0	2	2	0	0	2	1	0	1	0	0	1	1	4	5		
C	8	0	0	0	4	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0
G	9	0	0	0	0	8	3	4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
A	10	0	0	2	9	20	15	13	11	9	7	5	3	3	0	0	1	0	0	0	0	0	0	0	0	1	
C	11	0	0	0	5	9	5	2	0	0	0	0	1	0	2	0	0	0	0	0	0	0	0	0	0	0	0
G	12	0	0	0	0	3	13	12	7	6	3	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	13	0	1	3	7	15	24	19	17	15	13	11	9	7	7	3	1	5	0	1	0	1	1	4	4		

Figure 6-12: Excel Snapshot of Manual Computation of the C matrix (With Insertion)



PROTEIN HEADER [FF30]: >sp|A1Z7U3|OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN: REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK

DNA INPUT: GAGCGTGTAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTGATCCG

	ij	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A	0	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
T	1	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
G	2	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
A	3	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
T	4	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
C	5	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
A	6	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
C	7	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
C	8	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
G	9	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
A	10	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI
C	11	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TI

Figure 6-13: Excel Snapshot of Program Output vs. Manual Calculation Comparison (With Insertion)

PROTEIN HEADER [FF30]: >sp|A1Z7U3|OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN: REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK

DNA INPUT: ATGATCACCGACGTGCAATTTAGGGCCATTTCTCCAACGTTCTGGGCGTATTCTGTTCTGCTGGTGGTTGCCTATCATTATATTAATGCCAATACCG

		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	K
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
G	2	0	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2
A	3	0	-2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
T	4	0	3	-1	3	-2	-2	-2	3	3	3	3	3	3	3	3	3	3	3	3
C	5	0	3	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
A	6	0	3	3	2	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3
C	7	0	3	-1	3	-2	-2	-2	3	3	3	3	3	3	3	3	3	3	3	3
C	8	0	3	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
G	9	0	3	3	2	-2	-2	-2	3	3	3	3	3	3	3	3	3	3	3	3
A	10	0	3	-1	-1	3	-2	-2	-2	-2	3	3	3	3	3	3	3	3	3	3
C	11	0	3	3	4	4	-2	-2	3	3	3	3	3	3	3	3	3	3	3	3
G	12	0	3	3	3	-1	2	-2	-2	-2	3	3	3	3	3	3	3	3	3	3
T	13	0	3	3	-1	-1	3	-2	-2	-2	-2	3	3	3	3	3	3	3	3	3
G	14	0	3	3	-1	-1	4	-2	-2	-2	3	3	3	3	3	3	3	3	3	3

Figure 6-14: Excel Snapshot of Manual Traceback Process (With Insertion)

Manual Traceback	MITDVQL-AIFSNVLGVFLFLLVVAYHYINANTG-KPSAKA K
Program Traceback	MITDVQL-AIFSNVLGVFLFLLVVAYHYINANTG-KPSAKA K

Table 6-2: Traceback Summary (With Insertion)



PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 O																									
REF PROTEIN: MITDVQLAIFSNVLGVFLFLVVAYHYINANTGKPSAKAK																									
DNA INPUT: ATGATCAACGTGCAATTGGCCATTTCTCCAACGTTCTGGTATTTTCCTGCTGGTGGTTGCCTATCATTATATTAATAAT																									
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
A	0	-99%	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	
T	1	-99%	-5	0	-2	-4	-5	-4	-5	-3	-5	-4	-5	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3	-3	-4
G	2	-99%	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	3	-99%	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
T	4	-99%	-5	-4	4	2	0	-2	-4	-3	-4	-1	-3	-5	-5	-2	-2	-4	-2	-4	-3	-3	-3	-1	0
C	5	-99%	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	6	-99%	-5	-5	-5	-1	-3	-5	0	-2	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	7	-99%	-5	-5	-1	4	2	0	-2	-4	-5	-5	-4	-3	1	-1	-3	-2	-4	-5	-5	-5	-5	-5	-4
C	8	-99%	-5	-5	-5	5	3	1	-1	-3	-5	-5	-5	-4	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5
G	9	-99%	-5	-5	-5	-5	-2	-4	-4	-2	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
T	10	-99%	-5	-4	-2	-1	1	4	2	0	-2	-2	-4	-5	-4	5	3	1	2	0	-2	-4	-4	-4	-1
G	11	-99%	-5	-5	-5	0	2	0	-2	-4	-5	-5	-5	-5	-5	-2	-4	-5	-5	-5	-5	-5	-5	-5	-5
C	12	-99%	-5	-5	-5	-5	-1	-2	-1	-3	2	0	-2	-4	-5	-5	0	-2	-4	-4	-5	-5	-5	-5	-5
A	13	-99%	-5	-5	-5	-3	-1	-1	9	7	5	3	1	-1	-3	0	3	1	-1	-1	-3	-5	-5	-5	-5
A	14	-99%	-5	-5	-5	-2	-1	-1	0	-2	4	5	5	-4	-1	1	2	4	-5	-5	-5	-5	-5	-5	-5

Figure 6-15: Excel Snapshot of Manual Computation of the I matrix (With Deletion)

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosopl																											
REF PROTEIN: MITDVQLAIFSNVLGVFLFLVVAYHYINANTGKPSAKAK																											
DNA INPUT: ATGATCAACGTGCAATTGGCCATTTTCTCCAACGTTCTGGTATTTTTCCTGCTGGTGGTTGCCTATCATTATATTAATAATACCGGA																											
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	A	Y	
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
A	0	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%
T	1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
G	2	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%
A	3	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%
T	4	-5	0	-4	-5	-5	-4	-5	-3	-5	-4	-5	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3	-3	-4	-4	-5	-5
C	5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	6	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	7	-5	-2	4	-1	-3	-2	-5	-3	-4	-1	-4	-5	-5	-2	-2	-5	-2	-4	-3	-3	-3	-1	0	-1	-5	-5
C	8	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5
G	9	-5	-5	-5	-1	-5	-5	0	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
T	10	-5	-4	2	4	0	-3	-2	-5	-5	-3	-4	-3	1	-4	-4	-2	-4	-5	-5	-5	-5	-3	-2	-3	-3	-5
G	11	-5	-5	-5	5	0	-2	-4	-5	-5	-5	-5	-3	-1	-4	-5	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5
C	12	-5	-5	-5	-3	-2	-5	-2	-2	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	13	-5	-4	0	2	1	4	-1	-1	-5	-2	-5	-5	-1	5	0	-2	2	-3	-4	-5	-4	-4	-1	0	-3	-4



Figure 6-16: Excel Snapshot of Manual Computation of the D matrix (With Deletion)

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila m																												
REF PROTEIN: MITDVQLAIFSNVLGVFLFLVWAYHYINANTGKPSAKAK																												
DNA INPUT: ATGATCAACGTGCAATTGGCCATTTCCTCCAACGTTCTGGTATTTTCCTGCTGGTGGTTGCCTATCATTATATTAATAATACCGGAAAGC																												
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	A	Y	H	
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
T	1	0	5	1	0	0	1	0	2	0	1	0	0	0	1	2	0	1	0	2	0	2	2	1	1	0	0	0
G	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
A	3	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
T	4	0	1	9	4	2	3	0	2	1	4	1	0	0	3	3	0	3	1	2	2	2	4	5	4	0	0	0
C	5	0	0	0	1	0	0	0	0	1	0	0	4	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0
A	6	0	0	0	4	0	0	5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
A	7	0	0	4	9	5	2	3	0	0	0	1	2	6	1	0	3	0	0	0	0	0	0	1	2	2	0	1
C	8	0	0	0	10	5	3	1	0	0	0	0	1	4	1	0	0	0	0	0	0	0	0	0	1	0	0	0
G	9	0	0	0	0	3	0	1	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	10	0	1	3	4	6	9	4	4	0	3	0	0	1	10	5	3	7	2	1	0	1	1	4	5	2	1	0
G	11	0	0	0	5	7	4	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0	0	1	0	0
C	12	0	0	0	0	4	3	4	0	7	2	0	1	0	0	5	1	0	1	0	0	0	0	0	0	5	0	0
A	13	0	0	0	2	4	4	14	9	7	5	3	1	0	5	8	3	2	4	0	0	0	0	0	2	4	1	1
A	14	0	0	0	2	6	4	5	0	0	0	0	4	6	4	3	4	0	0	0	0	0	0	0	0	0	0	1

Figure 6-17: Excel Snapshot of Manual Computation of the C matrix (With Deletion)

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=:																										
REF PROTEIN: MITDVQLAIFSNVLGVFLFLVWAYHYINANTGKPSAKAK																										
DNA INPUT: GACGCTGTAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTGATCCGCGTCAGTCAG																										
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L					
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20				
A	0	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
T	1	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
G	2	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
A	3	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
T	4	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
C	5	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
A	6	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
C	7	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
C	8	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
G	9	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
A	10	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
C	11	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
G	12	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
T	13	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
G	14	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1
C	15	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	1

Figure 6-18: Excel Snapshot of Program Output vs. Manual Calculation Comparison (With Deletion)



PROTEIN HEADER [FF30] >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=3 SV=1																																													
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																																													
DNA INPUT: ATGATCAACGTGCAATTGGCCATTTCTCCAACGTTCTGGTATTTTCTCGTGGTGGTGGCTATCATTATATTAATAACCGGAAAGCCAGTGCCAAGGCATGA																																													
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L	V	V	A	Y	H	Y	I	N	A	N	T	G	K	P	S	A	K	A	K				
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40			
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
T	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	M		
G	2	0	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2		
A	3	0	-2	2	2	2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2		
T	4	0	3	3	-2	-2	3	-2	3	3	3	3	3	3	3	-2	3	3	3	3	3	3	3	3	3	3	3	3	3	-2	3	3	3	3	3	3	3	3	3	3	3	3	3	I	
C	5	0	3	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	T	
A	6	0	3	3	2	2	3	-2	3	3	-2	3	2	3	3	3	3	3	3	3	3	3	2	2	3	3	3	3	3	3	2	3	3	3	3	3	3	3	3	3	3	3	3	3	
A	7	0	-1	-1	3	3	-2	3	-2	-2	-1	3	3	3	-2	3	-2	3	3	3	3	-1	3	3	3	3	3	3	-1	3	-2	3	-2	3	3	3	3	3	3	3	2	3	3	3	D
C	8	0	3	3	4	-2	-2	-2	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	-2	-2	3	3	3	3	3	3		
G	9	0	3	3	-1	2	-2	3	3	3	3	3	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	2	2	2	3	3	3	3	3	3	3	3	3	3		
T	10	0	3	3	-1	3	3	-2	3	-2	3	3	-1	3	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	3	V	

Figure 6-19: Excel Snapshot of Manual Traceback Process (With Deletion)

Manual Traceback	MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK
Program Traceback	MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK

Table 6-3: Traceback Summary (With Deletion)

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein g																							
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																							
DNA INPUT: ATGATCACTACCGACGTCAATTGGCATTCTCTCCAACGTTCTGGGCGATATTTCTGCTT																							
			M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L	L
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
A	0	-998	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
T	1	-998	-5	0	-2	-4	-5	-4	-5	-3	-5	-4	-5	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3
G	2	-998	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	3	-998	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
T	4	-998	-5	-4	4	2	0	-2	-4	-3	-4	-1	-3	-5	-5	-2	-2	-4	-2	-4	-3	-3	-3
C	5	-998	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-3	-5	-5	-5	-5	-5	-5	-5	-5
A	6	-998	-5	-5	-5	-2	-4	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5
C	7	-998	-5	-5	-1	9	7	5	3	1	-1	-3	-3	-3	-5	-5	-3	-4	-5	-4	-5	-5	-4
T	8	-998	-5	-3	-3	-1	-3	-4	-5	-1	-3	-2	-4	-5	-4	-3	-1	-3	-4	-5	-1	-3	-1
A	9	-998	-5	-5	-5	-5	2	0	-2	-4	-5	-5	-2	-4	-5	-5	-5	-5	-5	-2	-4	-2	-4
C	10	-998	-5	-5	-3	4	8	6	4	2	0	-2	-4	-2	-3	-5	-5	-5	-4	-5	-5	-5	-5

Figure 6-20: Excel Snapshot of Manual Computation of the I matrix (With Insertion and Deletion)



PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--prote																						
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																						
DNA INPUT: ATGATCACTACCGACGTCAATTGGCATTCTCTCCAACGTTCTGGGCGATATTTCTGC																						
			M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L	F	L
	i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	0	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%
T	1	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
G	2	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%
A	3	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%	-99%
T	4	-5	0	-4	-5	-5	-4	-5	-3	-5	-4	-5	-5	-5	-4	-3	-5	-4	-5	-3	-5	-3
C	5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
A	6	-5	-5	-5	-5	-3	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5	-5
C	7	-5	-2	4	-1	-3	-2	-5	-3	-4	-1	-4	-5	-5	-2	-2	-5	-2	-4	-3	-3	-3
T	8	-5	-5	-5	-4	-5	-5	-5	-5	-4	-5	-5	-1	-4	-5	-5	-5	-5	-5	-5	-5	-5
A	9	-5	-5	-5	-2	-5	-5	-5	-5	-5	-5	-5	-5	-4	-5	-5	-5	-5	-5	-5	-5	-5
C	10	-5	-4	2	9	4	2	0	-2	-3	-3	-3	-3	-5	-4	-3	-4	-4	-4	-5	-5	-4
C	11	-5	-3	-3	-1	-5	-4	-5	-1	-5	-2	-5	-3	-4	-3	-1	-5	-4	-5	-1	-5	-1
G	12	-5	-5	-5	-4	2	-1	-3	-5	-5	-5	-2	-5	-5	-5	-5	-5	-5	-2	-5	-2	-5

Figure 6-21: Excel Snapshot of Manual Computation of the D matrix (With Insertion and Deletion)



PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=1																			
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																			
DNA INPUT: ATGATCACTACCGACGTCAATTGGCATTCTCTCCAACGTTCTGGGCGATATTTCTGCTTCCTC																			
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L
i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
T	1	0	5	1	0	0	1	0	2	0	1	0	0	0	1	2	0	1	0
G	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
A	3	0	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0
T	4	0	1	9	4	2	3	0	2	1	4	1	0	0	3	3	0	3	1
C	5	0	0	0	1	0	0	0	0	1	0	0	4	1	0	0	0	0	0
A	6	0	0	0	3	0	0	0	0	0	0	0	0	1	0	0	0	0	0
C	7	0	0	4	14	9	7	5	3	2	0	2	2	0	0	2	1	0	1
T	8	0	2	2	4	0	1	0	4	0	3	0	0	1	2	4	0	1	0
A	9	0	0	0	0	7	4	2	0	0	0	3	0	0	0	0	0	3	0
C	10	0	0	2	9	13	9	6	4	3	1	0	3	2	0	0	0	1	0
C	11	0	0	0	1	9	4	2	0	3	0	0	0	0	0	2	0	0	0

Figure 6-22: Excel Snapshot of Manual Computation of the C matrix (With Insertion and Deletion)

PROTEIN HEADER [FF30]: >sp A1Z7U3 OST4_DROME Dolichyl-diphosphooligosaccharide--protein glycosyltransferase subunit 4 OS=Drosophila melanogaster OX=7227 GN=CG33774 PE=1																			
REF PROTEIN: MITDVQLAIFSNVLGVFLFLLVVAYHYINANTGKPSAKAK																			
DNA INPUT: GACGCTGTAGGCCATAACATGATTGCCACTTGCTACAGTAGGCTGAACCCGCCCGATGTGACAGAGGCCCTGCAGCACTACCAACGTTCTATTCAAATTGATCCGCGTCAGTCAG																			
		M	I	T	D	V	Q	L	A	I	F	S	N	V	L	G	V	F	L
i,j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A	0	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
T	1	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	2	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
A	3	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
T	4	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	5	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
A	6	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	7	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	8	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	9	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
A	10	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	11	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	12	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
T	13	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
G	14	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
C	15	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

Figure 6-23: Excel Snapshot of Program Output vs. Manual Calculation Comparison (With Insertion and Deletion)



The image shows an Excel spreadsheet used for manual traceback. The columns represent amino acids (M, I, T, D, V, Q, L, A, I, F, S, N, V, L, G, V, F, L, L, V, V, A, Y, H, Y, I, N, A, N, T, G, K, P, S, A, K, A, K, K) and rows represent protein sequences. The cells contain numerical values representing alignment scores. Some cells are highlighted in blue, indicating specific alignment points or gaps.

Figure 6-24: Excel Snapshot of Manual Traceback Process (With Insertion and Deletion)

Manual Traceback	MITDVLQA-IFSNVLGVFLFLLVVAYHYINANTGKPSAK
Program Traceback	MITDVLQA-IFSNVLGVFLFLLVVAYHYINANTGKPSAK

Table 6-4: Traceback Summary (With Insertion and Deletion)

In addition to cell-wise matrix checking and traceback validation, another level of verification was performed to confirm that the program identified the correct protein alignment among the entire reference proteome dataset.

For each test case, a known DNA substring was processed and aligned against all protein entries in the dataset. The output protein ID and alignment score were recorded. A secondary manual inspection ensured that the best match corresponded to the expected protein. The program was also verified to return:

- The correct start and end indices of the alignment on the protein sequences
- The correct score, confirming that the alignment was indeed optimal.

Sequential					Parallel					Correctness Test		
Length	Score	s_index	e_index	Time (in ms)	Top Score	Protein Index	Length	Score	s_index	e_index	Time (in ms)	Top Score
0	774	58	158	208	3.885543333	245	0	774	58	158	208	0.9624042667
1	88	37	46	84	0.5204369667	Found in	1	88	37	46	84	0.2420682667
2	2718	67	1988	2022	16.19419333	10	2	2718	67	1988	2022	2.8658496
3	1966	57	1734	1776	10.65783667	Total Runtime (in sec)	3	1866	57	1734	1776	2.003924267
4	965	61	873	923	4.968356667	76.90024328	4	965	61	873	923	1.162852267
5	780	67	104	151	3.96821	Total Loops	5	780	67	104	151	0.9868736
6	343	47	11	56	1.724896667	30	6	343	47	11	56	0.5337802667
7	834	58	47	88	4.30576		7	834	58	47	88	1.049781333
8	2036	74	1643	1691	11.83943		8	2036	74	1643	1691	2.198458667
9	400	57	209	259	1.916886667		9	400	57	209	259	0.5634738667
10	2176	245	529	577	12.52604		10	2176	245	529	577	2.349426133
11	446	62	191	239	2.202053333		11	446	62	191	239	0.6138112
12	668	55	529	572	3.49637		12	668	55	529	572	0.84944
13	279	46	216	252	1.397126667		13	279	46	216	252	0.4553685333
14	1876	61	1643	1696	10.71636667		14	1876	61	1643	1696	2.0112512
15	523	48	182	214	2.563923333		15	523	48	182	214	0.6985333333
16	285	49	159	213	1.437566667		16	285	49	159	213	0.449568
17	340	50	300	340	1.646823333		17	340	50	300	340	0.5172864
18	396	57	206	245	1.933706667		18	396	57	206	245	0.5813376
19	635	49	122	172	3.1475		19	635	49	122	172	0.8367765333



Figure 6-26: BLASTX Results for Perfect Match

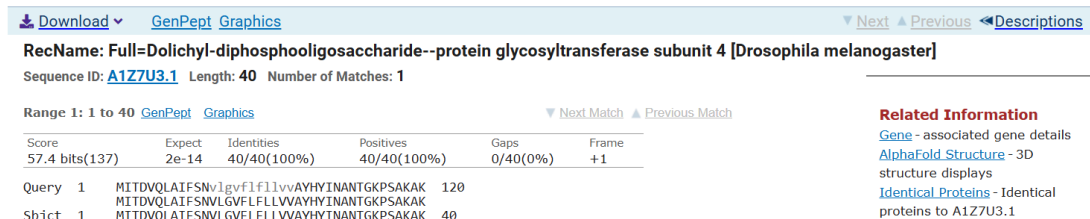


Figure 6-27: BLASTX Results for Alignment (Perfect Match)

This discrepancy can be explained by examining the BLASTX output parameters in detail. One key parameter is Query Cover, which indicates the percentage of the input sequence that aligns with the subject protein sequence. In ideal conditions, a perfect match should yield a query coverage close to 100%. However, BLASTX's heuristic approach often leads it to stop aligning once a sufficiently high score is reached, even if longer or more optimal alignments are possible. As a result, it may align only a portion of the input sequence, thus reducing the query cover.

Another parameter is the E-value, or Expectation value, which estimates the number of hits one might expect to find by chance in a database of a given size. Lower E-values indicate more statistically significant alignments. In our case, the E-value was extremely low, confirming the significance of the match. Nevertheless, because BLASTX performs only partial alignments in some cases, even perfect matches can yield non-zero E-values depending on how much of the input sequence it chooses to align.

Accession Length is another BLASTX metric, indicating the full length of the matched protein in the database. Even when BLASTX aligns only part of this protein, the total length is still displayed. This helps researchers understand how much of the target sequence was covered.

Finally, one of the most telling comparisons is between the raw alignment score and the bit score. The raw score in BLASTX is based on substitution and gap penalties, while the bit score is a normalized measure calculated using statistical constants. Our implementation computed a raw score of 198, while BLASTX reported a lower raw score of 137. This is because BLASTX's heuristic optimization allows it to stop early once it finds a reasonably high-scoring segment, which, while statistically significant, may not represent the absolute best alignment possible. This is further proven by the manual Excel computation, which also shows a score of 137.

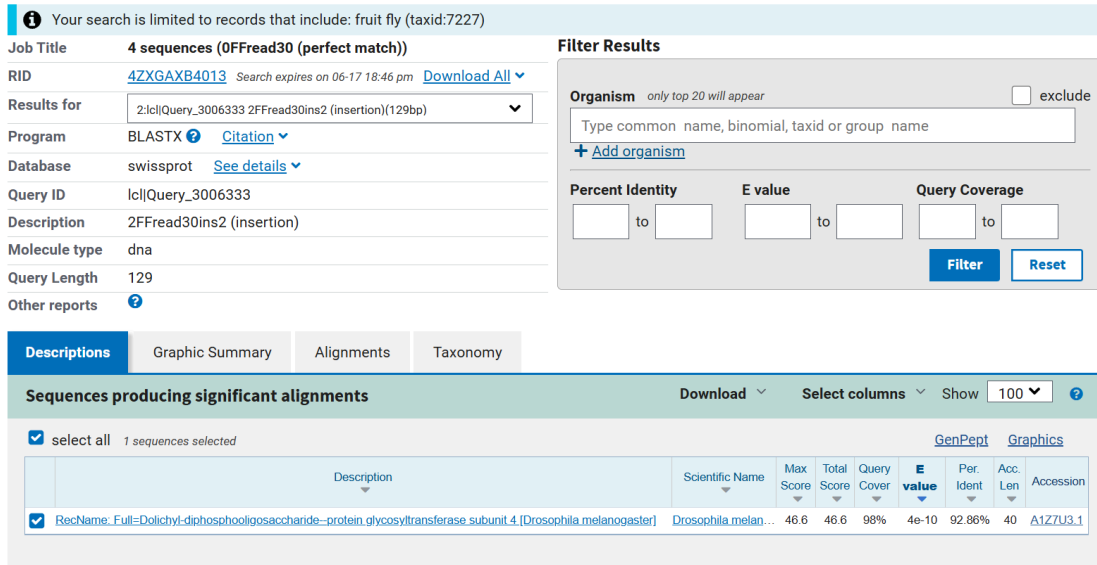


Figure 6-28: BLASTX Results for With Insertion

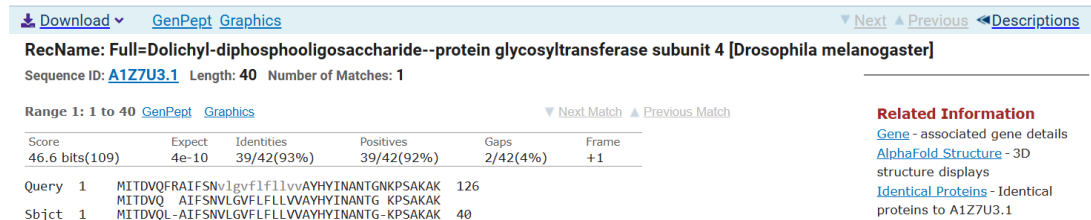


Figure 6-29: BLASTX Results for Alignment (With Insertion)

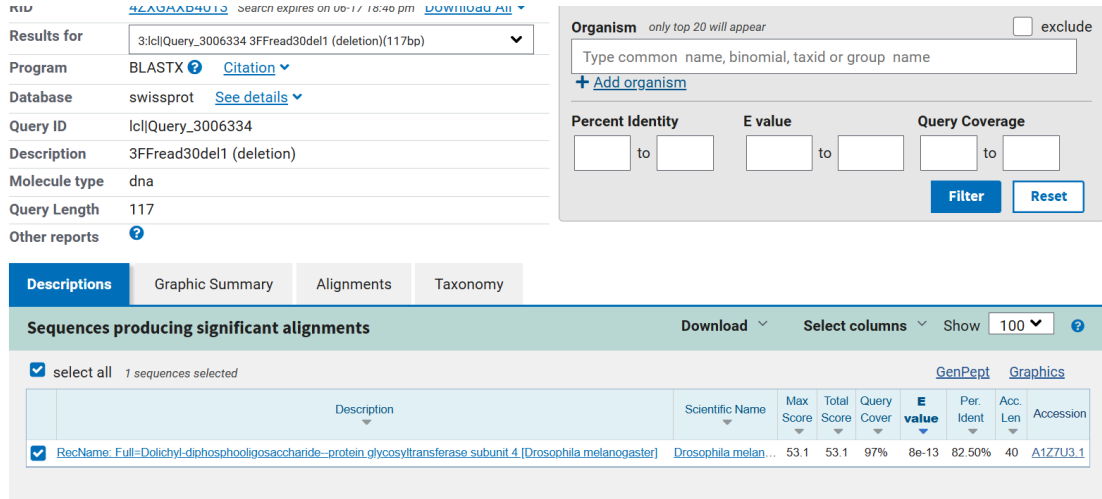


Figure 6-30: BLASTX Results for With Deletion

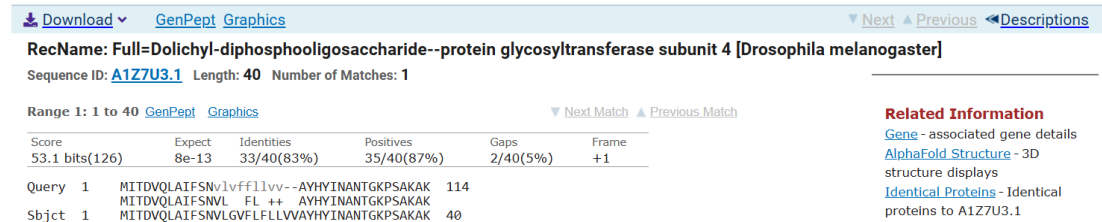


Figure 6-31: BLASTX Results for Alignment (With Deletion)

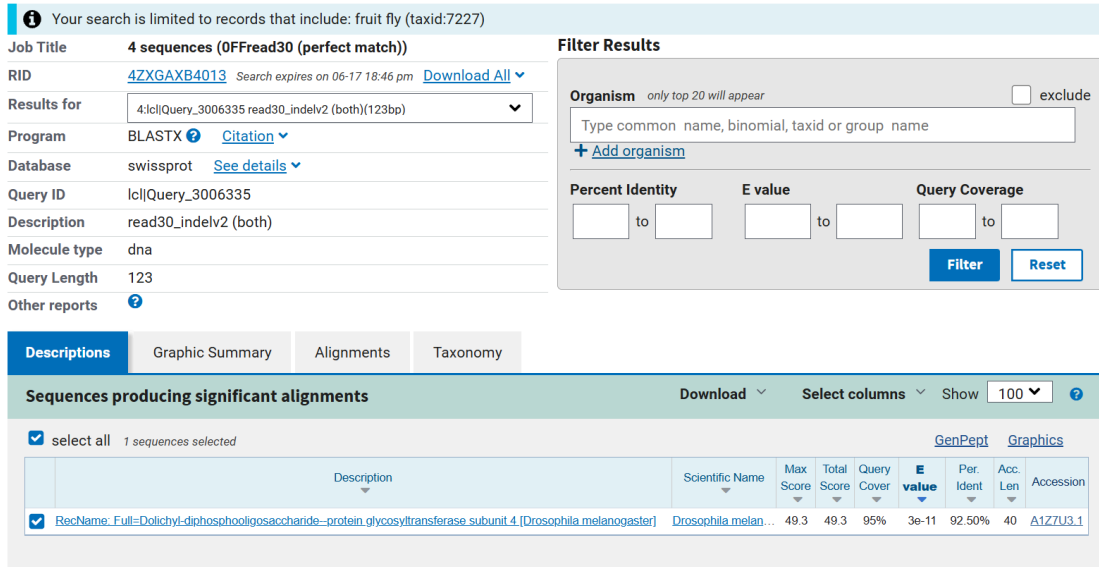


Figure 6-32: BLASTX Results for With Insertion and Deletion

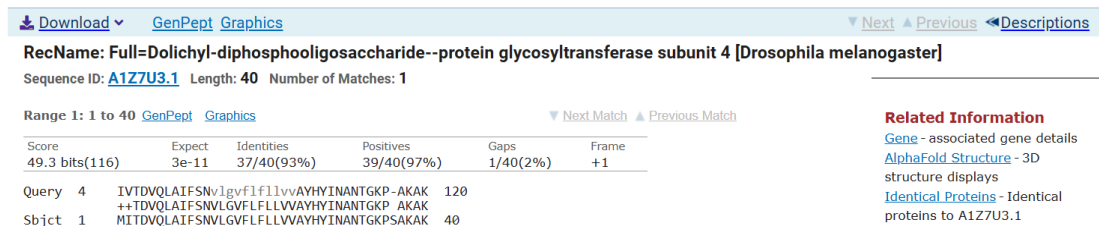


Figure 6-33: BLASTX Results for Alignment (With Insertion and Deletion)

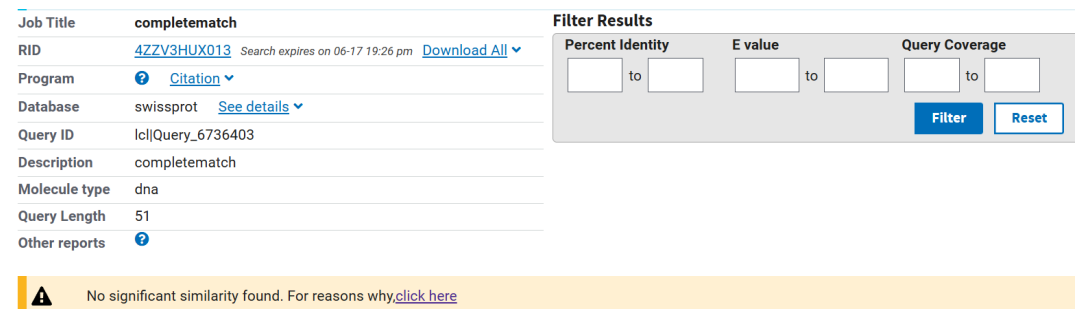


Figure 6-34: BLASTX Results for Complete Mismatch



Furthermore, we ensured correctness across all four DNA variants, including insertions and deletions. Overall, the implementation consistently identified accurate alignment positions and sequences.

6.1.2.2 Zhang's Six-Frame Algorithm in SIMD

To further ensure that the algorithm implementation is correct, the scores from the SIMD implementation and the SIMT implementation will be compared for verification.

This will make sure that there will be no discrepancy with regard to the scoring of the DNA to protein frame alignment. The values of the constants used for both programs were $gap = 3$ (gap opening penalty), $gap = 2$ (gap extension penalty), and $\pi = 4$ (shift penalty).

Protein	Length	Score (SIMD)	Score (CUDA)	
0	774	58	58	TRUE
1	88	37	37	TRUE
2	2718	67	67	TRUE
3	1866	57	57	TRUE
4	965	61	61	TRUE
5	780	67	67	TRUE
6	343	47	47	TRUE
7	834	58	58	TRUE
8	2036	74	74	TRUE
9	400	57	57	TRUE
10	2176	245	245	TRUE
11	446	62	62	TRUE
12	668	55	55	TRUE
13	279	46	46	TRUE
14	1076	64	64	TRUE

Figure 6-35: Excel Snapshot of SIMD and CUDA Score Results Comparison

6.2 Runtime Test

The Execution Time Test aimed to assess the runtime performance of the CUDA implementation in comparison to the sequential and SIMD implementations. This was done by recording the time before and after the execution of the kernel and computing the elapsed time. Each version of the algorithm was tested on identical DNA and protein sequences to ensure fair comparison. The datasets used for this test consisted of sequences from *Drosophila melanogaster* (fruit fly) and *Arabidopsis thaliana* (mouse-ear cress), both provided in FASTA format. Additionally, runtime measurements were averaged across 30 runs to reduce the impact of variability.



6.2.1 SIMT vs Sequential

To evaluate the performance difference between the sequential and SIMT (CUDA) implementations of Zhang's Six-Frame Alignment Algorithm, both versions were benchmarked on the first testing platform, the desktop with an NVIDIA RTX 4070S GPU and an Intel Core i7-12700F processor. Each implementation was run using the same input dataset comprising a pair of 150bp DNA read sequences and the fruit fly proteome with 21,449 protein sequences. A real time stopwatch test was conducted to observe the real elapsed time of one iteration on the two implementations. Additionally, to account for memory caching and initialization overhead in CUDA, two consecutive runs (CUDA 1st Loop and CUDA 2nd Loop) of the parallel implementation were recorded.

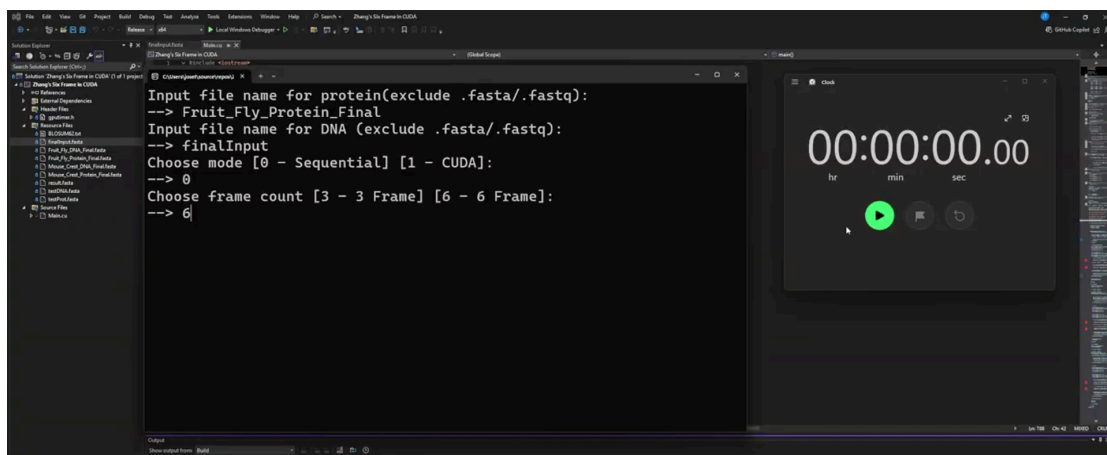


Figure 6-36: Screenshot for Sequential Runtime (START)

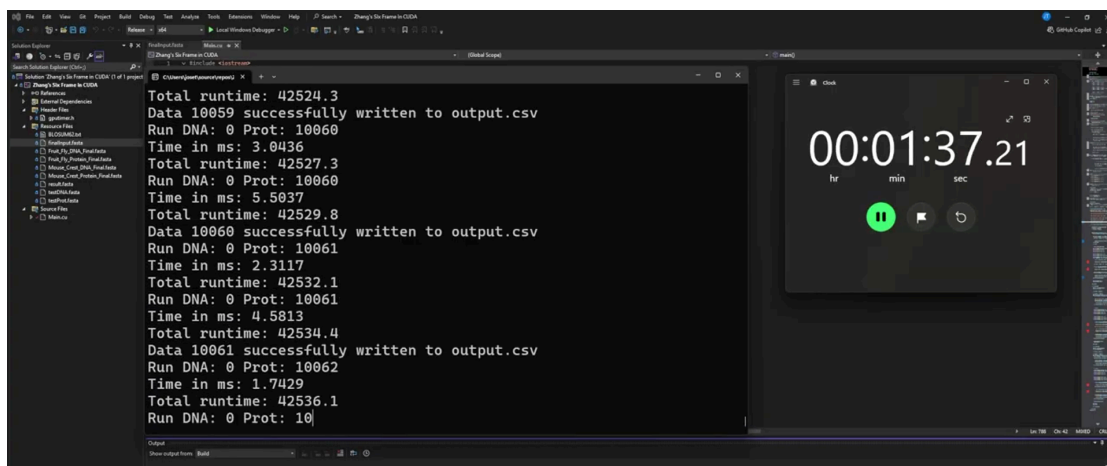




Figure 6-37: Screenshot for Sequential Runtime (MIDDLE)

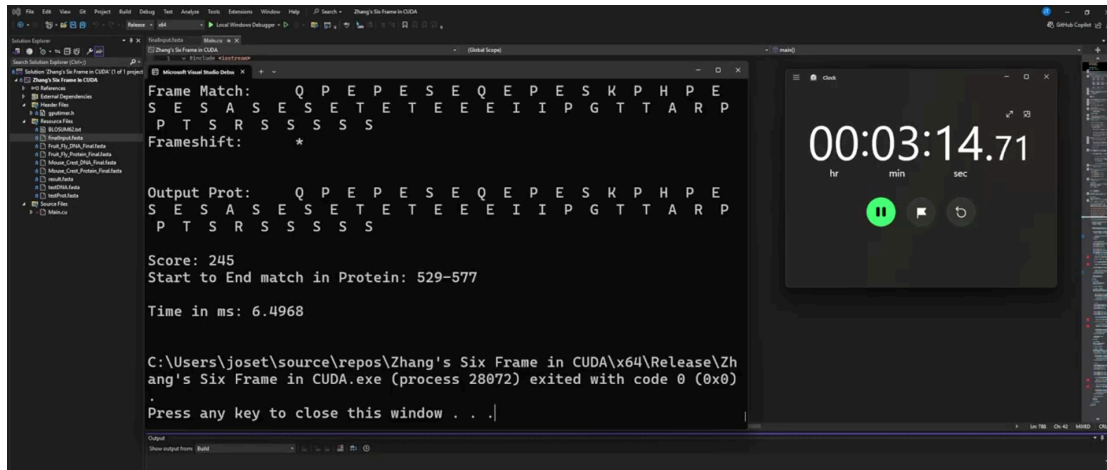


Figure 6-38: Screenshot for Sequential Runtime (END)

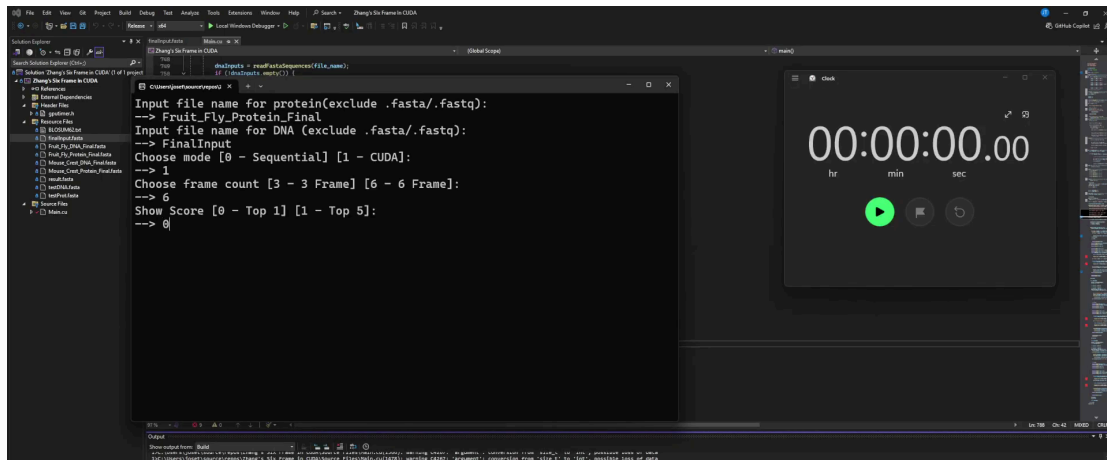


Figure 6-39: Screenshot for CUDA Runtime (START) - 1st Loop

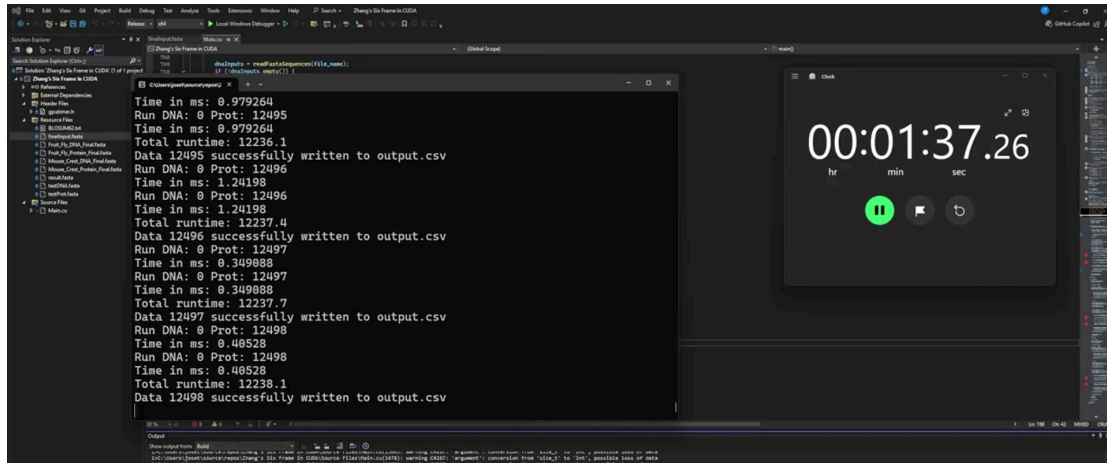


Figure 6-40: Screenshot for CUDA Runtime (MIDDLE) - 1st Loop

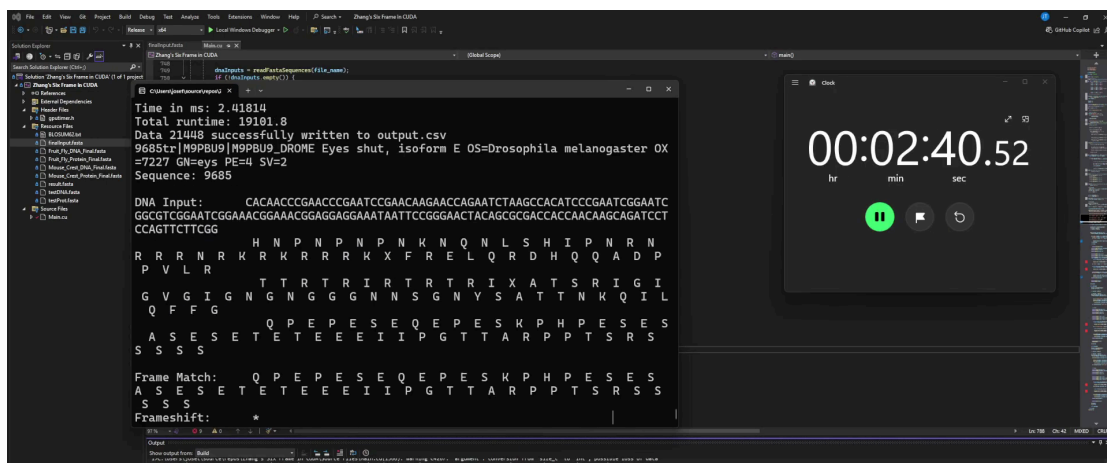


Figure 6-41: Screenshot for CUDA Runtime (END) - 1st Loop

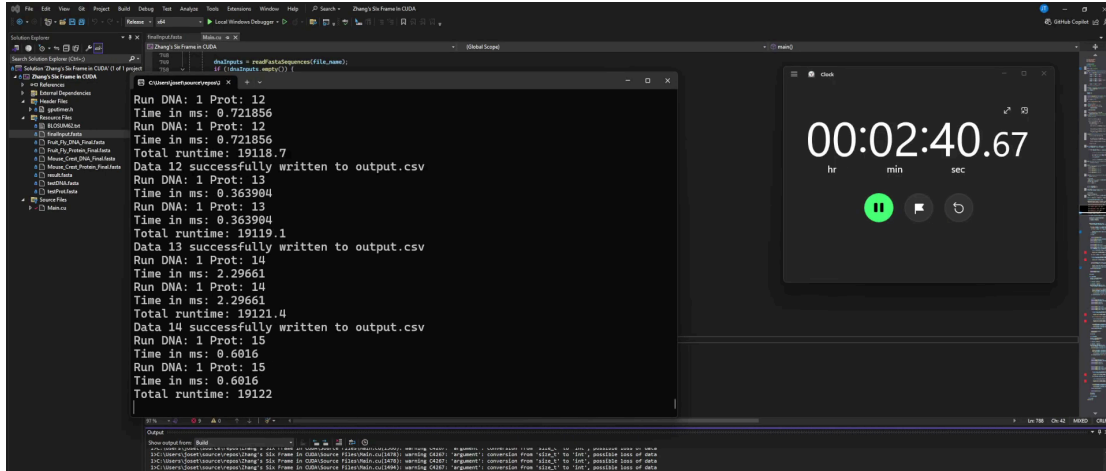


Figure 6-42: Screenshot for CUDA Runtime (START) - 2nd Loop

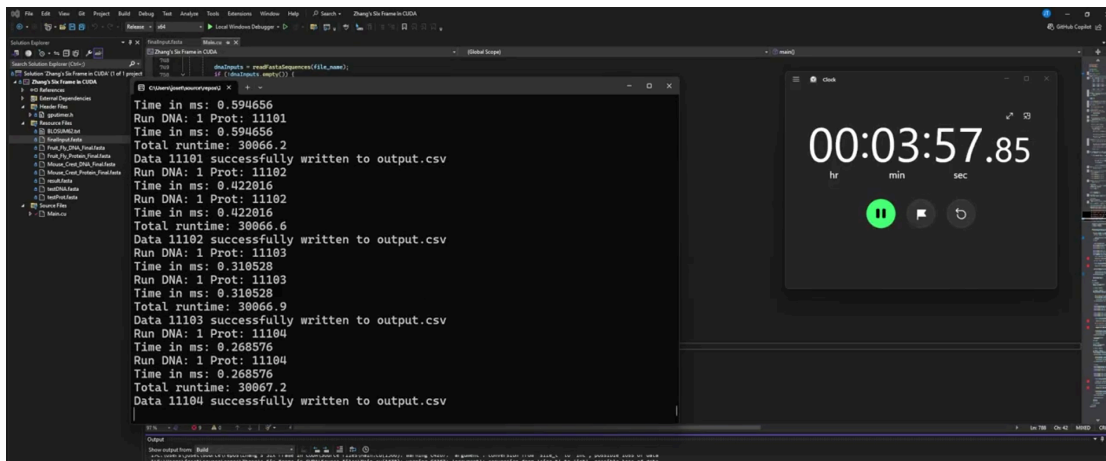


Figure 6-43: Screenshot for CUDA Runtime (MIDDLE) - 2nd Loop

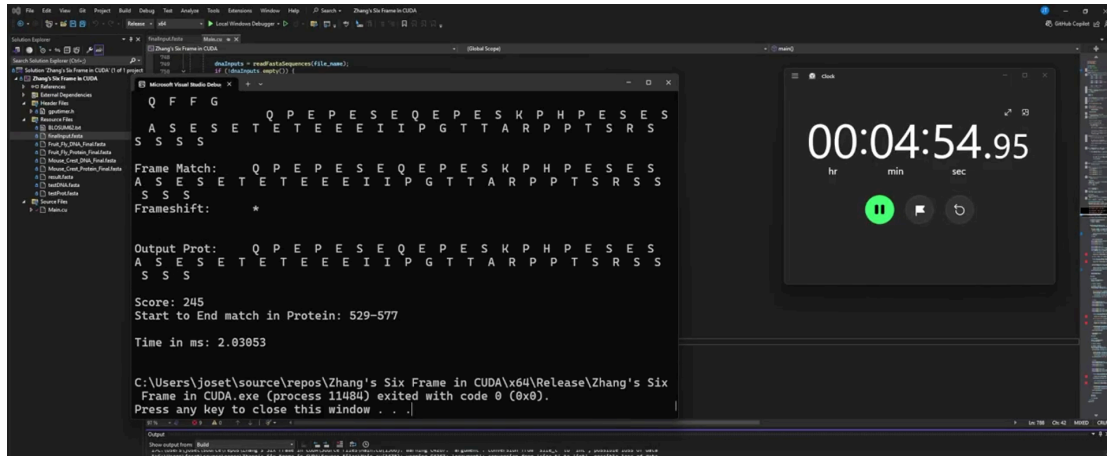


Figure 6-44: Screenshot for CUDA Runtime (END) - 2nd Loop

As seen in the Screenshots above, the real-time clock showed that the sequential implementation took 3 minutes and 14 seconds to complete the task. Meanwhile, the CUDA 1st Loop completed the same task in 2 minutes and 40 seconds, and the CUDA 2nd Loop further reduced the elapsed time to 2 minutes and 14 seconds. This demonstrates that the parallel implementation is faster, especially the CUDA 2nd Loop.

Additionally, the results also showcase the reduction of runtime between the two loops. This reduction can be attributed to several factors. First, CUDA must allocate memory, compile kernels, and initialize device-specific settings. These steps introduce additional overhead that does not apply in subsequent runs, hence making the runtime faster. Second is caching and memory reuse. After the first execution, the GPU may retain some compiled kernels and memory buffers in its internal cache, which reduces latency during future calls.

The different stages of the program were also timed. The program was split into 4 stages: program setup, matrix initialization, matrix scoring, and traceback. Given that the implementation aims to speed up the scoring stage of the algorithm, this allows us to quantify what fraction of the program is sped up. The results are shown in Table 6-5.

Stage	Sequential (in ms)	Fraction of the program	CUDA (in ms)	Fraction of the program
Program Setup	0.0117	0.27%	1.8238	48.09%



Matrix Initialization	0.2768	6.24%	0.2748	7.25%
Matrix Scoring	3.5892	80.98%	0.8726	23.01%
Traceback	0.5544	12.51%	0.8216	21.66%
Total	4.4321	100%	3.7928	100%

Table 6-5: Average Runtime Contribution of Each Phase For Single Read

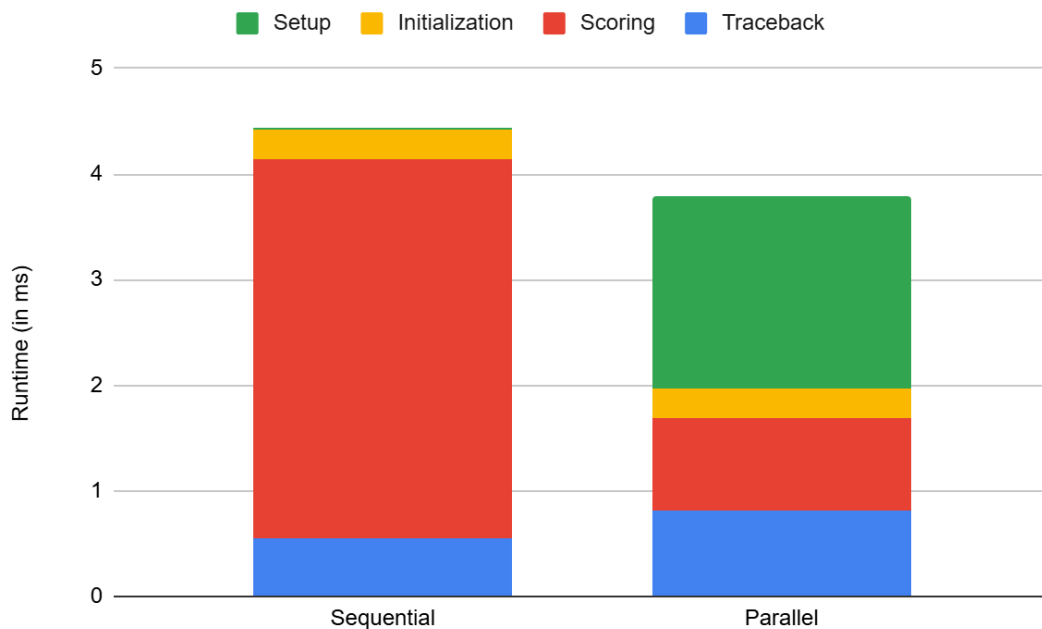


Figure 6-45: Visualization of Average Runtime Contribution of Each Stage For Single Read

Based on Table 6-5 and Figure 6-45, it can be seen that scoring, the core computational step in Zhang's Six-Frame Alignment algorithm, dominates the total runtime for the sequential implementation, showcasing the high computational cost of scoring. In the sequential version, scoring takes 3.59 ms, comprising 80.98% of a single read runtime, while in the CUDA implementation, scoring takes 0.87 ms, comprising only 23.01% of a single read, achieving an average speedup of approximately 4.13x in this phase alone.



However, unlike the sequential implementation, the setup stage is significantly higher in the CUDA implementation, with sequential setup only taking 0.01 ms, comprising only 0.27% of a single read. Setup in the parallel implementation takes 1.82 ms, and thus comprises 48.09%. This is largely due to CUDA-specific data transfer operations, given that the implementation utilized synchronous `cudaMemcpy` calls to transfer data on the host to the device, whereas no similar additional steps are necessary on the sequential implementation. This overhead contributes to the higher setup time. Additionally, the longer setup time for CUDA may also be attributed to the `checkCudaErrors` macro since it always checks for possible errors during memory allocation, setting of base values, and copying of the memory from host to device and vice versa.

Furthermore, the initialization phase, which involves setting up the scoring and traceback matrices in preparation for scoring, remains consistent across both implementations, with times of 0.2768 ms and 0.2748 ms. This behavior is expected as the initialization stages are identical for both implementations, being performed on the host, with the data transfer overhead accounted for in the setup stage.

Lastly, the traceback also exhibits a slightly higher runtime in the parallel version, with sequential traceback taking 0.55 ms and parallel traceback taking 0.8216 ms. This difference can be attributed to the data transfer calls from the device to the host that take place after scoring to enable the system to perform the traceback.

These findings show that the scoring phase is the most computationally intensive in the sequential implementation, and that the CUDA implementation was able to reduce that computational intensity. With that in mind, the following evaluations will focus on capturing and analyzing the runtime improvements of the scoring phase alone, since the proposed implementation aimed to improve only the scoring stage. This allows for a better assessment of the parallel efficiency of the matrix computation and the effectiveness of the hybrid multi-level anti-diagonal wavefront implementation scheme used in the parallel implementation.

6.2.1.1 Stress Test

For the first part of the stress test, 2 DNA input read queries of length 150bp are aligned to a total of 21,449 protein sequences of the fruit fly reference proteome. The protein sequences vary in length, from 11 bp to 22,949 bp. The execution times of individual sequence alignments are graphed in Figure 6-46. Summary statistics can be found in Tables 6-6 and 6-7.

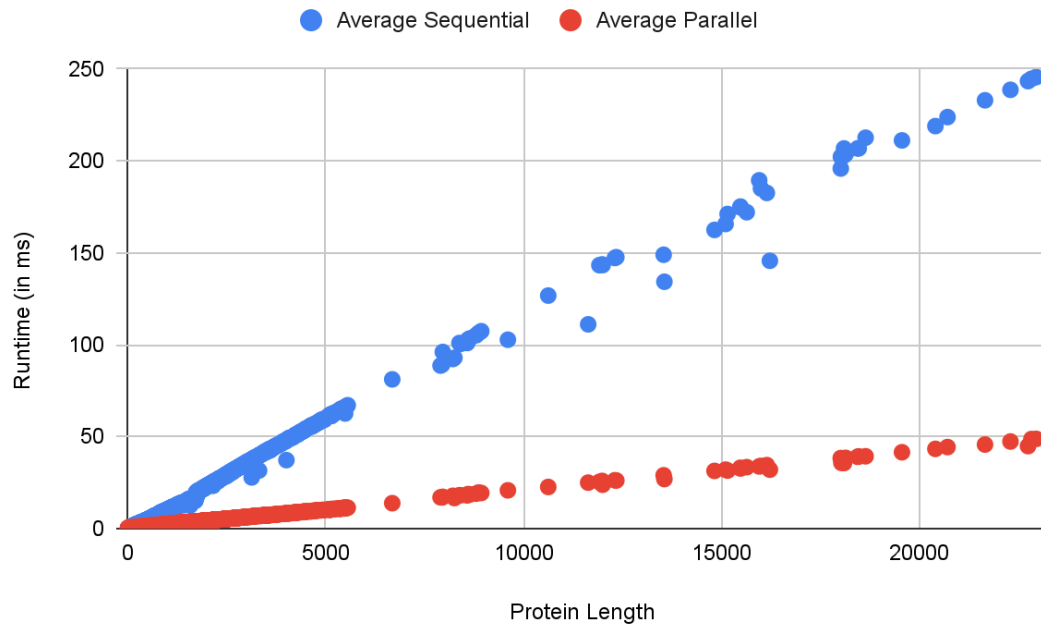


Figure 6-46: Runtime vs. Sequence Length for Sequential and Parallel Implementations (for Fruit Fly Dataset)

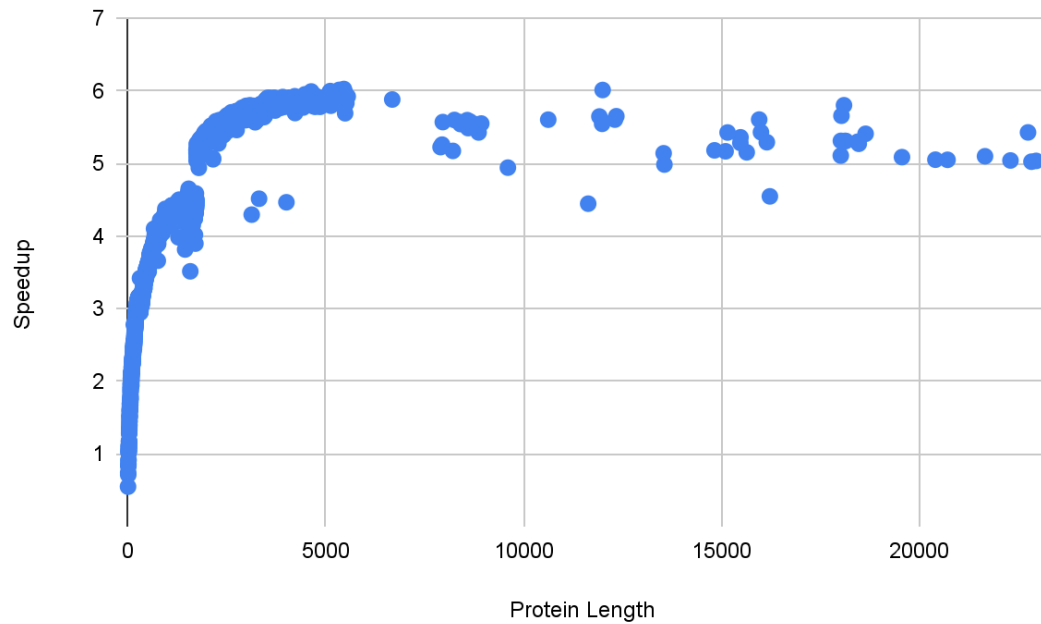


Figure 6-47: Speedup vs. Sequence Length for Sequential and Parallel Implementations (for Fruit Fly Dataset)

	Sequential (in ms)	SIMT (in ms)	Speedup
Mean	7.1402	1.7300	3.5104x
Median	4.5557	1.3112	3.4797x
S. Dev	10.8383	1.9551	—
Min	0.1606	0.2903	0.5468x
Max	245.6083	48.7752	6.0252x



Table 6-6: Summary Statistics for Sequential and SIMT Implementation (for Fruit Fly Dataset)

Sequential (in s)	SIMT (in s)	Speedup
153.1505	37.1068	4.1273x

Table 6-7: Total Runtime and Speedup for Sequential and SIMT Implementation (for Fruit Fly Dataset)

The results show that the CUDA implementation offers significant speedups, especially for longer sequences; however, for shorter sequences, it can become slower. This performance behaviour is expected in GPU-based systems, where parallel overhead favors batch processing of larger, more complex workloads, and for shorter sequences, this overhead may overtake any potential speedups by the implemented parallelism, resulting in slowdowns. Additionally, the average speedup of 3.51x demonstrates that CUDA's parallel processing model is moderately effective for this type of dynamic programming algorithm. Notably, the speedup increases with longer sequences, reaching up to 6.02x speedup, showing that GPU parallelization becomes more effective as the computational workload per alignment increases.

Given the sparseness of protein lengths in our dataset, relying solely on the mean speedup could be misleading. Hence, to provide an additional form of performance representation, the median and standard deviation are also provided. The median speedup is 3.48x, which suggests that the distribution of speedups is fairly symmetrical and not heavily skewed by outliers. This indicates that the mean speedup of 3.51x is a reasonably good representation of the typical speedup for this particular dataset.

When obtaining the total runtime for the fruit fly dataset, the CUDA implementation obtains an overall speedup of 4.13x. This showcases the improvements given by the implementation for the given dataset.

To further reinforce our analysis, a second stress test on a total of 39,116 proteins of the Mouse Ear Cress reference proteome is aligned to the 2 DNA input read queries. The protein sequences vary in length, from 5 bp to 5400 bp. The execution times of individual sequence alignments are graphed in Figure 6-48. Summary statistics can be found in Tables 6-8 and 6-9.

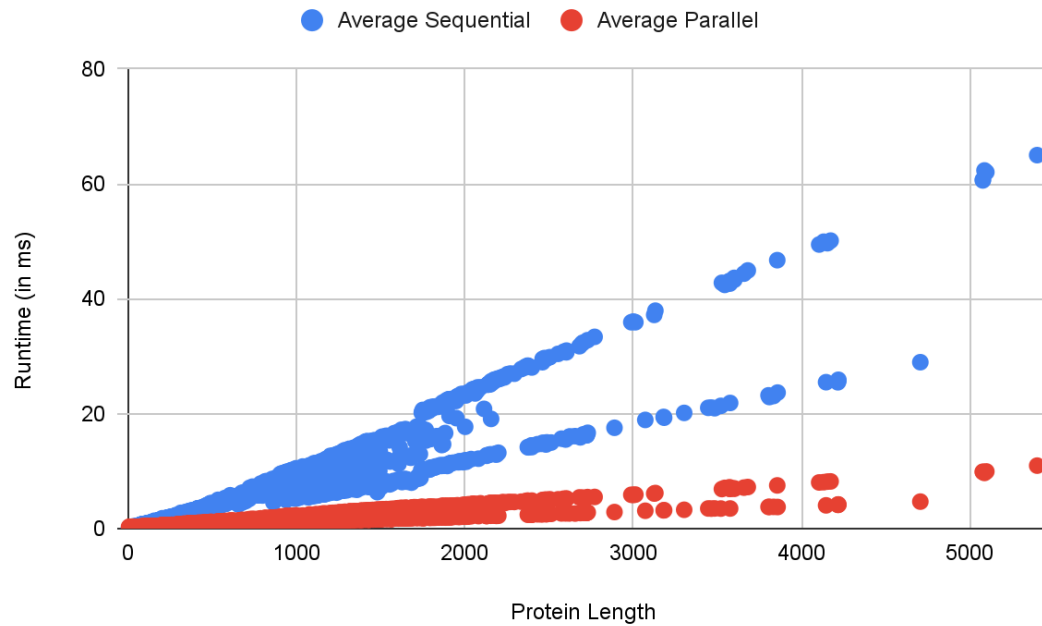


Figure 6-48: Runtime vs. Sequence Length for Sequential and Parallel Implementations (for Mouse Ear Cress Dataset)

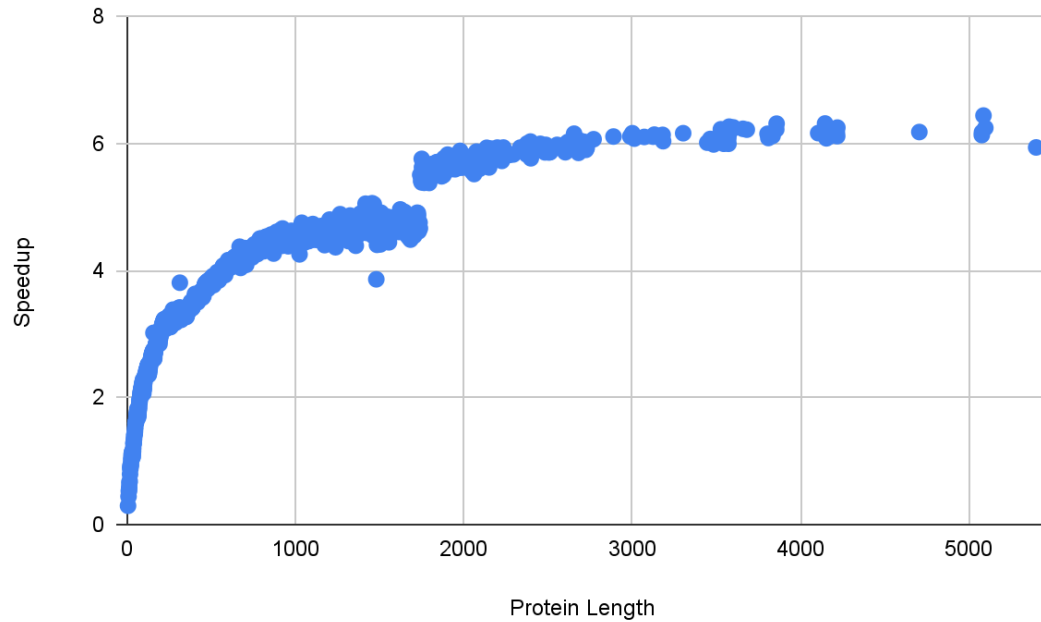


Figure 6-49: Speedup vs. Sequence Length for Sequential and Parallel Implementations (for Mouse Ear Cress Dataset)

	Sequential (in ms)	SIMT (in ms)	Speedup
Mean	3.4489	0.9023	3.4412x
Median	2.5876	0.7459	3.4098x
S. Dev	3.1978	0.6162	—
Min	0.0822	0.1450	0.2954x
Max	65.0062	10.9524	6.4371x

Table 6-8: Summary Statistics for SIMT and Sequential Implementation (for Mouse Ear Cress Dataset)



Sequential (in s)	SIMT (in s)	Speedup
134.9088	35.2936	3.8224x

Table 6-9: Total Runtime and Speedup for Sequential and SIMT Implementation (for Mouse Ear Cress Dataset)

The results further showcase the consistent performance of the CUDA implementation across different datasets. The data showed the same behaviour as the stress test for fruit fly, in which for very short sequences, GPU overhead results in lower speedups or even slowdowns, while increasing sequence lengths favor the parallel implementation. Despite the overall shorter protein lengths in this dataset, the SIMT version achieved a 3.44x average speedup, with maximum speedup reaching over 6.43x, which is similar to the results in the previous dataset.

In addition to this, the median speedup is 3.41x, which exhibits similar behavior to the fruit fly dataset and suggests a fairly symmetrical distribution of speedups, which indicates that the mean speedup is a reasonably good representation of the speedup for the mouse ear cress dataset.

6.2.2 SIMT vs SIMD

In this section, a comparison of the SIMT and SIMD implementations of the Algorithm was made. It was performed on the secondary testing platform, the laptop with an NVIDIA RTX 3050 Ti Mobile GPU and Ryzen 7 5800H processor, due to the previously mentioned Windows version compatibility issues. Additionally, the program was tested on the fourth testing platform, the desktop with a Ryzen 5 7600 processor. The program for SIMD was only executed for 9 loops due to the configuration of the program, with each run aligning a 150 bp DNA read sequence against 4096 protein sequences from the fruit fly reference proteome.

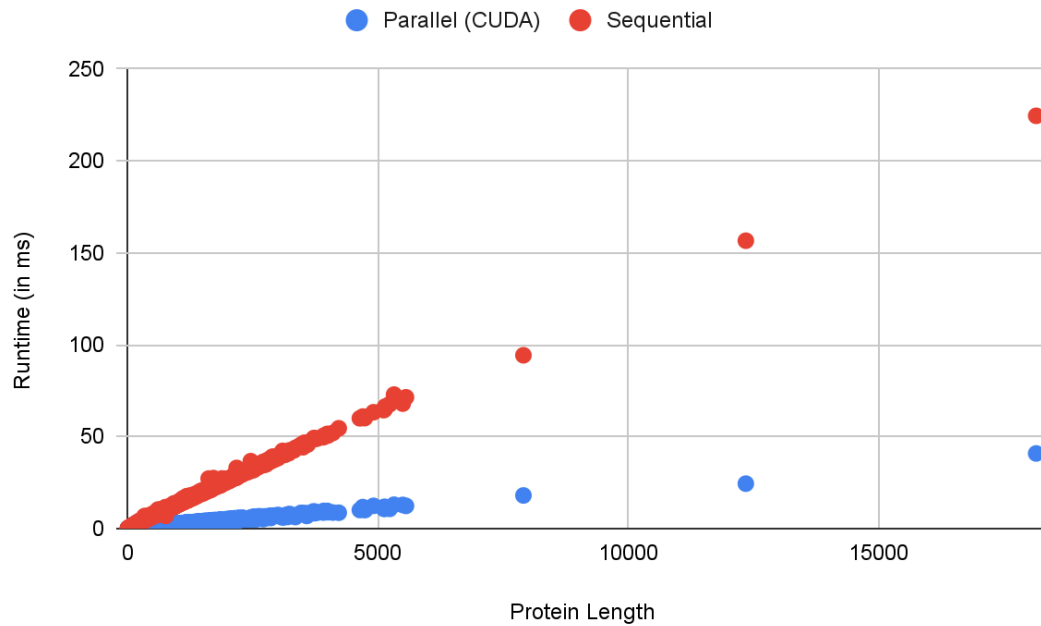


Figure 6-50: TARA CUDA Parallel Implementation vs Sequential Implementation

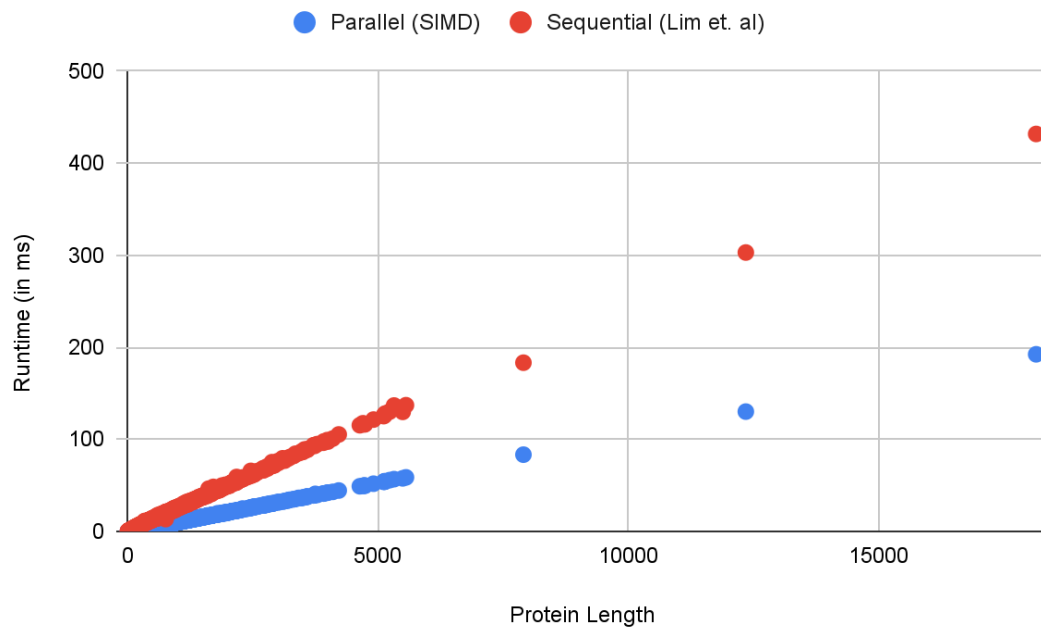




Figure 6-51: FICS Parallel (SIMD) implementation vs Sequential Implementation

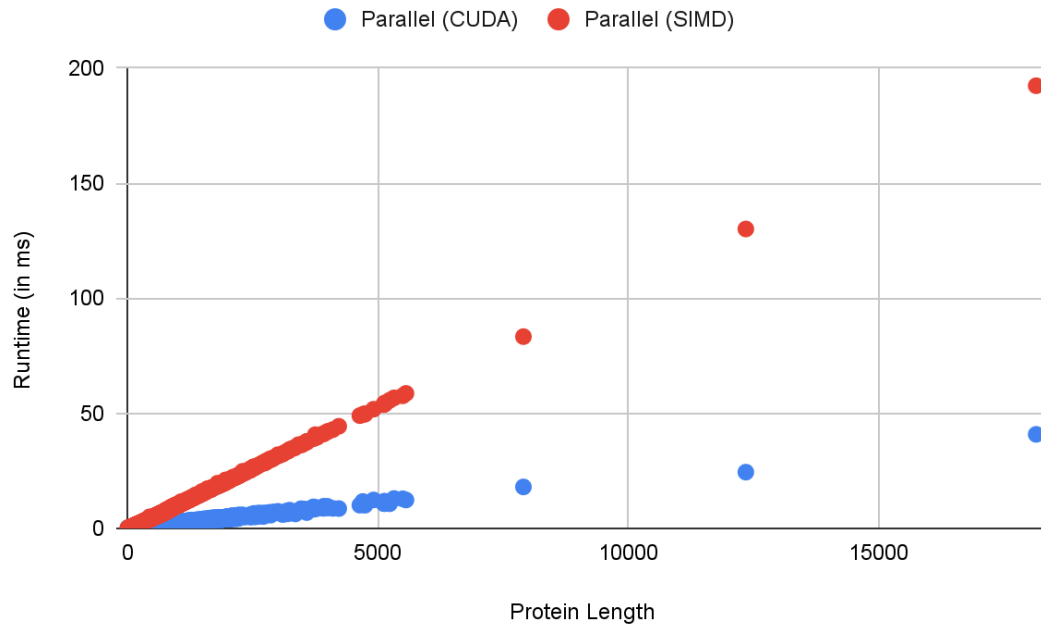


Figure 6-52: TARA CUDA Parallel Implementation vs FICS Parallel (SIMD) Implementation

	SIMD (in ms)	SIMT (in ms)	Speedup
Mean	6.4093	1.7712	3.1435x
Median	4.4573	1.4263	3.1018x
S. Dev	7.4766	1.5485	—
Min	0.1064	0.2353	0.4518x
Max	192.3529	40.8517	5.6077x

Table 6-10: Summary Statistics for SIMD and SIMT Implementation

From the results, the SIMD implementation recorded an average runtime of 6.4093 ms, while the CUDA implementation achieved a faster average runtime of 1.7712 ms. This resulted in an average speedup of approximately 3.14x. The median speedup of 3.10x once again validates the reliability of the mean to represent the speedup of this test.



Furthermore, the tests show that for longer sequences, the SIMT implementation shows a significant improvement in speedup compared to SIMD, achieving a maximum of 5.61x; however, there are slowdowns for CUDA when it comes to shorter sequences. This is because of the fixed-width vector registers for AVX2, which are readily available within the CPU and can be used for parallelization, and the fixed, minimal overhead of AVX2 instructions is easily amortized. CUDA, however, makes use of a GPU, which incurs overhead, especially across multiple kernel launches and data transfers, making it slower for shorter sequences.

To further examine the SIMD implementation and its varying device-to-device performance, the program was tested on an additional device with a Ryzen 5 7600. The results of this comparison are shown in Figures 6-53 and 6-54 and Table 6-11.

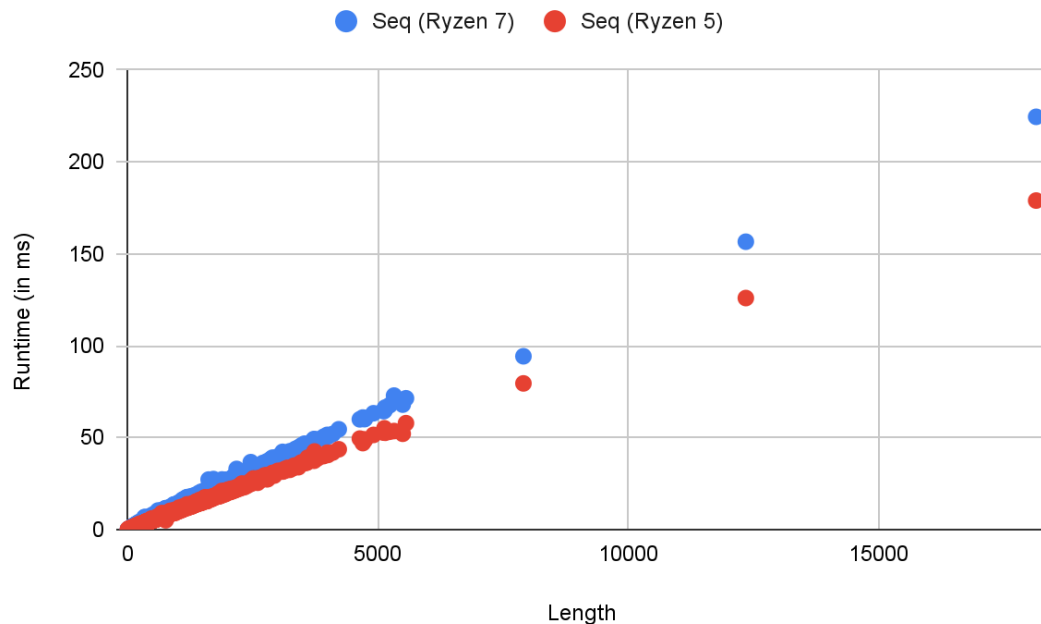


Figure 6-53: Sequential Runtime Comparison (Ryzen 7 vs Ryzen 5)

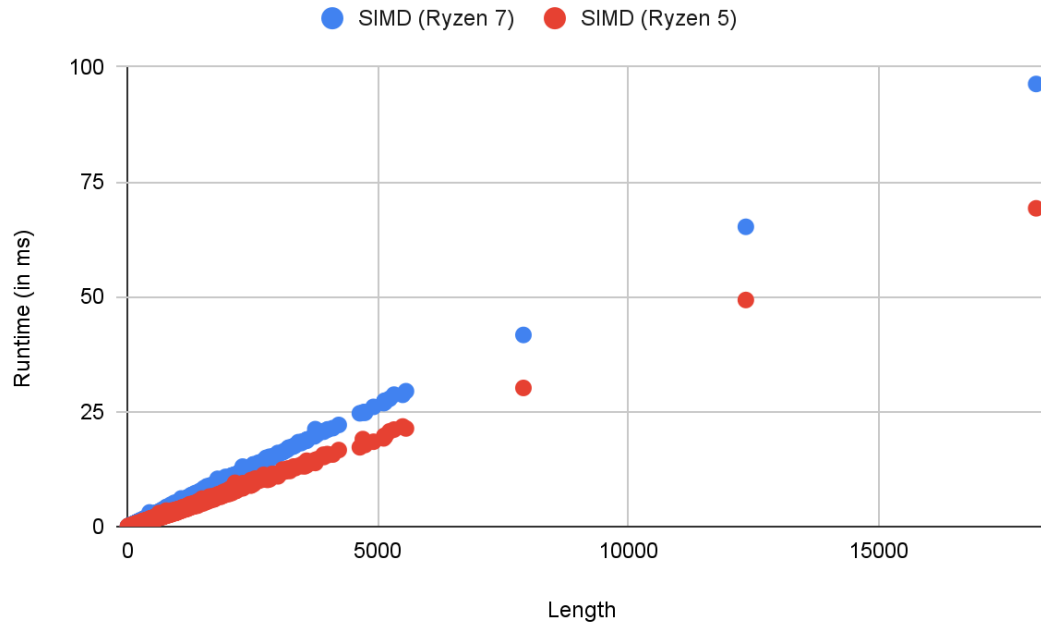


Figure 6-54: SIMD Runtime Comparison (Ryzen 7 vs Ryzen 5)

	Ryzen 7 5800H	Ryzen 5 7600
Average Sequential (in ms)	8.7565	6.7836
Average SIMD (in ms)	3.2152	2.2749
Average Speed-up	2.9271x	3.1326x

Table 6-11: Summary of Average Speed-up and Runtime per Implementation based on Different CPU

Based on the results in Table 6-11, the tests show that there is a performance gap with the Ryzen 7 5800H and Ryzen 5 7600 systems, having SIMD execution times of 3.21 ms and 2.27 ms respectively. This is attributed to the newer generation architecture of the 7000 series CPU which has better single-threaded performance, as well as being a desktop class CPU, given that AVX2 is still a single threaded operation when it comes to hardware. Despite that, both systems have similar performance with 2.93x and 3.13x speedups. This further shows the static nature of AVX2 parallelization.



6.3 Scalability Test

To evaluate the scalability and adaptability of TARA CUDA, two scalability tests were conducted: input scalability and compiler optimization. Input scalability examines how the runtime grows with increasing proteome input volume. While compiler optimization measures the impact of different compiler optimization levels on execution speed, particularly in the scoring phase.

6.3.1 Input Scalability

To evaluate the scalability of the developed implementation, a 2×2^k test configuration was conducted. In this test, a pair of DNA input sequences (150 bp) were aligned against varying numbers of sequences from the reference proteome, ranging from 2^8 to 2^{14} proteins in the Fruit Fly dataset and ranging from 2^8 to 2^{15} proteins in the Mouse Ear Cress dataset. The goal of this test is to observe how the runtime of both the sequential and CUDA implementations scales with increasing reference sequence count. The test was conducted on the primary testing platform (a Desktop with an NVIDIA RTX 4070S and Intel i7-12700F).

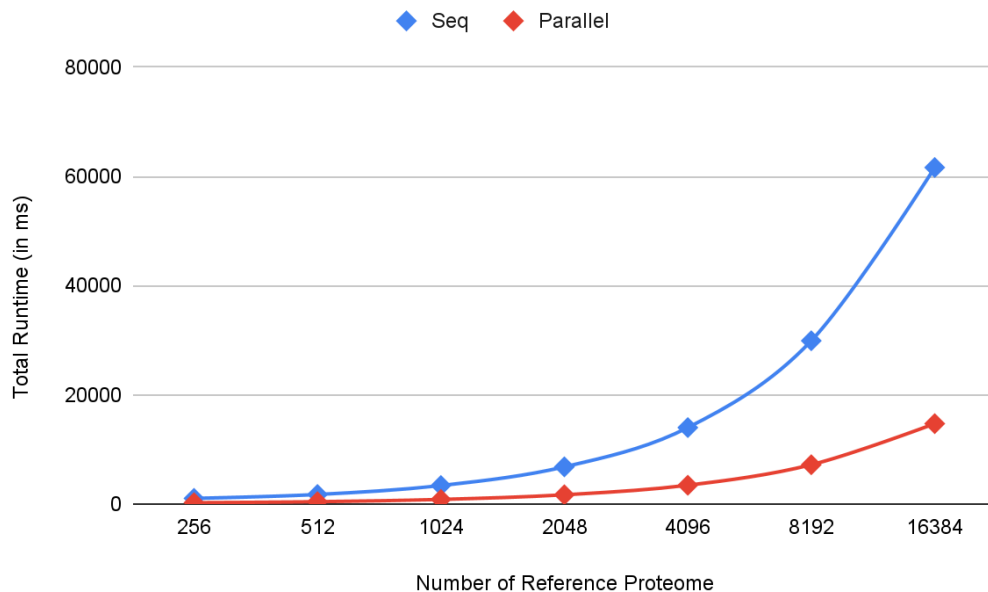


Figure 6-55: Scalability of Fruit Fly Dataset

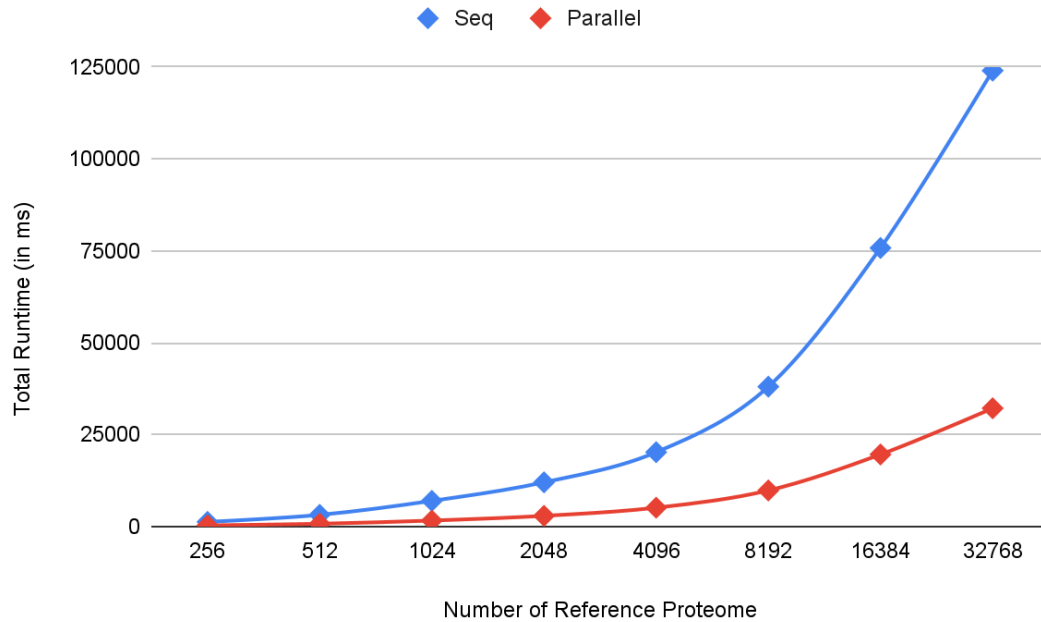


Figure 6-56: Scalability of Mouse Ear Cress Dataset

Figures 6-55 and 6-56 show that both the sequential and parallel implementations exhibit non-linear growth. Initially, at smaller sequence counts, the difference in execution times is also relatively small. As the sequence count increases, however, the absolute difference between the two implementations becomes significantly larger. This indicates that the parallel implementation scales more favorably as datasets become larger. While both implementations curve upward, the parallel execution time curve's slope is noticeably less steep. Thus, the results clearly show that the parallel implementation exhibits significantly better scalability as compared to the sequential one, and grows at a much slower rate. Additionally, we observe the different stages of the program and analyze the relationship between the percentages of the different stages of the program with reference sequence counts. The results are shown in Figures 6-57 and 6-58, and Tables 6-12 and 6-13.

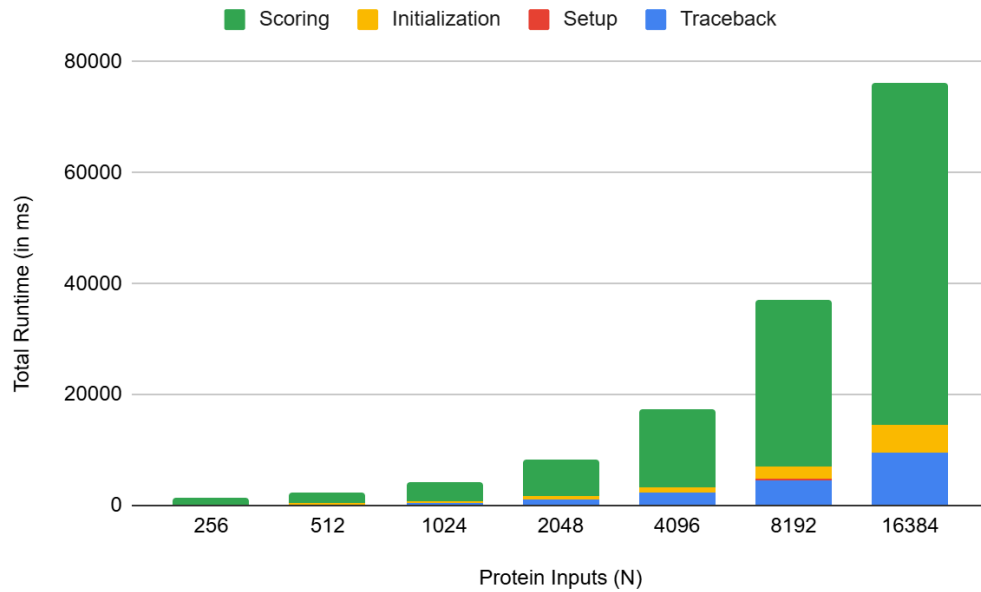


Figure 6-57: Scalability of the Runtime Contribution of Each Phase (Sequential)

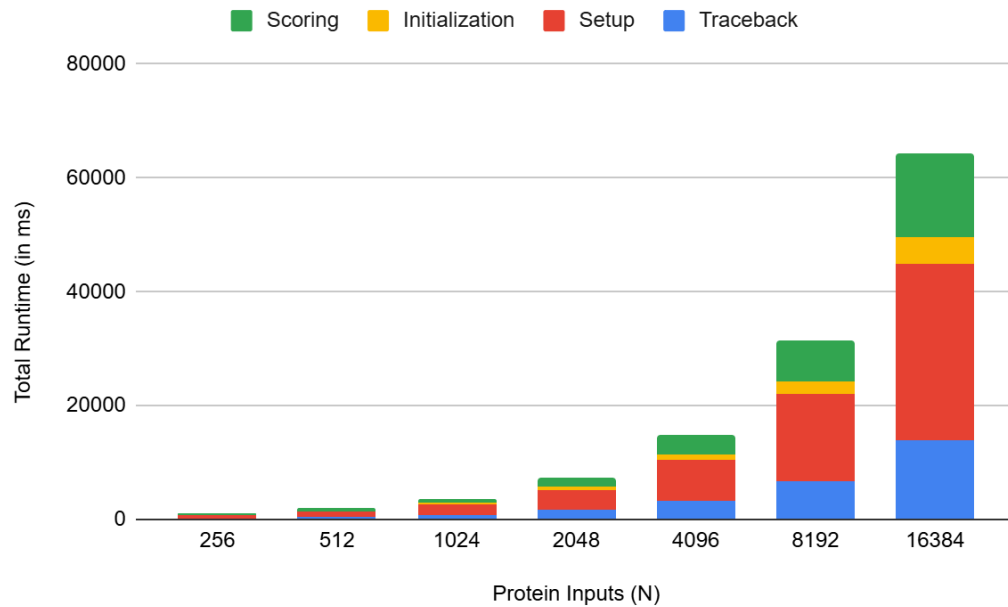


Figure 6-58: Scalability of the Runtime Contribution of Each Phase (Parallel)



	Setup (in ms)	Initialization (in ms)	Scoring (in ms)	Traceback (in ms)	Total
256	3.8860 (0.30%)	92.3057 (7.12%)	1041.9393 (80.42%)	157.4784 (12.15%)	1295.6095
512	6.5797 (0.30%)	148.7886 (6.84%)	1750.4340 (80.44%)	270.2063 (12.42%)	2176.0086
1024	12.1077 (0.29%)	266.9724 (6.37%)	3384.3033 (80.74%)	528.3830 (12.61%)	4191.7664
2048	23.4261 (0.28%)	513.5938 (6.11%)	6803.7110 (80.94%)	1065.6262 (12.68%)	8406.3572
4096	48.4944 (0.28%)	1082.7895 (6.26%)	13995.0016 (80.88%)	2177.4801 (12.58%)	17303.7656
8192	102.3685 (0.28%)	2372.0643 (6.41%)	29925.7021 (80.85%)	4614.8657 (12.47%)	37015.0007
16384	201.0066 (0.26%)	4836.5212 (6.34%)	61761.0711 (80.96%)	9486.8561 (12.44%)	76285.4550

Table 6-12: Summary of Total Runtime Contribution of Each Phase (Sequential)

	Setup (in ms)	Initialization (in ms)	Scoring (in ms)	Traceback (in ms)	Total
256	508.4128 (47.34%)	92.8346 (8.64%)	242.4676 (22.58%)	230.1847 (21.43%)	1073.8998
512	910.3084 (48.11%)	150.0760 (7.93%)	428.8005 (22.66%)	402.8938 (21.29%)	1892.0788
1024	1771.5835 (48.19%)	268.3095 (7.30%)	844.4160 (22.97%)	791.5481 (21.53%)	3675.8571



2048	3541.3975 (48.20%)	509.8643 (6.94%)	1701.1427 (23.15%)	1595.3716 (21.71%)	7347.7762
4096	7211.7521 (48.09%)	1074.5773 (7.17%)	3457.5802 (23.06%)	3251.9264 (21.69%)	14995.8360
8192	15059.1698 (47.79%)	2364.3335 (7.50%)	7243.2962 (22.99%)	6844.1329 (21.72%)	31510.9325
16384	30850.5456 (47.86%)	4804.9875 (7.45%)	14812.6448 (22.98%)	13989.0137 (21.70%)	64457.1915

Table 6-13: Summary of Total Runtime Contribution of Each Phase (Parallel)

Based on the results shown above, we can see that the proportions of the program for the different phases are consistent with each other as the dataset size grows for both the sequential and parallel implementations. Even when taking into account the setup overhead of the parallel implementation, it is still consistently outperforming the sequential implementation. This shows that the performance of the implementation does not deteriorate as the number of sequence reads increases.

6.3.2 Compiler Optimization

To further analyze the scalability of the program, the impact of compiler optimization levels was also considered. Different optimization settings in Visual Studio 2022 were used during compilation:

- /Od (Disable Optimization): This flag explicitly disables code optimizations. This is the default for debug builds.
- /O1 (Minimize Size): Prioritizes the generation of smaller executable code.
- /O2 (Maximize Speed): Prioritizes the generation of the fastest possible executable code. This is the default for release builds.
- /Ox (Full Optimization): A subset of /O2 that exclude certain options to balance speed and size optimizations.

In order to analyze the impact of compiler optimization on CUDA device code and the Parallel Thread Execution (PTX) code generated by nvcc, different compiler behavior optimization level flags on the command line when building the program were also tested:

- O0 (No Optimization): Disables most high-level and mid-level optimizations when generating PTX code. This is the default behavior for release builds.



- O1 (Basic Optimization): Applies a set of fundamental, lightweight optimizations to PTX generation.
- O2 (Moderate Optimization): Applies a comprehensive set of optimizations to maximize performance of generated PTX code.
- O3 (Aggressive Optimization): Applies aggressive and time-consuming optimizations during PTX code generation.

More importantly, it is possible to specify options directly to `ptxas`, the PTX optimizing assembler that generates the Streaming Assembler (SASS) code, using `-Xptxas`. Thus, optimization level flags were also specified:

- O0 (Disable PTXAS Optimization): Disables most optimizations performed when generating SASS code.
- O1 (Basic PTXAS Optimization): Applies fundamental optimizations during SASS generation.
- O2 (Moderate PTXAS Optimization): Applies a comprehensive set of aggressive, hardware-aware optimizations to maximize performance of generated SASS code.
- O3 (Aggressive PTXAS Optimization): Applies the most aggressive and time-consuming optimizations during SASS code generation. This is the default behavior for release builds.

Thus, when testing optimizations for the parallel implementation, both optimization levels were specified as shown in Figure 6-59. Each test used the same input: two DNA sequences, both 150 bp, aligned to a 4096 reference proteome (fruit fly). The results for sequential optimization are shown in Table 6-14 and Figure 6-59, while parallel optimization results are shown in Table 6-15 and 6-60.

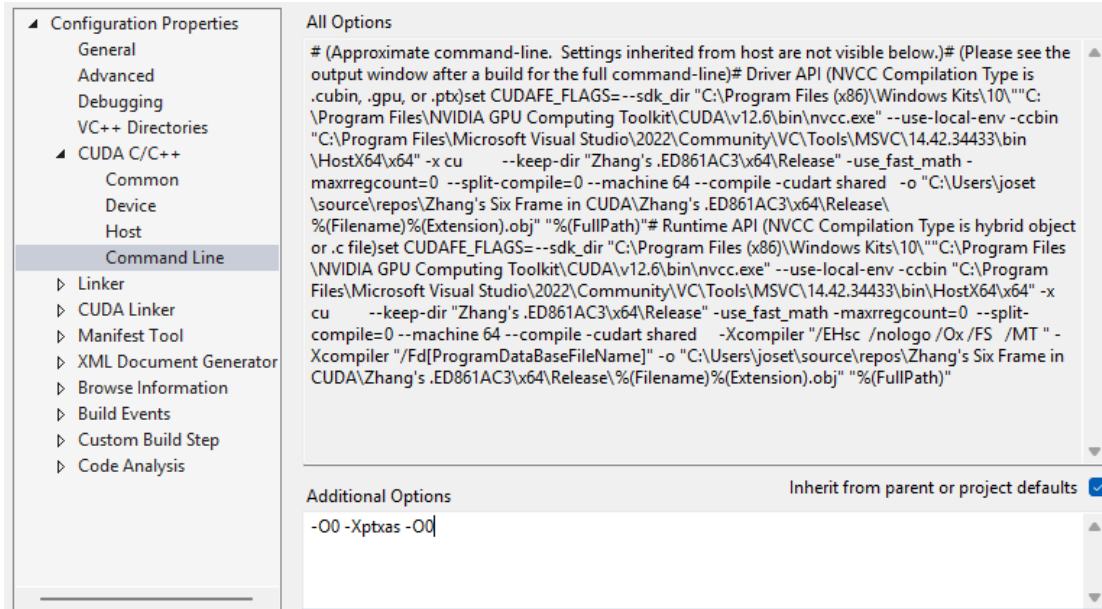


Figure 6-59: Sample Command Line and Additional Options

	0d	01	02	0x
Sequential (in ms)	19.6686	10.6795	10.4561	10.8321

Table 6-14: Average Runtime for Sequential Implementation on Different Compiler Optimization Levels

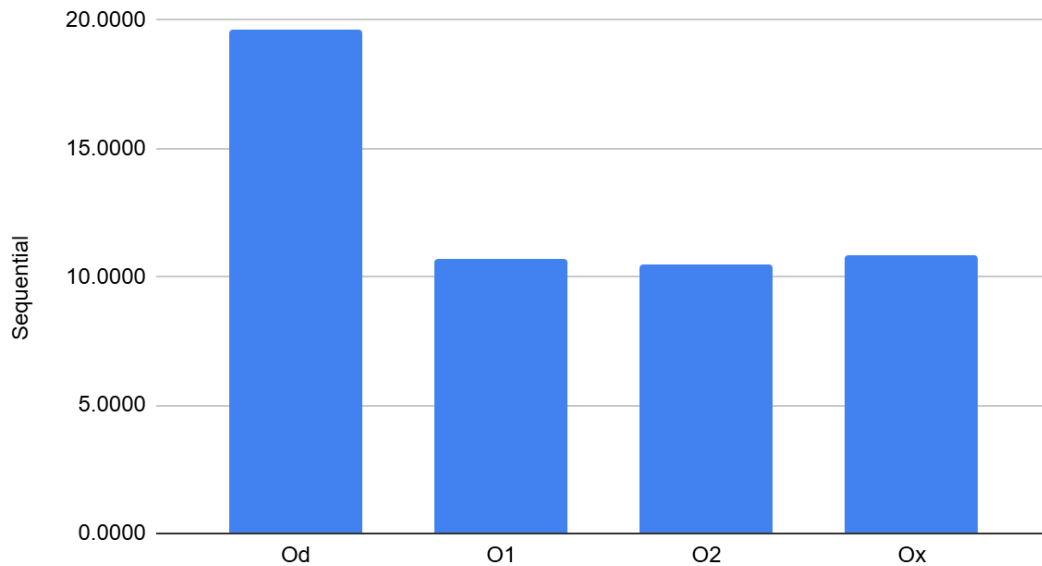


Figure 6-60: Average Runtime Comparison of Sequential Implementation Across Compiler Optimization Levels

For the impact of compiler optimization on the sequential implementation, as seen in Table 6-14 and Figure 6-60, we can observe an approximate $\sim 1.8x$ speedup for the code when optimized. The different optimization levels are generally close to each other, with /O2 pulling slightly ahead as expected, with /O1 following close behind and /Ox being the least performant optimization level. While it is expected for /O2 to perform the best, obtaining results where /O1 is ahead of /Ox suggests that the implementation is somewhat cache-sensitive which negates some of the optimizations, but not enough to overtake /O2.

	O0	O1	O2	O3
Parallel (in ms)	11.4083	2.3834	2.5280	2.3915

Table 6-15: Average Runtime for Parallel Implementation on Different Compiler Optimization Levels

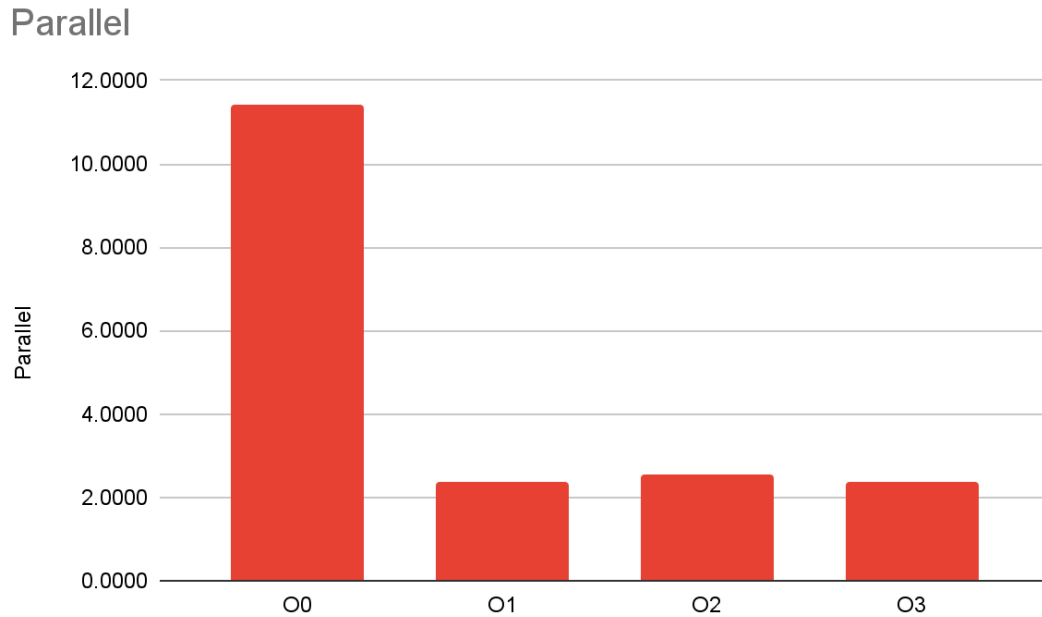


Figure 6-61: Average Runtime Comparison of Parallel Implementation Across Compiler Optimization Levels

For the impact of compiler optimization on the parallel implementation, as seen in Table 6-15 and Figure 6-61, we can observe a speedup of about $\sim 4.6x$ when comparing unoptimized code, which shows the significant impact of low-level and hardware-specific optimizations to the performance of CUDA programs. The different optimization levels are generally similar, with O1 performing best, followed by O3, then O2. This suggests some complexities on the combinations of the different optimization levels: O1 received basic O1 optimization for both the compiler and assembler, which suggests that less aggressive optimizations may allow the ptxas to produce more optimal SASS. This is shown with O2's more moderate optimizations for both which resulted in less performance. O3 received the default behavior optimizations; that is, O0 for the compiler but O3 for the assembler and resulted in performance that was only very slightly behind O1. This reinforces the idea that an optimal balance may be found to extract the most performance out of a given CUDA program.

6.4 Cross-Platform Performance Test

The Cross-Platform Performance Test was conducted to evaluate how the implemented program performs across different hardware platforms. This test provides insights into



the adaptability, efficiency, and performance scalability of the algorithm when executed on GPUs of varying capabilities and across different levels of platforms. Three platforms were examined in this section, namely, the desktop with an Intel i7-12700F and NVIDIA RTX 4070S, the laptop with a Ryzen 7 5800H and NVIDIA RTX 3050 Ti Mobile, and the CCSCloud Jupyter Environment with an QEMU Virtual CPU emulated Intel Xeon Silver 4210 and NVIDIA Tesla V100 PCIe 32 GB.

For each platform test, the implementation uses the same input sequences. The results are shown in Figures 6-61 to 62 and Tables 6-16 to 18.

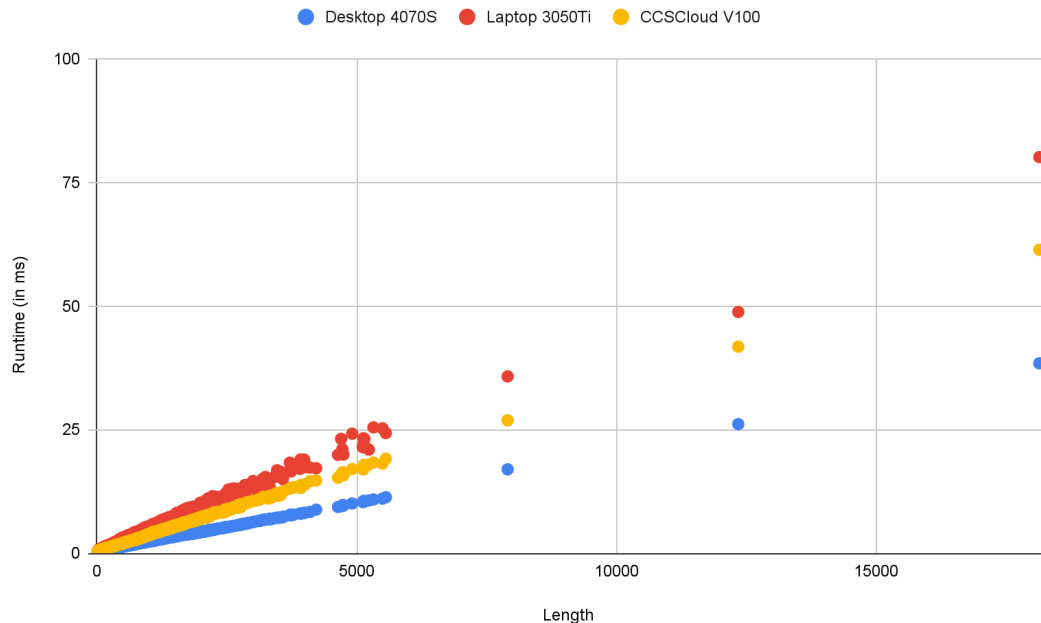


Figure 6-62: Parallel Implementation Runtime Across GPU Platforms

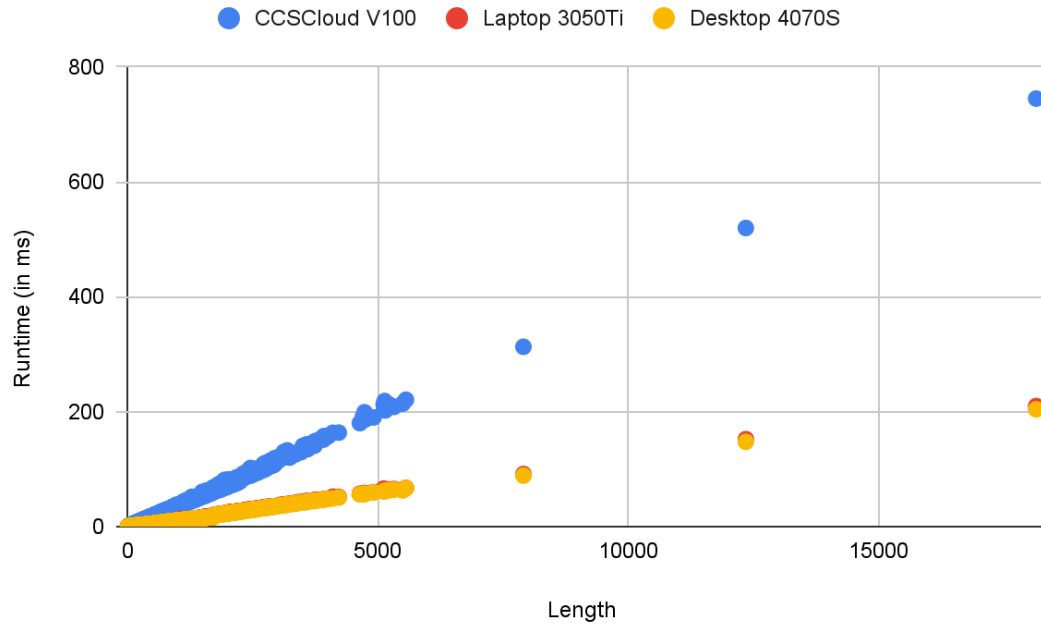


Figure 6-63: Sequential Implementation Runtime Across GPU Platforms

	Laptop 3050 Ti	CCSCloud V100	Desktop 4070S
Average	1.7544x	8.5373x	3.5212x
Min	0.3225x	1.1129x	0.5468x
Max	3.2502x	12.4351x	5.9915x

Table 6-16: Speedup Comparison of CUDA vs Sequential Across GPU Platforms

	Laptop 3050 Ti	CCSCloud V100	Desktop 4070S
Speedup	1.000x	1.3284x	2.1081x

Table 6-17: Speedup Comparison of CUDA Across Different GPUs



	Laptop 3050Ti (in ms)		CCSCloud V100 (in ms)		Desktop 4070S (in ms)	
	Sequential	SIMT	Sequential	SIMT	Sequential	SIMT
Average	11.1005	5.0417	37.4361	3.7952	10.8321	2.3915
Min	0.1619	0.4788	0.4767	0.421	0.1638	0.2983
Max	209.7535	80.1586	744.9806	61.4128	204.0286	38.4254

Table 6-18: Runtime Comparison Across GPU Platforms

From the speedup results, we can initially conclude that the Tesla V100 may have the best performance with the highest average speedup of 8.54x. However, this exceptional speedup is influenced by the unusually high sequential runtime on the CCSCloud platform, as seen in Table 6-18. While the systems that tested 3050 Ti and 4070S have sequential runtimes averaging around 11 ms, the Tesla V100 takes over 37 ms on average, which is 3.5x slower than the other platforms.

This discrepancy is likely due to the Intel Xeon Silver 4210 emulated by the virtualized CPU environment used in CCSCloud, which is older and has relatively poorer single thread performance, given that it is designed for workstations, whereas the Intel i7-12700F and the Ryzen 7 5800H are newer consumer CPUs and possess better performance.

As a result, the RTX 4070S records the lowest average runtimes for the parallel implementation at 2.39 ms, outperforming both the V100 and 3050 Ti, which have average runtimes of 3.80 ms and 5.04 ms, respectively. With the slowest device (RTX 3050 Ti) as a baseline, the Tesla V100 obtains a 1.33x speedup, while the RTX 4070S obtains a 2.10x speedup.

This behavior aligns with expectations, as one of the many factors that directly affects the performance of a GPU would be the number of CUDA cores, as well as the architecture and thus, performance of each core. The 4070S features 7168 CUDA cores based on the latest Ada Lovelace architecture. In contrast, the Tesla V100, equipped with 5120 CUDA cores, is built on the older Volta architecture, and the 3050 Ti Mobile has only 2560 CUDA cores built on the Ampere architecture.



7. Conclusion & Recommendations

7.1 Conclusion

This study demonstrated the capabilities of the SIMT execution model to reduce computational costs and speed up execution times of Frame alignment algorithms that possess inherent data dependencies, specifically Zhang's Six-Frame Algorithm. The researchers were able to show that the wavefront anti-diagonal paradigm can be utilized and implemented in CUDA SIMT, and achieved this by leveraging a hybrid multi-level anti-diagonal wavefront technique, a modified version of the anti-diagonal technique to fit the parallel framework of the SIMT model. Through performing several evaluation methods, including multiple correctness checks, sequential vs CUDA runtime comparison, SIMD (AVX2) vs CUDA runtime comparison, scalability testing, and cross-platform performance evaluations, the study showcased the improved performance of the core computational step of the algorithm, specifically on the scoring portion and overall performance of the program, how the implementation behaves with varying input data, the effectiveness of optimization options, and the speed difference of the program across different platforms used. Noticeably, the CUDA implementation obtains a speedup over the Sequential implementation of about 3.5x, and is also competitive with the SIMD (AVX2) implementation, obtaining speedups of 3.1x on average. Compiler optimization also significantly affects the speed of both the sequential and CUDA implementations, but there are some nuances, especially with regard to optimizing CUDA programs. It also explored the performance capability of the program across different platforms, reinforcing the significant direct impact of GPU hardware quality on the performance of GPU code. This study serves as proof of concept for the utilization of GPUs in the field of bioinformatics, especially for more complex algorithms such as Zhang's Six Frame. This will enable faster processing of data in the field of bioinformatics, which will allow faster downstream analysis in the field of genetics.

7.2 Recommendations

The findings of the study bring to light several recommendations for future work. Further optimizations to the implementation can be made. First, the researchers' decision to utilize a hybrid implementation scheme inherently introduces some overhead, given the implementation's partial reliance on host-side code. Thus, it is recommended to explore the possibility of greater performance of a purely device-code implementation. To further improve the scoring process of Zhang's Six-Frame Algorithm in CUDA, it can be recommended to explore the use of libraries of CUDA C/C++ that can lessen the overhead/computation cost brought by the use of native C/C++ functions and code that may not be optimized for use in GPU code. Given the level of proficiency of the



researchers in writing CUDA C/C++ code currently, this is the most efficient method that the researchers utilized in this implementation. The use of a purely device implementation scheme also allows for more extensive performance profiling and analysis, particularly with regard to using NVIDIA's Nsight suite of developer tools. Roofline analysis can be conducted on the implemented code, which was deemed unusable for the implemented hybrid approach, and is thus recommended over the hybrid multilevel scheme currently utilized. Additionally, the possibility of exploring speedups in the other phases of the program can be considered, including performing initialization using device code, as well as in the setup phase, particularly with regard to data transfers between the device and the host. These memory transfers can be performed asynchronously and concurrently, further reducing overhead and improving the true speedup of the overall implementation. Some improvements to the testing methodology can also be made, particularly with regard to testing performance across multiple devices. Given that the focus of the study is the performance of the GPU code, it would be more advisable to use a single system as a test bench and replace the GPU being tested to provide a more accurate comparison across different devices.



Appendix A. Bibliography

- [1] S. Tamang, "Sequence Alignment- Definition, Types, Methods, uses," Microbe Notes, May 25, 2024. <https://microbenotes.com/local-global-multiple-sequence-alignment/?form=MG0AV3>
- [2] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *Journal of Molecular Biology/Journal of molecular biology*, vol. 147, no. 1, pp. 195–197, Mar. 1981, doi: [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5).
- [3] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic local alignment search tool," *Journal of Molecular Biology/Journal of molecular biology*, vol. 215, no. 3, pp. 403–410, Oct. 1990, doi: [https://doi.org/10.1016/s0022-2836\(05\)80360-2](https://doi.org/10.1016/s0022-2836(05)80360-2).
- [4] H. Li and R. Durbin, Fast and accurate short read alignment with Burrows-Wheeler Transform. *Bioinformatics*, 25:1754-60, 2010, Sourceforge.net, 2010. <https://bio-bwa.sourceforge.net/>.
- [5] Yao, Y., & Frith, M. C. (2023). Improved DNA-Versus-Protein Homology Search for Protein Fossils. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 20(3), 1691–1699. <https://doi.org/10.1109/tcbb.2022.3177855>
- [6] Frith, M. C. (2022). Paleozoic Protein Fossils Illuminate the Evolution of Vertebrate Genomes and Transposable Elements. *Molecular Biology and Evolution*, 39(4). <https://doi.org/10.1093/molbev/msac068> -39
- [7] Peiris, M., J. A. A. M. Jayaweera, & Ganegoda, G. U. (2022). Melanoma Detection by Analysing Mutations in Gene DNA Sequences and Their Primary Protein Structures. 2, 1–6. <https://doi.org/10.1109/iciprob54042.2022.9798719>
- [8] Santiago, K. C., & Shrestha, A. (2024). DNA-protein quasi-mapping for rapid differential gene expression analysis in non-model organisms. *BMC Bioinformatics*, 25(S2). <https://doi.org/10.1186/s12859-024-05924-1>
- [9] Custer, J. M., White, R., Taylor, H., Schmidlin, K., Fontenele, R. S., Stainton, D., Kraberger, S., Briskie, J. V., & Arvind Varsani. (2021). Diverse single-stranded DNA viruses identified in New Zealand (Aotearoa) South Island robin (*Petroica australis*) fecal samples. *Virology*, 565, 38–51. <https://doi.org/10.1016/j.virol.2021.10.004>
- [10] Frith, M. C., Wan, R., & Horton, P. (2010). Incorporating sequence quality data into alignment improves DNA read mapping. *Nucleic Acids Research*, 38(7),



e100–e100. <https://doi.org/10.1093/nar/gkq010>

[11] BLAST. (2015). blastx: Align translated DNA with proteins using BLAST. Nih.gov.

https://blast.ncbi.nlm.nih.gov/Blast.cgi?PAGE=Translations&PROGRAM=blastx&BLAST_PROGRAMS=blastx&PAGE_TYPE=BlastSearch&BLAST_SPEC=blast2seq

[12] Buchfink, B., Xie, C., & Huson, D. H. (2014). Fast and sensitive protein alignment using DIAMOND. *Nature Methods*, 12(1), 59–60. <https://doi.org/10.1038/nmeth.3176>

[13] Z. Zhang, W. R. Pearson, and W. Miller, “Aligning a DNA Sequence with a Protein Sequence,” *ResearchGate*, Feb. 1997. https://www.researchgate.net/publication/13942672_Aligning_a_DNA_Sequence_with_a_Protein_Sequence.

[14] Lim, S. E., Lim, S. V., Ting, C. L., Wong, A. E., & Uy, R. L. (2021). SIMD Implementation of Local Alignment-Based Six-Frame Alignment Algorithm. *Philippine Computing Journal*, 17(1).

[15] R. Stark, M. Grzelak, and J. Hadfield, “RNA sequencing: the teenage years,” *Nature reviews. Genetics*, vol. 20, no. 11, pp. 631–656, Jul. 2019, doi: <https://doi.org/10.1038/s41576-019-0150-2>.

[16] S. Sarkar, Gaurav Ramesh Kulkarni, Partha Pratim Pande, and Ananth Kalyanaraman, “Network-on-Chip Hardware Accelerators for Biological Sequence Alignment,” *I.E.E.E. transactions on computers/IEEE transactions on computers*, vol. 59, no. 1, pp. 29–41, Jan. 2010, doi: <https://doi.org/10.1109/tc.2009.133>.

[17] N. J. Curtis, K. E. Niemeyer, and C.-J. Sung, “Using SIMD and SIMT vectorization to evaluate sparse chemical kinetic Jacobian matrices and thermochemical source terms,” *Combustion and flame*, vol. 198, pp. 186–204, Dec. 2018, doi: <https://doi.org/10.1016/j.combustflame.2018.09.008>.

[18] M.-S. Yu, C.-Y. Yuan, T.-L. Chen, and J.-K. Lee, “Case Study: Optimization Methods with TVM Hybrid-OP on RISC-V Packed SIMD,” *ResearchGate*, Jan. 2024. <https://doi.org/10.1109/ACCESS.2024.3397195>

[19] NVIDIA, “Whitepaper NVIDIA’s Next Generation CUDA™ Compute Architecture: Fermi™ V1.1.”, 2009, Available: https://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf



[20] K. Karimi, N. Dickson, and F. Hamze, "A Performance Comparison of CUDA and OpenCL." Available: <https://arxiv.org/vc/arxiv/papers/1005/1005.2581v1.pdf>

[21] Stanley Vincent Lim, Steven Edward Lim, Carlos Louis Ting, Aaron Eldrich Wong, and Roger Luis Uy, "SIMD Implementation of Modified Zhang's Three-Frame Alignment Algorithm," 2021 IEEE 13th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM), Nov. 2021, doi: <https://doi.org/10.1109/hnicem54116.2021.9731997>.

[22] A. Khajeh-Saeed, S. Poole, and J. B. Perot, "Acceleration of the Smith–Waterman algorithm using single and multiple graphics processors," *Journal of Computational Physics*, vol. 229, no. 11, pp. 4247–4258, Jun. 2010, doi: 10.1016/j.jcp.2010.02.009..

[23] Ranasinghe D. N. and Siriwardena T. R. P. "An efficient implementation of Smith Waterman algorithm on GPU using CUDA, for massively parallel scanning of sequence databases," *IEEE Conference Publication | IEEE Xplore*, May 01, 2009. <https://ieeexplore.ieee.org/abstract/document/5160931>

[24] X. Guo, H. Wang, and V. Devabhaktuni, "A systolic Array-Based FPGA parallel architecture for the BLAST algorithm," *ISRN Bioinformatics*, vol. 2012, pp. 1–11, Sep. 2012, doi: 10.5402/2012/195658.

[25] P. L. Miller, P. M. Nadkarni, and N. M. Carriero, "Parallel computation and FASTA: confronting the problem of parallel database search for a fast sequence comparison algorithm," *Bioinformatics*, vol. 7, no. 1, pp. 71–78, Jan. 1991, doi: 10.1093/bioinformatics/7.1.71.

[26] Y. S. Lee, Y. S. Kim, and R. L. Uy, "Serial and parallel implementation of Needleman-Wunsch algorithm," *IJAIN (International Journal of Advances in Intelligent Informatics)*, vol. 6, no. 1, p. 97, doi: 10.26555/ijain.v6i1.361.

[27] F. Muhamad, R. Ahmad, S. Asi, and M. Murad, "Performance analysis of Needleman-Wunsch Algorithm (Global) and Smith-Waterman Algorithm (Local) in reducing search space and time for DNA sequence alignment," *Journal of Physics. Conference Series*, vol. 1019, p. 012085, Jun. 2018, doi: 10.1088/1742-6596/1019/1/012085.

[28] Kielbasa, S., Wan, R., Sato, K., Horton, P., & Frith, M. (2010). Adaptive seeds



tame genomic sequence comparison. Computational Biology Research Center. <https://genome.cshlp.org/content/suppl/2011/01/06/gr.113985.110.DC1/kielbasa-som.pdf>

[29] Zhuo, Y., Zhang, T., Du, F., & Liu, R. (2023). A parallel particle swarm optimization algorithm based on GPU/CUDA. *Applied Soft Computing*, 144, 110499–110499. <https://doi.org/10.1016/j.asoc.2023.110499>

[30] Chakraborty, A. (2019). Data parallelism vs Task parallelism. Tutorialspoint. <https://www.tutorialspoint.com/data-parallelism-vs-task-parallelism>

[31] Nvidia. (2024). CUDA C++ Programming Guide. Nvidia.com. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

[32] NVIDIA. (2024). CUDA Driver API Reference Manual. In CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/cuda-driver-api/index.html>

[33] S. Wang, W. Li, Y. Liu, H. Li, Y. Ma, and Z. Zhang, “Comparative transcriptome analysis of shortened fruit mutant in woodland strawberry (*Fragaria vesca*) using RNA-Seq,” *Journal of integrative agriculture/Journal of Integrative Agriculture*, vol. 16, no. 4, pp. 828–844, Apr. 2017, doi: [https://doi.org/10.1016/s2095-3119\(16\)61448-x](https://doi.org/10.1016/s2095-3119(16)61448-x).

[34] Cláudia Raposo de Magalhães, Kenneth Sandoval, Ferenc Kagan, Grace McCormack, Denise Schrama and Raquel Carrilho, “Transcriptomic changes behind *Sparus aurata* hepatic response to different aquaculture challenges: An RNA-seq study and multiomics integration,” *PloS one*, vol. 19, no. 3, pp. e0300472–e0300472, Mar. 2024, doi: <https://doi.org/10.1371/journal.pone.0300472>.

[35] M. Kamitani, A. J. Nagano, M. N. Honjo, and Hiroshi Kudoh, “A Survey on Plant Viruses in Natural Brassicaceae Communities Using RNA-Seq,” *Microbial ecology*, vol. 78, no. 1, pp. 113–121, Oct. 2018, doi: <https://doi.org/10.1007/s00248-018-1271-4>.

[36] Koichi Akashi, Hiromi Kumamaru, Yasuyuki Ohkawa, Hirokazu Saiwai, Hisakata Yamada, Kensuke Kubota, and Kazu Kobayakawa, “Direct isolation and RNA-seq reveal environment-dependent properties of engrafted neural stem/progenitor cells,” *Nature communications*, vol. 3, no. 1, Oct. 2012, doi: <https://doi.org/10.1038/ncomms2132>.

[37] X. Guan and E. Uberbacher, “Alignments of DNA and protein sequences containing frameshift errors,” *Computer applications in the biosciences : CABIOS*, Feb. 1996.



<https://www.semanticscholar.org/paper/Alignments-of-DNA-and-protein-sequences-containing-Guan-Uberbacher/98ec4f29b58f4d86ebcc58499e91d8abf2ee451b>.

[38] P. D. Vouzis and N. V. Sahinidis, "GPU-BLAST: using graphics processors to accelerate protein sequence alignment," *Bioinformatics*, vol. 27, no. 2, pp. 182–188, Nov. 2010, doi: 10.1093/bioinformatics/btq644.

[39] Amgen Scholars Program. (2024). BLOSUM62 Substitution Matrix. Labxchange.org. https://www.labxchange.org/library/items/lb:LabXchange:24d0ec21:lx_image:1

[40] National Library of Medicine. (2023). BLAST Scoring and Statistics. Nih.gov; U.S. National Library of Medicine. https://www.nlm.nih.gov/ncbi/workshops/2023-08_BLAST_evol/blast_score.html

[41] UniProt Consortium, "Proteome ID: UP000000803," UniProt. Accessed: Oct. 22, 2024. [Online]. Available: <https://www.uniprot.org/proteomes/UP000000803>

[42] EMBL. (2024). European Molecular Biology Laboratory. Embl.org. <https://www.embl.org/research/>

[43] Expasy - Translate tool. (2024). Expasy.org. <https://web.expasy.org/translate/>

[44] Bao, W., Jurka, M. G., Kapitonov, V. V., & Jurka, J. (2009). New Superfamilies of Eukaryotic DNA Transposons and Their Internal Divisions. *Molecular Biology and Evolution*, 26(5), 983–993. <https://doi.org/10.1093/molbev/msp01>

[45] Altschul, S., Madden, T. L., Schaffer, A. A., Zhang, J., Zhang, Z., Miller, W., & Lipman, D. J. (1997). Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic Acids Research*, 25(17), 3389–3402. <https://doi.org/10.1093/nar/25.17.3389>

[46] Wheeler, D., & Bhagwat, M. (2007). *Comparative Genomics*. Nih.gov; Humana Press. <https://www.ncbi.nlm.nih.gov/books/NBK1734/#:~:text=A%20common%20use%20of%20the,shifts%20in%20these%20coding%20regions>.